MINISTRY OF EDUCATION AND TRAINING
CAN THO UNIVERSITY
COLLEGE OF INFORMATION AND
COMMUNICATION TECHNOLOGY
DEPARTMENT OF SOFTWARE ENGINEERING

GRADUATION THESIS
BACHELOR OF ENGINEERING IN
INFORMATION TECHNOLOGY

(HIGH-QUALITY PROGRAM)

BUILDING A FASHION E-COMMERCE APPLICATION WITH IMAGE-BASED PRODUCT SEARCH AND MODEL BENCHMARKING

Student: Nguyen Thanh Phat
Student ID: B2005853
Class: DI20V7F1 (K46)
Advisor: Dr. Tran Cong An

Can Tho, December 2025

MINISTRY OF EDUCATION AND TRAINING
CAN THO UNIVERSITY
COLLEGE OF INFORMATION AND COMMUNICATION TECHNOLOGY
DEPARTMENT OF SOFTWARE ENGINEERING

GRADUATION THESIS
BACHELOR OF ENGINEERING IN INFORMATION TECHNOLOGY

(HIGH-QUALITY PROGRAM)

BUILDING A FASHION E-COMMERCE APPLICATION WITH IMAGE-BASED PRODUCT SEARCH AND MODEL BENCHMARKING

Student: Nguyen Thanh Phat
Student ID: B2005853
Class: DI20V7F1 (K46)
Advisor: Dr. Tran Cong An

Can Tho, 01/2026

EVALUATION OF ADVISOR

.....

.....

.....

.....

.....

.....

.....

Advisor:

Dr. Tran Cong An

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to Dr. Tran Cong An, my thesis advisor, for his guidance, continuous feedback, and support throughout this research.

I also thank the members of the Thesis Defense Committee for their time, thorough evaluation, and constructive comments.

My sincere appreciation goes to the Faculty of Information Technology, Can Tho University, for providing the academic foundation that supported this work, and to the High-Quality Program for fostering the engineering discipline applied throughout this thesis.

Finally, I am deeply grateful to my family for their constant encouragement, patience, and unwavering support throughout my studies and the completion of this work.

Sincerely,

Can Tho, December 2025

Nguyen Thanh Phat

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS

Abbreviation	Description
API	Application Programming Interface
ANN	Approximate Nearest Neighbor
CBIR	Content-Based Image Retrieval
CNN	Convolutional Neural Network
CQRS	Command Query Responsibility Segregation
DDD	Domain-Driven Design
DSR	Design Science Research
EF Core	Entity Framework Core
HNSW	Hierarchical Navigable Small World
JWT	JSON Web Token
mAP	Mean Average Precision
P@K	Precision at K
R@K	Recall at K
REST	Representational State Transfer
SPA	Single Page Application
ViT	Vision Transformer
VSA	Vertical Slice Architecture

ABSTRACT

E-commerce platforms rely primarily on text-based search, yet fashion products are inherently visual. Patterns, silhouettes, and textures resist keyword description. This thesis presents a fashion e-commerce platform with integrated Content-Based Image Retrieval (CBIR) that enables customers to search for products by uploading images rather than typing keywords. The system implements a modular architecture with a .NET backend, Vue.js frontend, and a Python machine learning sidecar for embedding generation.

A systematic benchmark evaluates four pre-trained deep learning models across both retrieval accuracy and operational efficiency on a curated fashion product dataset. Fashion-CLIP, a CLIP variant fine-tuned on over 700,000 fashion images, achieved the highest mean Average Precision at 0.8788 (SD 0.0022), outperforming the generic CLIP (mAP 0.8341, SD 0.0043) by 5.4%, EfficientNet-B0 (mAP 0.8158, SD 0.0007) by 7.7%, and ResNet-50 (mAP 0.8120, SD 0.0052) by 8.2%. At the opposite end of the speed spectrum, EfficientNet-B0 delivered inference at 23.9 ms per image (SD 2.5) while maintaining competitive retrieval quality, representing a 3.8x speed advantage over Fashion-CLIP (92.0 ms, SD 5.8) for a 7.7% accuracy trade-off.

Results demonstrate that domain-specific pre-training provides measurable advantages for visual fashion retrieval. The pluggable model architecture, switchable via a single environment variable, allows production deployments to select the optimal accuracy-speed profile for their infrastructure. The thesis provides a reference implementation for systematic comparison of embedding models in the fashion domain.

Keywords: e-commerce, visual search, deep learning, modular architecture, computer vision, benchmarking

TÓM TẮT

Các sàn thương mại điện tử phụ thuộc nhiều vào tìm kiếm văn bản, một cơ chế kém hiệu quả trong lĩnh vực thời trang, nơi hoa văn, chất liệu và kiểu dáng khó diễn đạt chính xác qua từ khóa. Hệ thống được xây dựng theo hướng module, bao gồm ba thành phần chính: backend .NET xử lý logic nghiệp vụ, frontend Vue.js phục vụ giao diện người dùng, và dịch vụ Python chuyên biệt cho tác vụ trích xuất đặc trưng hình ảnh từ các mô hình học sâu tiền huấn luyện.

Thực nghiệm được thiết lập trên bộ dữ liệu ảnh sản phẩm thời trang với quy trình đánh giá chéo ba lần (3-fold cross-validation), so sánh bốn mô hình embedding trên hai chiều: chất lượng truy xuất (đo bằng mAP, Precision at K, Recall at K) và hiệu năng vận hành (đo bằng độ trễ suy luận, thông lượng, và dung lượng lưu trữ). Mô hình Fashion-CLIP, được tinh chỉnh trên tập dữ liệu hơn 700.000 ảnh thời trang, đạt mAP cao nhất 0,8788 với độ lệch chuẩn 0,0022. CLIP gốc đạt mAP 0,8341 (SD 0,0043) và EfficientNet-B0 đạt 0,8158 (SD 0,0007). Ở chiều ngược lại, mô hình EfficientNet-B0 cho tốc độ suy luận nhanh nhất, 23,9 ms mỗi ảnh (SD 2,5), nhanh gấp 3,8 lần Fashion-CLIP (92,0 ms, SD 5,8) trong khi chỉ đánh đổi 7,7% về chất lượng truy xuất.

Kết quả thực nghiệm khẳng định giá trị của huấn luyện chuyên biệt theo lĩnh vực trong bài toán truy xuất hình ảnh thời trang, đồng thời cung cấp dữ liệu định lượng cho việc lựa chọn mô hình dựa trên ràng buộc hạ tầng thực tế. Kiến trúc mô hình hoán đổi linh hoạt, được điều khiển qua biến môi trường, cho phép hệ thống thích ứng với nhiều cấu hình triển khai khác nhau mà không cần thay đổi mã nguồn.

Từ khóa: thương mại điện tử, tìm kiếm hình ảnh, học sâu, kiến trúc module, thị giác máy tính, đánh giá hiệu năng

PART 1: INTRODUCTION

I. CONTEXT AND MOTIVATION

The global fashion e-commerce market exceeded 770 billion U.S. dollars in revenue during 2024, with projections surpassing one trillion by 2030 [1]. Despite this scale, the dominant interface, the text search bar, remains poorly suited to the domain. Fashion products are defined by attributes that resist keyword description: silhouette shape, fabric drape, print density, and colour relationships.

A well-documented barrier is the **semantic gap**: the discrepancy between the visual richness of a garment and a user's ability to express that richness in words. A customer may identify a desired aesthetic instantly in a photograph yet fail to produce query terms that retrieve matching items. When catalogue indexing uses inconsistent terminology (one vendor tags a pattern as "floral," another as "botanical," a third as "flower print"), relevant products are systematically excluded from search results. Industry data suggests that approximately 30 percent of online shoppers abandon a session after an unsuccessful search [2], a loss rate with direct commercial consequence.

> A customer may easily recognize a specific pattern, silhouette, or texture but struggle to articulate it using standardized metadata terms like "bohemian asymmetric maxi dress with botanical motifs."
>
> — The semantic gap in fashion search

Content-Based Image Retrieval (CBIR) addresses this gap by replacing textual intermediaries with direct visual comparison. Rather than indexing products by human-authored labels, CBIR systems encode images into dense vector representations (embeddings) and measure similarity through mathematical distance functions. A query image of a dress with a particular neckline and print pattern retrieves visually similar products without any keyword translation step. This capability has been advanced substantially by pre-trained convolutional neural networks [3], [4], vision transformers [5], and fashion-specific models trained on domain-relevant corpora [6].

The contribution of this thesis is therefore not algorithmic but architectural. It investigates how to embed existing CBIR capabilities into a practical e-commerce system built with conventional web technologies, and provides empirical data on which embedding models deliver the optimal balance of accu-

racy, latency, and resource efficiency within that setting. The work bridges two distinct software ecosystems: the Python machine learning stack and the .NET enterprise web stack, under real-time latency constraints appropriate for interactive search.

II. PROBLEM STATEMENT

Keyword-reliant search for fashion discovery suffers from inefficiencies that compound across four dimensions.

Linguistic inconsistency. Catalogue descriptors vary across vendors and brands. A single visual pattern appears under multiple labels (“floral,” “botanical,” “flower print”), fragmenting result sets across queries and forcing users to reformulate searches iteratively. This mismatch between a catalogue’s indexing vocabulary and a customer’s search vocabulary silently excludes relevant products from results.

Visual inexpressibility. Attributes such as fabric drape, texture gradient, silhouette proportion, and pattern rhythm cannot be reliably captured through text queries. A customer can identify a desired aesthetic instantly in a photograph yet cannot produce keywords that retrieve visually matching products. The user recognises what they want; the search engine does not.

Cold start data scarcity. Recommendation models based on collaborative filtering require historical user-item interaction data. Newly listed products, by definition, lack this data and are invisible to interaction-based recommenders at the point of highest commercial value: their initial release window. Visual feature extraction bypasses this limitation: product images are available from catalogue ingestion, and embeddings computed from those images enable similarity-based discovery without any interaction history.

Polyglot integration complexity. The Python deep learning ecosystem (PyTorch, HuggingFace, and the broader scientific Python stack) does not natively interoperate with the .NET transactional backend common in enterprise e-commerce. Integrating pre-trained vision models into an existing .NET application stack requires architectural design that isolates the machine learning workload from the main application while preserving sub-second response latency. This integration problem, bridging two software ecosystems with incompatible package managers, runtime environments, and deployment conventions, is a recurring engineering challenge in applied machine learning that the system architecture must directly address.

III. OBJECTIVES

The project builds a functional fashion e-commerce platform with integrated image-based search and evaluates the effectiveness of pre-trained deep learning

models within this system. The contribution is not a novel AI architecture but the engineering demonstration of embedding existing models into a conventional web application stack.

Technical Objectives

- Integrate pre-trained vision models into a PostgreSQL and .NET e-commerce stack, establishing a reference pattern for teams with existing web infrastructure.
- Architect a polyglot system in which a dedicated Python sidecar handles AI inference while the .NET backend manages transactional logic, business rules, and API routing.
- Validate pgvector (an open-source PostgreSQL extension) as the sole vector storage and retrieval layer, evaluating whether it meets real-time search latency requirements at catalogue scales representative of small-to-medium fashion retailers.
- Benchmark multiple embedding models spanning convolutional and transformer architectures on shared hardware, producing empirical guidance for model selection in resource-constrained deployments.

Research Questions

Three questions guide the investigation and are answered empirically in Chapter 3.

RQ1 addresses model comparison: how do fashion-specific embedding models compare with general-purpose CNN and ViT architectures on fashion product retrieval? This question tests whether domain-specific training on fashion data yields measurable improvements over models trained on generic image corpora.

RQ2 addresses the accuracy-speed trade-off: what trade-offs exist between retrieval accuracy and inference latency across pre-trained embedding models, and which model offers the best balance for real-time search? This question acknowledges that the most accurate model is rarely the fastest, and that deployment decisions require weighing both dimensions.

RQ3 addresses architecture viability: can a service-oriented architecture with a dedicated AI sidecar separate image inference from the main application while maintaining interactive response times? This question evaluates whether the chosen polyglot pattern (Python ML service alongside a .NET application) is practical for production use, not merely a laboratory demonstration.

Tasks Completed

- **Build an AI service.** A Python FastAPI service that loads multiple pre-trained embedding models, generates feature vectors from product images, and returns results within interactive latency bounds.
- **Set up vector search.** PostgreSQL configured with pgvector to store and index high-dimensional embeddings, with similarity queries validated for correctness and performance on a catalogue of representative size.
- **Connect the services.** A .NET backend layer that orchestrates image upload, embedding generation, vector database query, and result assembly into a single end-to-end search flow.
- **Create the user interface.** A Vue.js storefront with drag-and-drop image upload and a results grid displaying visually similar products with similarity scores.
- **Evaluate the results.** A systematic benchmark measuring retrieval accuracy via standard information retrieval metrics, comparing inference speed across models, and analyzing the operational trade-offs that inform production model selection.

IV. SCOPE AND LIMITATIONS

The thesis encompasses the design, implementation, and empirical evaluation of a fashion e-commerce platform with integrated visual search. Five areas define the included scope:

- Visual search from the storefront, with image upload via drag-and-drop or file selection.
- Product recommendations derived from embedding similarity scores.
- Core e-commerce functionality: product catalogue browsing, shopping cart, and simulated checkout.
- Multi-model comparison spanning several CNN and transformer architectures.
- End-to-end latency, throughput, and storage footprint measurement.

The following areas are explicitly excluded:

- Real payment processing (transactions are simulated for demonstration purposes).
- Shipping, logistics, and warehouse management workflows.
- User authentication via social login providers.
- Mobile application development (the system targets desktop and tablet web browsers).
- Custom model training or fine-tuning (all models are used as published).

Known Limitations

Four limitations define the boundaries of this work and are revisited in the concluding chapter.

- **Dataset size.** Evaluation uses 5,000 fashion product images from the Fashion Product Images dataset [7]. Controlled comparative benchmarking is feasible at this scale, but results may not extrapolate to production catalogues containing millions of items.
- **Hardware.** Experiments ran on consumer-grade hardware (Intel i7-1165G7, 16 GB RAM) with all inference executed on CPU. Reported latency and throughput figures are relative to this CPU-only profile. A dedicated inference server with GPU acceleration would likely improve both metrics.
- **Evaluation method.** Evaluation is exclusively quantitative: retrieval accuracy, inference latency, and throughput. Search output was reviewed qualitatively through visual inspection, but no formal user experience study was conducted. The relationship between measured accuracy metrics and subjective user satisfaction remains an open question.
- **Model training.** All embedding models were used as published by their original authors, without fine-tuning on the evaluation dataset. Domain-specific fine-tuning, particularly for models pre-trained on generic image corpora rather than fashion data, might improve retrieval quality but was beyond scope.

V. RESEARCH METHODOLOGY

This thesis follows a Design Science Research (DSR) methodology [8], [9], a problem-solving paradigm that produces and evaluates an IT artifact (here, a fashion e-commerce platform with integrated visual search) to address a defined problem domain.

The project progressed through four phases:

- **Research and planning.** Literature survey on visual search in fashion and e-commerce architectures; exploration of available pre-trained embedding models; evaluation of vector database options; technology stack selection.
- **Design.** Formalization of a modular .NET monolith architecture with a Python AI sidecar communicating via HTTP; database schema design supporting both relational and vector data within a single PostgreSQL instance.
- **Implementation.** Construction of three principal components: the .NET backend following vertical slice architecture, the Python embedding service with lazy model loading, and the Vue.js storefront.
- **Test and evaluation.** Systematic benchmark measuring retrieval accuracy through standard information retrieval metrics; latency and throughput measurement on consumer-grade hardware; qualitative review of search output.

This methodology suits the project's engineering focus: the primary output is not a theoretical contribution to machine learning research but a working system accompanied by empirical data on the performance trade-offs practitioners encounter when integrating academic models into production web stacks.

VI. THESIS OUTLINE

The thesis is organized into five chapters across three parts.

Part I: Introduction. Chapter 1 establishes research context, defines the problem, states objectives and research questions, delineates scope and limitations, describes the methodology, and provides the present outline.

Part II: Thesis Content contains three chapters:

- **Chapter 1: Background and Related Work.** Surveys vector embeddings, convolutional and transformer-based neural network architectures, vector database technologies including pgvector, prior academic and commercial work in fashion image retrieval, and the technology stack selected for this project.
- **Chapter 2: Design and Implementation.** Translates the problem into functional and non-functional requirements (Section 2.1), presents the system architecture including domain-driven design, C4 diagrams, database design, API design, and security model (Section 2.2), and describes the concrete implementation of the .NET backend, Python ML sidecar, and Vue.js storefront (Section 2.3).
- **Chapter 3: Testing and Evaluation.** Reports a systematic benchmark comparing retrieval accuracy and inference efficiency across multiple embedding models using a cross-validation protocol on 5,000 fashion product images, and analyzes the accuracy-speed trade-offs that inform model selection.

Part III: Conclusion. Chapter 4 synthesizes findings against each research objective, evaluates contributions and limitations, and proposes directions for future work.

PART 2: THESIS CONTENT

1 BACKGROUND AND RELATED WORK

This chapter is organised into six sections. Fashion E-commerce establishes the market context and problem domain that motivates the thesis. Content-Based Image Retrieval introduces the core concepts and mathematical foundations of visual search. Machine Learning Models surveys the neural architectures used to generate image embeddings. Vector Databases covers the storage and indexing technologies that enable efficient similarity search. Software Engineering Technologies describes the architectural patterns and specific technologies composing the ReSys.Shop platform. Related Work and Research Gap positions this thesis within the broader landscape of academic research and commercial visual search systems.

1.1 FASHION E-COMMERCE

Global fashion e-commerce exceeded 770 billion U.S. dollars in 2024, with projections surpassing one trillion by 2030 [1]. Yet the primary interface between customers and catalogues, the text search bar, struggles with the defining attributes of fashion products: silhouette, drape, pattern rhythm, and colour relationships resist keyword description. The semantic gap between visual richness and linguistic expression drives search abandonment rates of approximately 30 percent [2], a loss with direct commercial consequence. This section establishes the market context and problem domain that motivates the thesis.

1.2 CONTENT-BASED IMAGE RETRIEVAL

Content-Based Image Retrieval replaces text queries with image queries. Rather than indexing products by human-authored labels, CBIR systems encode images into dense vector representations and measure similarity through mathematical distance functions. This section introduces the core concepts and mathematical foundations of CBIR as applied to fashion product search.

1.2.1 Visual Search Concepts

At the heart of visual search is a simple idea: turning images into lists of numbers that a computer can compare. These lists are called **vector embeddings**, sometimes **feature vectors**. When an AI model processes an image, it outputs a fixed-length sequence of numbers representing the visual content. Visually

similar products produce similar number sequences; dissimilar products produce different ones.

Content-Based Image Retrieval (CBIR) replaces text queries with image queries. Rather than requiring users to describe what they want in words, CBIR lets them search by example: upload a photo of a garment, and the system retrieves visually similar products from the catalog. This approach bypasses the need for consistent, complete textual labels. A photograph of a dress with a distinctive neckline retrieves visually similar products regardless of how the catalog describes them. The embedding becomes a universal description that captures shape, texture, colour, and pattern automatically.

1.2.2 The Semantic Gap

For fashion e-commerce, visual search addresses a fundamental limitation of text-based product discovery. Shoppers often know what they want visually but struggle to articulate it in search terms. A customer who sees a dress they like can search with the image itself, discovering products that share its visual characteristics without needing to name the style, cut, or pattern.

The gap between visual richness and linguistic expression, known as the semantic gap, is the central challenge in image retrieval. Global fashion e-commerce exceeded 770 billion U.S. dollars in 2024 [1], yet search abandonment rates hover around 30 percent when customers cannot find products through text queries [2]. CBIR bridges this gap by operating directly on visual content rather than on human-authored labels.

1.2.3 Mathematical Foundations of Embeddings

When an AI model processes an image, it outputs a fixed-length sequence of numbers representing the visual content:

```
[0.23, -0.15, 0.87, 0.42, ..., -0.31]  (512 numbers)
```

These lists are called **vector embeddings**. Visually similar products produce similar number sequences; dissimilar products produce different ones. This transforms visual comparison into a mathematical operation.

1.2.3.1 The Latent Space

Embeddings can be understood as points in a high-dimensional space. A 512-dimensional vector occupies a space with 512 axes, far beyond the three spatial dimensions we can visualise. In this **latent space**:

- Similar images cluster close together
- Different images are far apart

- The word "latent" signifies that the dimensions do not correspond to obvious visual concepts like "redness" or "stripiness." They represent abstract features the model learned during training.

1.2.3.2 Measuring Similarity: Cosine Distance

To compare images, the system measures the angle between their embedding vectors using **cosine similarity**:

$$\text{cosine similarity} = \frac{A \cdot B}{\|A\| \times \|B\|}$$

Where A and B are the embedding vectors being compared, $A \cdot B$ is their dot product, and $\|A\|$ and $\|B\|$ are their Euclidean norms.

Cosine similarity ranges from +1.0 (vectors point in the same direction, very similar) to 0.0 (perpendicular, unrelated) to −1.0 (opposite directions, very dissimilar). For fashion images, values above 0.7 typically indicate visual similarity a user would recognise. The key advantage is that the same mathematical operation works for any image, any category, without the system needing to know what makes a "dress" or a "shoe."

1.2.3.3 The CBIR Pipeline

Content-Based Image Retrieval (CBIR) replaces text queries with image queries through the following pipeline:

1. The user uploads or selects a query image
2. The system generates an embedding for that image using a deep learning model
3. The system compares the query embedding against all product embeddings in the database using cosine distance
4. Results are ranked by similarity and displayed to the user

This approach bypasses the need for consistent, complete textual labels. A photograph of a dress with a distinctive neckline retrieves visually similar products regardless of how the catalog describes them. The embedding becomes a universal description that captures shape, texture, colour, and pattern automatically.

1.3 MACHINE LEARNING MODELS

Generating useful embeddings requires models that extract visual features at multiple levels of abstraction. Over the past decade, three families of architectures have emerged: convolutional neural networks, vision transformers, and multimodal CLIP-based models. This section surveys each family, describes the specific variants evaluated in this thesis, and identifies their trade-offs for fashion retrieval.

1.3.1 Convolutional Neural Networks

Convolutional neural networks (CNNs) process images through a hierarchy of learned filters. Early layers detect simple patterns (edges, colour transitions, texture directions), middle layers compose these into shapes and parts (lapels, button rows, sleeve boundaries), and deep layers recognise complete structures (dress vs. jacket, formal vs. casual). This layered organisation mirrors aspects of biological vision and has proven remarkably effective for visual recognition [3].

ResNet (Residual Network) introduced skip connections that allow information to bypass layers, solving the degradation problem that plagued deeper networks. ResNet-50, the 50-layer variant used in this thesis, remains a strong baseline for image retrieval with 25.6 million parameters and 2,048-dimensional embeddings.

EfficientNet uses compound scaling to simultaneously balance network depth, width, and input resolution [4]. EfficientNet-B0, the smallest variant evaluated here, achieves competitive accuracy with 5.3 million parameters and produces 1,280-dimensional embeddings. Its compact design makes it well-suited to CPU-only or memory-constrained deployments.

Table 1.

Table 1: CNN-based models evaluated in this thesis

Family	Model	Parameters	Embedding Dim	Training Data
CNN	ResNet-50	25.6M	2048	ImageNet (1.2M images)
CNN	ResNet-101	44.5M	2048	ImageNet (1.2M images)
CNN	EfficientNet-B0	5.3M	1280	ImageNet (1.2M images)
CNN	EfficientNet-B4	19.3M	1792	ImageNet (1.2M images)

1.3.2 Vision Transformers

While CNNs capture local patterns through their layered filter design, their reliance on small receptive fields (typically 3 by 3 pixel windows) limits their ability to model relationships between distant image regions. Vision Transformers (ViTs) address this by applying the **self-attention** mechanism that was originally developed for natural language processing to image data [10].

A ViT divides an image into a grid of fixed-size patches (typically 16 by 16 pixels), treats each patch as a “token” analogous to a word in a sentence, and passes the sequence through transformer layers. Self-attention computes

pairwise relationships between all patches simultaneously. For fashion, this means a ViT can relate a collar detail in one corner to a hemline pattern at the opposite edge without needing many intermediate layers, a capability especially valuable for garment retrieval where global silhouette matters as much as local texture.

DINOv2, developed by Meta AI, takes a different approach: rather than training on human-labelled images, it uses self-supervised learning on a large, uncurated collection of images [11]. The model learns visual features by solving a prediction task on its own representations, without ever seeing a category label. DINOv2 produces 384-dimensional embeddings (ViT-S variant) or 768-dimensional embeddings (ViT-B variant). Its self-supervised training makes it adaptable to domains where curated labels are scarce.

Table 2.

Table 2: Vision Transformer models evaluated in this thesis

Family	Model	Parameters	Embedding Dim	Training Method
ViT	DINOv2 ViT-S/14	21M	384	Self-supervised (142M images)
ViT	DINOv2 ViT-B/14	86M	768	Self-supervised (142M images)
ViT	CLIP ViT-B/32	151M	512	Contrastive (400M pairs)
ViT	CLIP ViT-B/16	150M	512	Contrastive (400M pairs)
ViT	CLIP ViT-L/14	428M	768	Contrastive (400M pairs)

1.3.3 CLIP and Fashion-CLIP: Bridging Vision and Language

CNNs and ViTs operate purely in the visual domain, mapping images to embedding spaces that have no connection to human language. **CLIP** (Contrastive Language-Image Pre-training) bridges this gap through a dual-tower architecture: one tower encodes images, a parallel text encoder processes natural language descriptions, and both are trained jointly on 400 million (image, caption) pairs from the public web [5]. A contrastive objective pulls matching image-text pairs together in a shared embedding space while pushing non-matching pairs apart. The result is a model that can both see and read.

Fashion-CLIP extends CLIP by fine-tuning it on over 700,000 fashion images paired with domain-specific text descriptions [6]. The fine-tuning adjusts

model weights to emphasise fashion-relevant attributes (garment categories, fabric textures, style descriptors, occasion labels) while retaining general visual understanding. Fashion-CLIP uses the ViT-B/16 architecture inherited from CLIP, producing 512-dimensional embeddings. The original paper reports a 15 to 20% improvement on fashion retrieval over general CLIP, a result confirmed in the benchmark evaluation presented in Chapter 3 of this thesis.

The dual-tower design also enables **multimodal queries** unavailable in pure vision models like DINOv2 or EfficientNet. A user searching for "red floral summer dress" does not need a reference image; the text encoder maps the description directly into the same embedding space as catalog images. A hybrid query combining an uploaded photo with a textual refinement, "like this, but in blue," becomes possible by encoding both modalities and merging the results.

Table 3.

Table 3: CLIP variants evaluated in this thesis

Model	Architecture	Training	Do main
CLIP ViT-B/32	Dual-tower (ViT + text transformer)	Con- trastive (400M image- text pairs)	General
CLIP ViT-B/16	Dual-tower (ViT + text transformer)	Con- trastive (400M image- text pairs)	General
CLIP ViT-L/14	Dual-tower (ViT + text transformer)	Con- trastive (400M image- text pairs)	General
Fashion-CLIP	Dual-tower (ViT + text transformer)	Con- trastive, fine- tuned on 700K fashion images	Fashion-specific

1.4 VECTOR DATABASES

Once images are converted to embeddings, those vectors must be stored and searched efficiently. For catalog-scale data, brute-force distance computation is impractical. This section introduces approximate nearest neighbour search, two index algorithms (HNSW and IVFFlat), and the pgvector extension that provides vector search within a standard PostgreSQL database.

1.4.1 Approximate Nearest Neighbour Search

Once images are converted to embeddings, those vectors must be stored and queried efficiently. For a catalog of n products, a naive brute-force search requires n distance computations per query. At 10,000 items this is manageable on modern hardware; at millions it becomes impractical.

The solution is **Approximate Nearest Neighbour** (ANN) search. Rather than exhaustively computing the distance to every stored vector, ANN algorithms use index structures that organise vectors into navigable graphs or trees. A query navigates directly toward the neighbourhood of likely matches, skipping the vast majority of irrelevant vectors. The accuracy trade-off is modest: typically 97 to 99% recall of the true top matches, for a speed improvement of several orders of magnitude. For product search, where returning 20 visually similar items matters far more than guaranteeing the absolute 21st best match, this trade-off is entirely acceptable.

1.4.2 HNSW: Hierarchical Navigable Small World

- **Theory.** HNSW constructs a multi-layered navigable graph over the embedding space. Top layers are sparse, connecting distant regions and enabling long-range jumps; bottom layers are dense and refine the search locally. A query enters at the top layer, greedily traverses edges toward the nearest neighbour, then descends to the next layer and repeats. The process converges on the query neighbourhood with logarithmic time complexity $O(\log n)$.
- **Hyperparameters.** Three: `M` (maximum neighbours per node), `ef_construction` (search breadth during build), and `ef_search` (search breadth during query). These require coordinated tuning — increasing `M` improves recall but slows build and query; increasing `ef_search` improves recall linearly at logarithmic cost.
- **Build cost.** Graph construction is computationally expensive. At moderate catalogue scales (10^4–10^5 vectors), build time is measured in minutes rather than seconds.
- **Recall.** High. HNSW consistently exceeds 95 percent recall@10 at query latencies under 10 ms, sustained across catalogue scales up to 10^7 vectors.

- **Application.** HNSW is the preferred index for production-scale ANN search in this project. Its logarithmic query cost and sustained recall make it suitable for interactive fashion retrieval at millions of catalogue items, where sub-100 ms latency is required.

1.4.3 IVFFlat: Inverted File with Flat Compression

- **Theory.** IVFFlat partitions the vector space into k clusters via k-means at index-build time. Each vector is assigned to its nearest centroid; the resulting clusters (lists'‘) are stored as inverted files mapping centroids to member vectors. A query computes distance to all k centroids, selects the nearest `probes` clusters, and exhaustively searches within those selected lists only. Search cost scales as $O\big(k + \frac{np}{k}\big)$ where p is the number of probes.
- **Hyperparameters.** Two: `lists` (k, number of clusters, typically $4\sqrt{n}$ for n vectors) and `probes` (number of lists searched per query). The recall-speed trade-off is controlled by `probes`: increasing `probes` improves recall at linear cost to query latency.
- **Build cost.** Low. Clustering is a single pass over the data; for 5,000 vectors and 100 lists, build time is under one second. Memory overhead is minimal beyond the vectors themselves (centroid storage plus inverted list pointers).
- **Recall.** Moderate. IVFFlat achieves 65–72 percent recall@10 at sub-10 ms latency for 5,000 vectors. Recall depends on the `lists:probes` ratio; at fixed `probes`, recall degrades as n grows because each list contains more candidates.
- **Application.** IVFFlat suits small-to-intermediate scales and evaluation scenarios where build speed and hyperparameter simplicity are valued over peak recall. This project uses IVFFlat for model comparison benchmarks (Chapter 3), where the objective is ranking embedding models rather than optimising index performance.

1.4.4 pgvector: PostgreSQL with Vector Search

pgvector is an open-source PostgreSQL extension that implements vector storage and similarity search within the standard relational database. This project uses pgvector 0.8+.

- **Transactional consistency.** Vectors and product metadata live in the same database. A product update and its embedding update occur within a single ACID transaction, eliminating the dual-database problem where a separate vector store drifts out of sync with the relational source of truth.
- **Combined SQL queries.** Vector similarity and relational filtering combine in a single query plan: find products visually similar to a query image, restricted to a specific category and price range, using a single SQL statement.

- **Index support.** pgvector 0.8+ supports both IVFFlat (Inverted File with Flat Compression) and HNSW (Hierarchical Navigable Small World). This project uses each index type for a distinct purpose.

Index assignment in this project. The benchmark evaluation (Chapter 3) uses IVFFlat with 100 lists at the 5,000-vector catalogue scale. The production architecture designates HNSW as the long-term index target. The project migration strategy progresses through three phases:

1. **Exact search** (current state). At the current catalogue scale, the pgvector `<=>` operator without an index provides adequate query latency.
2. **IVFFlat** (intermediate scale). As the catalogue grows beyond 10,000 items, IVFFlat with tuned `lists` and `probes` provides approximate search with minimal build overhead.
3. **HNSW** (production scale). At millions of catalogue items, HNSW's superior recall-speed trade-off and sustained sub-100 ms latency become decisive.

- **Variable-length vectors.** pgvector accommodates the different embedding dimensions produced by different models: 384 (DINOv2-S), 512 (FashionCLIP), 768 (DINOv2-B), 1280 (EfficientNet-B0), and 2048 (ResNet-50).
- **Distance metric.** Cosine distance, using the `<=>` operator. Cosine is bounded in $[0, 2]$ and produces interpretable similarity thresholds for fashion retrieval.

Table 4.

Table 4: pgvector configuration: benchmark evaluation and production target

Property	**Benchmark Value**	**Production Target**	**Rationale**
Extension	pgvector 0.8+	pgvector 0.8+	Open-source, zero additional infrastructure
Index type	IVFFlat (100 lists)	HNSW	IVFFlat for fast build and simple tuning during evaluation; HNSW for optimal recall at scale
Distance metric	Cosine	Cosine	Bounded range, interpretable thresholds for fashion similarity
Model metadata	model_name, model_version columns	model_name, model_version columns	Enables per-model filtering and A/B testing

1.5 SOFTWARE ENGINEERING TECHNOLOGIES

Building a production-capable e-commerce platform with integrated machine learning requires deliberate architectural and technology choices. This section

surveys architectural patterns, describes each technology in the ReSys.Shop stack, and provides the rationale for each selection.

1.5.1 E-commerce Platform Architectures

Having covered how embeddings are generated and stored, this section examines the architectural patterns that organise the ReSys.Shop platform around these capabilities. Modern web applications are shaped by three architectural patterns, each trading off simplicity, scalability, and operational cost.

1.5.1.1 Monolith

A monolith packages the user interface, business logic, and data access into a single deployable unit. Development is straightforward: one codebase, one build pipeline, one deployment. At small scale this works well. As the system grows, subsystems accumulate coupling. Changing the checkout flow requires understanding the catalog module; deploying a payment fix means redeploying the entire application. The monolith does not scale with team size or codebase age.

1.5.1.2 Microservices

Microservices decompose an application into independently deployable services, each owning a discrete business capability. Teams can work in parallel using different technology stacks per service. The trade-off is operational complexity: service discovery, inter-service authentication, network latency, partial failure modes, and distributed transaction management. For a system where the primary research contribution lies in machine learning integration rather than infrastructure engineering, this overhead is disproportionate.

1.5.1.3 Modular Monolith

The modular monolith occupies the middle ground. Code is organised into logically isolated business modules within a single process. Compile-time boundaries prevent direct cross-module references, preserving the logical independence of bounded contexts. There is one build, one deployment, and one shared database. The nine business modules in ReSys.Shop (Catalog, Ordering, Payment, Inventory, Identity, Profile, Shipping, Location, Dashboard) communicate through an in-process message bus with no namespace-level dependencies between them. A shared PostgreSQL instance allows relational product data and vector embeddings to coexist within the same transactional boundary, maintaining consistency between catalog updates and search index changes.

The machine learning capability is the one exception to the single-process rule. It runs as a dedicated Python sidecar service, isolated because PyTorch and the broader Python scientific stack have incompatible runtime requirements with .NET. The sidecar communicates with the main application over HTTP,

exposing a narrow embedding-generation interface while keeping GPU resource contention isolated from the e-commerce API.

1.5.1.4 Architectural Decision

ReSys.Shop adopts the modular monolith with a machine learning sidecar. The decision is guided by three trade-offs:

- **Deployment.** One process for the core application avoids service discovery, inter-service authentication, and distributed transaction orchestration. The Python sidecar runs as a separate process because PyTorch and .NET have incompatible runtime environments, but the sidecar exposes a narrow HTTP interface restricted to embedding generation. A GPU failure in the sidecar does not affect e-commerce API availability.
- **Data consistency.** A single PostgreSQL instance hosts both relational product data and pgvector embeddings. Catalog updates and embedding index changes share the same transactional boundary, eliminating the class of stale-index bugs that arise when a vector store and relational database drift out of sync.
- **Module boundaries.** Nine business modules (Catalog, Ordering, Payment, Inventory, Identity, Profile, Shipping, Location, Dashboard) are isolated by namespace convention within one assembly. Inter-module communication uses an in-process message bus. There are no direct cross-module references at compile time, preserving bounded-context independence without the operational cost of separate deployment units.

1.5.2 .NET Backend

The backend is built on .NET 10, a high-performance runtime with ahead-of-time compilation and native asynchronous I/O. Its architecture is organised around five core libraries:

- **Carter** extends ASP.NET minimal APIs with module-based endpoint registration. Each business module declares its own routes independently, keeping endpoint definitions co-located with their handlers rather than centralised in a startup file.
- **MediatR** implements CQRS, routing commands (writes) and queries (reads) to handlers through an in-process message bus. Handlers are discovered at startup and dispatched by request type, with no direct coupling between modules.
- **Entity Framework Core** maps C# domain objects to PostgreSQL tables, including pgvector column types for embedding storage. Migrations are version-controlled and applied at startup.

- **FluentValidation** enforces input rules at the application boundary. Each request type has an associated validator that runs before the handler executes, rejecting invalid data before it reaches business logic.
- **Vertical slice architecture** organises each feature (e.g., CreateProduct, SearchByImage) as a self-contained folder containing the handler, request, response, endpoint, and validator. This colocation keeps feature concerns together rather than spread across technical layers.

1.5.3 Vue.js Frontend

The frontend uses Vue 3 with TypeScript and the Vite build tool. Two surfaces share a common component library and state management:

- **Storefront.** Product catalog browsing with category trees, faceted filters (price, size, colour), and paginated result grids. Visual search with image upload, similarity-ranked results displaying thumbnail, price, and similarity score. Cart management with session-based guest support and a multi-step checkout flow spanning address entry, delivery selection, payment confirmation, and order completion.
- **Administration interface.** Complete lifecycle management for products, variants, taxonomies, and option types. Order processing with fulfilment status tracking. User and role administration with permission assignment. Analytics overview summarising sales and inventory metrics.
- **Pinia**, a state management library, organises client-side state through typed stores. Each bounded context has its own store (catalog, cart, auth, orders), mirroring the backend module boundaries.

1.5.4 PostgreSQL and pgvector

PostgreSQL 17 hosts both relational business data and vector embeddings in a single database.

- **Relational schema.** Tables for products, variants, orders, users, and supporting entities are organised by bounded context. Foreign keys reference entities within the same context; cross-context references use identifier columns without database-level constraints, preserving logical isolation.
- **pgvector extension.** Adds a `vector(N)` column type and cosine distance operator (`<=>`). An HNSW index on the embedding column enables sub-10ms ANN search on catalog-scale datasets.
- **Transactional consistency.** A product update and its embedding update share the same ACID boundary. New images trigger embedding generation; catalog

modifications and index changes are committed together, eliminating stale-index drift.

- **Performance.** Composite indexes on query-critical combinations (user status, session status, product slug) optimise frequent access patterns. Query plans combine vector similarity and relational filtering in a single execution.

1.5.5 Redis Caching

Redis 7 operates alongside the .NET **HybridCache** abstraction in a two-tier arrangement.

- **L1: in-process.** Frequently accessed data (taxonomy trees, front-page product lists) is held in application memory with sub-millisecond read latency. Cache entries expire on a configurable sliding window, typically five minutes.
- **L2: Redis.** The shared tier synchronises cache across application instances. Redis also backs Hangfire job queues and guest session storage. On cache miss at L1, the value is retrieved from Redis and promoted to L1, ensuring subsequent hits stay in-process.

1.5.6 Python ML Sidecar

The machine learning capability runs as a dedicated Python 3.12 service, isolated from the .NET backend due to incompatible runtime dependencies (PyTorch requires Python, .NET requires the CLR).

- **Framework.** FastAPI provides async HTTP endpoints with automatic OpenAPI schema generation. Uvicorn serves as the ASGI runtime.
- **Model management.** A singleton **ModelManager** lazy-loads models from the HuggingFace hub on first request. Once loaded, models persist in GPU memory (or CPU, if no GPU is available) for the lifetime of the service. The manager supports multiple architectures through a common embedding interface.
- **API surface.** `POST /embeddings` accepts raw image bytes (JPEG, PNG, WebP) and returns a JSON array of floating-point values. `GET /health` reports the currently loaded model, its embedding dimension, and the most recent inference latency.
- **Security.** Requests require an `X-API-Key` header validated at the middleware layer. The sidecar listens only on the internal Docker network, inaccessible from the public internet.

1.5.7 .NET Aspire Orchestration

.NET Aspire coordinates the multi-container development and deployment environment.

- **Service discovery.** Components resolve each other by name: the .NET backend reaches the Python sidecar at `http://ml-service`, PostgreSQL at `postgres`, and Redis at `redis`. Configuration is injected via environment variables managed by the Aspire host.
- **Lifecycle.** Aspire enforces startup ordering (database before API, sidecar before backend health check) and gates each component behind a readiness probe. Containers restart on failure with configurable back-off.
- **Observability.** OpenTelemetry SDKs in both .NET and Python services export distributed traces, request metrics, and structured logs to the Aspire dashboard. Correlation IDs propagate across the HTTP boundary between backend and sidecar, linking a search request to its embedding generation span.

1.5.8 Hangfire Background Jobs

Hangfire processes operations that should not block HTTP requests. Jobs are persisted in Redis for resilience across application restarts.

- **Cart expiry.** A recurring job runs daily, removing carts with no activity for seven days. This prevents abandoned carts from accumulating indefinitely.
- **Embedding queue.** When an admin uploads new product images, embedding generation tasks are enqueued for asynchronous processing. The upload endpoint returns immediately; the embedding appears in search results once the job completes.
- **Index maintenance.** A periodic job rebuilds the HNSW index on the embedding column, optimising search performance as the catalog grows.

1.5.9 Identity, Authentication, and Authorisation

- **ASP.NET Identity** provides user account management: password hashing with salted PBKDF2, email confirmation workflows, and optional two-factor authentication via TOTP.
- **JWT tokens.** Access tokens carry claims (user ID, roles, permissions) and expire after 15 minutes. Refresh tokens enable silent renewal: the client exchanges a refresh token for a new access token, and each refresh token is single-use. Reuse of an already-consumed refresh token triggers revocation of all tokens for that user, mitigating token theft.

- **Guest sessions.** Anonymous users receive a signed session cookie. This cookie links to a server-side session backed by Redis, enabling cart operations and product browsing without authentication. On registration or login, the guest session is transferred to the authenticated user context.
- **Permission model.** Authorisation uses granular claims in the format `domain:category:action` (e.g., `catalog:products:create`). Roles aggregate commonly used permission sets. Endpoint-level attributes evaluate claims at the middleware layer, so handler code contains no authorisation logic.

1.5.10 Benchmark Framework

The benchmark framework is a Python 3.12 pipeline for systematic, reproducible evaluation of embedding models on fashion product retrieval. It is separate from the application codebase and produces the experimental results reported in Chapter 3.

- **Modes.** Three evaluation modes are supported: a one-shot comparison across all configured models, a three-fold cross-validation protocol with stratified category-based splits (the default for thesis results), and a pgvector pipeline mode that measures end-to-end latency including database query and network round-trip time.
- **Models.** 11 pre-trained architectures are implemented: CNN-based (ResNet-50, ResNet-101, ResNet-152, EfficientNet-B0, EfficientNet-B4), vision transformers (DINOv2 ViT-S/14, DINOv2 ViT-B/14), and CLIP variants (CLIP ViT-B/32, CLIP ViT-B/16, CLIP ViT-L/14, Fashion-CLIP). Each model has a thin adapter implementing a common `generate_embeddings` interface.
- **Caching.** Embeddings are cached to disk per model, per fold, and per split (query vs. catalog), avoiding recomputation when re-running evaluation on the same configuration.
- **Metrics.** Retrieval accuracy is measured through mAP, Precision at K, Recall at K, and nDCG. Operational efficiency is measured through inference latency (mean and SD per image), throughput (images per second), model load time, on-disk storage, and RAM usage.
- **Outputs.** Results are exported in JSON, CSV, Markdown, and Typst table formats. The Typst output files (`thesis_aggregate.typ`, `thesis_efficiency.typ`) embed directly into the thesis without manual transcription.
- **Multi-label pipeline.** A separate enriched-dataset mode supports evaluation across three label schemes of increasing granularity (category only, category

and colour, and category, colour and pattern), enabling analysis of how embedding quality varies with annotation detail.

1.5.11 Technology Stack Summary

The preceding sections introduced the principal technologies that compose the ReSys.Shop platform. Table Table 5 consolidates the complete stack.

Table 5.

Table 5: Technology stack of the ReSys.Shop platform

Layer	**Technology**	**Role**
Frontend	Vue 3, TypeScript, Vite	Customer storefront and administration interface; reactive UI with Pinia state management
Backend API	.NET 10, Carter, MediatR	REST endpoints via minimal APIs; CQRS command-query separation across business modules
Database	PostgreSQL, pgvector	Relational data and vector embeddings in a single ACID database with HNSW-indexed similarity search
Caching	Redis, HybridCache	Two-tier cache (in-memory L1 and Redis L2); Hangfire job queue and session state backing store
ML Sidecar	Python 3.12, FastAPI, PyTorch	Dedicated embedding generation service with lazy model loading and GPU acceleration
Orchestration	.NET Aspire	Container lifecycle management, service discovery, and reproducible local development environment
Background Jobs	Hangfire	Persistent job processing for cart expiry, embedding queue, and maintenance tasks
Auth and Identity	JWT, ASP.NET Identity	Access tokens, refresh rotation, permission-based authorisation, guest sessions
Benchmarking	Python 3.12, PyTorch	Systematic 11-model comparison with cross-validation, accuracy and efficiency metrics

Together these technologies form a polyglot stack spanning three languages (C#, TypeScript, Python) orchestrated through a unified containerised environment. The .NET backend hosts transactional e-commerce logic; Vue 3 delivers the customer-facing interface; PostgreSQL serves as the single source of truth for both business data and embeddings; and the Python sidecar provides the bridge to GPU-accelerated model inference. A dedicated benchmarking framework, separate from the application codebase, provides the systematic evaluation infrastructure that produces the experimental results in Chapter 3.

1.6 RELATED WORK AND RESEARCH GAP

This section positions the ReSys.Shop platform within the landscape of fashion image retrieval research and commercial visual search systems, and identifies the engineering gap that this thesis addresses.

1.6.1 Academic Research

The **DeepFashion** dataset, introduced by Liu et al., established the foundational benchmark for fashion recognition and retrieval with over 800,000 images annotated with attributes, landmarks, and in-shop-to-consumer photo pairs [12]. This dataset catalysed much of the subsequent work in fashion AI.

FashionIQ extended retrieval to the conversational setting, where users modify queries through natural language feedback (“like this dress but shorter”) [13]. While compelling, the interactive dialogue paradigm requires infrastructure beyond the scope of this project, which focuses on single-turn visual and text queries.

The **Fashion-CLIP** work demonstrated that domain-specific fine-tuning of CLIP on 700,000 fashion images improves retrieval by 15 to 20% over the general model [6]. This thesis follows that approach, using pre-trained models without custom training, and extends the evaluation to additional architectures (ResNet, EfficientNet, DINOv2) for systematic comparison.

1.6.2 Commercial Systems

Several platforms have deployed visual search at production scale.

Table 6.

Table 6: Comparison of commercial visual search products

Product	Key Strength	Limitation

Google Lens	Massive scale, general-domain coverage	Closed ecosystem, not customisable
Pinterest Lens	Over 600M monthly searches, style-aware	Proprietary, requires Pinterest integration
ASOS Style Match	Fashion-specific accuracy	Restricted to ASOS catalog only
ViSenze	API-based, good accuracy	Paid service with recurring per-query costs

These products share common limitations for independent projects: they are proprietary and cannot be studied or modified, API access incurs costs at query volume, and reliance on external services creates vendor lock-in. This thesis demonstrates that comparable functionality is achievable with open-source tools, providing both a reference implementation and a cost-effective alternative for smaller deployments.

1.6.3 Contribution Differentiators

This project distinguishes itself from prior work by addressing the **engineering gap** between model research and production systems. Four contributions define this gap:

1. Polyglot architecture. Python's machine learning ecosystem (PyTorch, HuggingFace) does not natively interoperate with the .NET stack common in enterprise e-commerce. This thesis presents a modular monolith with a dedicated AI sidecar, combining .NET's type safety and transactional integrity with Python's access to state-of-the-art vision models, without the operational overhead of a full microservices deployment.

2. Vector-native consistency. By using pgvector within PostgreSQL, embeddings and product metadata share the same transactional boundary. Product updates, image replacements, and index maintenance occur atomically, eliminating stale-index bugs that arise when a vector store and relational database have independent consistency guarantees.

3. Commodity hardware benchmarking. Commercial visual search runs on cloud TPU clusters. This thesis evaluates 11 models on consumer-grade hardware, establishing that production-quality visual search is achievable without specialised infrastructure, lowering the barrier for small to medium e-commerce platforms.

4. Applied model comparison. Rather than chasing leaderboard metrics, this thesis compares models within realistic deployment constraints (inference latency budget, memory limits, storage cost). The resulting accuracy-efficiency

trade-off data, presented in Chapter 3, provides a pragmatic guide for practitioners selecting embedding models.

2 DESIGN AND IMPLEMENTATION

This chapter consolidates the system design. Section 2.1 defines the functional and non-functional requirements. Section 2.2 presents three representative use cases spanning the core research capability, the primary e-commerce workflow, and the evaluation infrastructure. Section 2.3 presents the system architecture, domain model, database schema, API design, and security model. Section 2.4 describes the implementation of the .NET backend, Python ML sidecar, and Vue.js storefront.

2.1 REQUIREMENTS ANALYSIS

This section defines the scope of the ReSys.Shop platform by identifying its actors, functional and non-functional requirements, and the classification of features into research contributions and supporting infrastructure. The analysis establishes what the system must do before proceeding to its architectural design and implementation. Use cases are treated separately in Section 2.2.

2.1.1 System Actors

The platform serves three categories of actors, defined by their access level, responsibilities, and interaction surface.

2.1.1.1 Customer: Product Discovery and Purchase

The **Customer** is the primary beneficiary of the research contribution. Customers interact through a browser-based **storefront** (Vue 3 SPA) and their capabilities span the full shopping workflow.

Product Discovery. Browse the catalog with faceted filters and hierarchical category navigation. Perform **keyword searches** by product name or attribute. Perform **visual searches** by uploading a reference image; the system returns visually similar products ranked by embedding similarity score.

Cart and Checkout. Add products to a persistent cart (guest or authenticated). Complete a **multi-step checkout** spanning address selection, delivery method choice, payment confirmation, and order finalisation. Cart contents survive page navigation and browser restarts.

Account. Register with email and password. Log in with credentials or Google OAuth. Manage profile details, shipping addresses, and wishlists. View order history and track fulfilment status.

Guest capability. Guest users can browse, search, and manage a cart without registration. Upon account creation, the guest session is promoted to the authenticated context, preserving cart contents.

2.1.1.2 Administrator: Data Management and Operational Oversight

The **Administrator** operates the **administration interface** (Vue 3 Admin SPA), a separate application surface from the storefront with independent authentication requirements.

Product Management. Create, update, archive, and delete products. Define **fashion-specific metadata**: style code, season, material composition, department, gender target, care instructions, and fit notes. Manage **variants** (size and colour combinations) with SKUs, barcodes, physical dimensions, and independent pricing per variant. Upload and organise product images; each upload automatically triggers the **embedding generation pipeline**.

Taxonomy and Classification. Define **hierarchical taxonomies** (e.g., Clothing → Dresses → Evening Dresses). Assign products to taxon nodes for category-based browsing. Manage **option types** (Size, Colour, Material) with predefined option values.

Order Fulfilment. Review incoming orders, update fulfilment status, process payment captures and refunds, and manage shipment tracking.

Inventory Monitoring. View real-time stock levels per product variant per warehouse location. Review stock movement history for audit purposes.

User Governance. Create and manage user accounts. Assign **roles** (Customer, Administrator) and granular **permissions** (e.g., `catalog:products:create`, `orders:fulfillment:update`). Review user activity and manage account status.

2.1.1.3 System: Automated Background Processes

The **System** actor represents automated processes executing without human interaction. These run as scheduled or event-driven **background jobs** within the .NET application, backed by durable Hangfire storage.

Embedding Generation. When an administrator uploads a product image, a background job sends the image to the Python ML sidecar, receives the embedding vector, and stores it in pgvector with model metadata. The upload endpoint returns immediately; the embedding becomes available for search once the job completes.

Cart Expiry. A daily scheduled job removes carts with no activity for seven days, releasing reserved inventory and preventing accumulation of abandoned session data.

Inventory Reservation. During checkout, stock quantities are temporarily held. If checkout is not completed within a configurable window (fifteen minutes), the reservation expires and stock is returned to availability.

Index Maintenance. Periodic HNSW index rebuilds on the embedding column maintain search performance as the catalog grows, preventing query degradation from index fragmentation.

Payment Webhook Processing. Incoming Stripe webhooks are validated via signature verification and processed asynchronously, updating payment intent state without blocking the HTTP response to the gateway.

2.1.2 Functional Requirements

The platform's capabilities are specified as traceable requirements organised by business module. Each module is introduced with a single sentence of context; the table that follows enumerates specific requirements with identifiers that can be referenced throughout the design and evaluation chapters.

2.1.2.1 Catalog Module

The Catalog module manages the product lifecycle, from creation and classification through image handling to CBIR infrastructure.

Table 7.

Table 7: Catalog module functional requirements.

ID	Requirement	Description	Priority
CAT-FR-01	Product creation	Create products with name, description, slug, and SEO metadata (meta title, meta description)	High
CAT-FR-02	Fashion metadata	Assign fashion-specific fields: style code, season, material composition, care instructions, fit notes, department, and gender target	Medium
CAT-FR-03	Product variants	Define sellable variants combining option values (e.g., Size M + Colour Red) with SKU, barcode, physical dimensions, weight, and independent pricing	High
CAT-FR-21	Variant option values	Assign, synchronise, retrieve, and revoke option	High

ID	Requirement	Description	Priority
		values on variants to define the specific attribute combination each variant represents	
CAT-FR-22	Variant pricing	Set, list, synchronise, and remove time-bound prices per variant with currency specification; support multi-currency pricing for international catalogues	Medium
CAT-FR-04	Variant images	Upload variant images with automatic thumbnail generation; support multiple images per variant with display ordering	High
CAT-FR-14	Variant image CRUD	Delete, download image by ID, list images by variant, and update image metadata (alt text, display order) for variant images	Medium
CAT-FR-15	Embedding regeneration	Regenerate vector embeddings for existing images when the configured model changes or the original embedding is corrupted	Medium
CAT-FR-05	Embedding generation	Generate vector embeddings for each uploaded variant image via the configured ML model and store in pgvector with model metadata (model name, version, dimension)	High
CAT-FR-06	Image-based search	Enable CBIR: upload query image, generate embedding, query pgvector with cosine distance, return similarity-ranked results	High
CAT-FR-16	Product availability	Expose real-time per-variant stock availability through the storefront product detail endpoint	High
CAT-FR-17	Similar products	Retrieve visually similar products via embedding similarity for a given product, enabling recom-	Medium

ID	Requirement	Description	Priority
		mendation on product detail pages	
CAT-FR-07	Search configuration	Configure minimum similarity threshold and maximum result count for CBIR queries	Medium
CAT-FR-08	Pluggable models	Switch embedding model via environment variable without code changes; filter search results by active model	High
CAT-FR-09	Taxonomy	Organise products via hierarchical taxonomies with nested taxon trees (e.g., Clothing → Dresses → Evening Dresses)	Medium
CAT-FR-18	Taxon rules	Define business rules attached to taxon nodes (attribute constraints, automatic assignments) with update, sync, and retrieval operations	Low
CAT-FR-19	Auto-classification	Automatically classify products into appropriate taxons based on taxon rules, invoked when product attributes are updated	Low
CAT-FR-10	Option types	Define option types (e.g., Size, Colour, Material) with ordered option values reusable across multiple products	Medium
CAT-FR-20	Product option type association	Assign, synchronise, retrieve, and revoke option types per product, enabling products to expose relevant variant dimensions	Medium
CAT-FR-11	Slug uniqueness	Enforce slug uniqueness across the entire catalog at the database level	High
CAT-FR-12	Status lifecycle	Support product status lifecycle: Draft, Active, Archived with configurable availability and discontinuation dates	Medium
CAT-FR-13	Master variant	Designate one variant per product as the master (de-	High

ID	Requirement	Description	Priority
		fault) variant used for catalog listing display	

2.1.2.2 Identity Module

The Identity module provides authentication, authorisation, and user management for both customer-facing and administrative functions.

Table 8.

Table 8: Identity module functional requirements.

ID	Requirement	Description	Priority
IDN-FR-01	Registration	Register new customer accounts with email, password, and basic profile information	High
IDN-FR-02	Password login	Authenticate users via email and password, returning JWT access token (15-minute lifetime) and refresh token	High
IDN-FR-03	OAuth login	Authenticate users via Google OAuth 2.0 as an alternative to password-based login	Medium
IDN-FR-04	Token rotation	Implement refresh token rotation: each refresh operation invalidates the previous token and issues a new access and refresh token pair	High
IDN-FR-05	Reuse detection	Detect refresh token reuse: presenting a previously consumed token revokes all active tokens for that user, containing credential theft	High
IDN-FR-06	Guest sessions	Support guest sessions via signed cookie-based identifiers, enabling anonymous cart usage and catalog browsing without registration	High
IDN-FR-07	Role-based access	Enforce role-based access control (Customer, Administrator) with permission granularity at domain:category:action level	High
IDN-FR-11	Role management	Create, update, delete, and list roles with paging; assign, synchronise, retrieve, and revoke permissions per role	Medium

ID	Requirement	Description	Priority
IDN-FR-12	User-role assignment	Assign, synchronise, retrieve, and revoke roles for individual users; support direct permission grants bypassing role inheritance	Medium
IDN-FR-13	User status management	Enable and disable user accounts via the admin interface without deleting the account or its associated data	Medium
IDN-FR-08	Password reset	Reset passwords via time-limited, single-use email token with configurable expiry	Medium
IDN-FR-14	Password change	Allow authenticated users to change their password while logged in, requiring current password verification	Medium
IDN-FR-15	Email lifecycle	Confirm email addresses via verification token, change email with re-verification, and resend verification emails on demand	Medium
IDN-FR-16	Session management	Retrieve current session details, refresh tokens, and explicitly terminate sessions via logout	High
IDN-FR-09	User management	Manage user accounts, role assignments, and permission grants through the administration interface	Medium
IDN-FR-10	Two-factor auth	Support optional two-factor authentication via time-based one-time password (TOTP)	Low

2.1.2.3 Inventory Module

The Inventory module tracks physical stock quantities, manages checkout-time reservations to prevent overselling, and records stock movements for audit trails.

Table 9.

Table 9: Inventory module functional requirements.

ID	Requirement	Description	Priority
INV-FR-01	Stock locations	Define stock locations (warehouses or stores) with name, address, and active status flag	Medium
INV-FR-02	Stock quantities	Track stock quantities per product variant per location with separate on-hand and reserved counters	High

ID	Requirement	Description	Priority
INV-FR-08	Stock item CRUD	Create, update, delete, retrieve by ID, and list all stock items with paging; support bulk adjustment across multiple items and CSV-based import for batch operations	Medium
INV-FR-09	Low stock alerting	Identify and list stock items where on-hand quantity falls below a configured threshold, enabling proactive replenishment	Low
INV-FR-03	Checkout reservation	Reserve inventory quantities during active checkout sessions; release on cart expiry, order cancellation, or checkout timeout	High
INV-FR-11	Cart-based reservation	Reserve stock at the cart level during active shopping sessions, retrieve reservation status, and release on cart abandonment or expiry	High
INV-FR-04	Overselling prevention	Reject checkout when requested quantity exceeds available stock (on-hand minus reserved) at time of order confirmation	High
INV-FR-05	Stock transfers	Record stock transfers between locations with source, destination, quantity, and timestamp metadata	Medium
INV-FR-10	Transfer lifecycle	Manage the full stock transfer lifecycle: create a transfer, record in-transit status, confirm receipt at the destination, and cancel pending transfers; list transfers with paging and detail view	Medium
INV-FR-06	Audit log	Maintain an immutable audit log for all stock level changes including manual adjustments with reason and operator identity	Medium
INV-FR-12	Movement audit trail	Provide a dedicated paged listing and detail view for all stock movements with source, destination, quantity, reason, and operator identity; support the stock movement entity as a first-class data model with query capabilities	Medium
INV-FR-07	Reservation expiry	Expire stale reservations after a configurable time window (default 15 minutes of checkout inactivity)	Medium

2.1.2.4 Ordering Module

The Ordering module handles the customer purchase workflow from cart through multi-step checkout to completed order, tracking payment and shipment state independently.

Table 10.

Table 10: Ordering module functional requirements.

ID	Requirement	Description	Priority
ORD-FR-01	Shopping cart	Support guest and authenticated shopping carts with product variant selection, quantity management, and per-item pricing	High
ORD-FR-10	Cart management	Create, delete, and empty shopping carts; update item quantities and remove specific items from the cart	High
ORD-FR-02	Cart association	Associate a guest cart with a user account upon login or registration, merging contents without data loss	Medium
ORD-FR-03	Cart expiry	Auto-expire abandoned carts after seven days of inactivity via scheduled background job	Medium
ORD-FR-04	Checkout states	Enforce forward-only checkout state machine: Address, Delivery, Payment, Confirm, Complete	High
ORD-FR-11	Checkout validation	Validate checkout constraints (stock availability, address completeness, shipping method selection) before proceeding to payment	High
ORD-FR-12	Shipping rate selection	Select a shipping rate from available options within the cart before proceeding to payment	Medium
ORD-FR-05	Order totals	Calculate item total, adjustment total (discounts, surcharges), and shipment total independently; derive order total as their sum	High
ORD-FR-06	Dual state tracking	Track payment state (unpaid, authorised, captured, refunded) and shipment state (pending, shipped, delivered) independently per order	Medium
ORD-FR-07	Cancellation	Allow order cancellation at any pre-confirmation stage; cancelled	Medium

ID	Requirement	Description	Priority
		orders release reserved inventory and void payment intents	
ORD-FR-13	Admin order lifecycle	Approve, complete, and resume orders through the admin interface; update order status, shipping method, and shipping/billing addresses	Medium
ORD-FR-14	Customer order history	Allow authenticated customers to list their orders with paging and view individual order detail including line items, payments, and shipment tracking	Medium
ORD-FR-08	Order numbering	Generate unique, sequential, human-readable order numbers within a database transaction to prevent collisions under concurrent requests	Medium
ORD-FR-09	Order immutability	Mark completed orders as immutable; prevent modification of line items, adjustments, or totals after finalisation	High

2.1.2.5 Payment Module

The Payment module manages the lifecycle of payment intents across the Stripe production gateway and a Bogus test gateway.

Table 11.

Table 11: Payment module functional requirements.

ID	Requirement	Description	Priority
PAY-FR-01	Intent creation	Create payment intents with amount, currency, order identifier, and gateway selection	High
PAY-FR-02	Intent confirmation	Confirm payment intents to authorise fund capture at checkout completion	High
PAY-FR-08	Setup intent	Create Stripe SetupIntents for saving customer payment methods for future use, separate from transactional PaymentIntents	Low
PAY-FR-03	Capture and refund	Capture, void, and refund payment intents with idem-	High

ID	Requirement	Description	Priority
		potency keys to prevent duplicate processing on retry	
PAY-FR-04	Webhook processing	Process Stripe webhooks with cryptographic signature verification to confirm payment state transitions	High
PAY-FR-09	Payment void	Void authorised but uncaptured payments as a distinct operation from refund; void all payments associated with a cancelled order via a cross-cutting command	Medium
PAY-FR-10	Payment method management	Create, update, delete, activate, and deactivate payment methods through the admin interface with paged listing; expose active methods to the storefront for customer selection	Medium
PAY-FR-05	Refund validation	Validate that refund amount does not exceed captured amount before processing the refund	Medium
PAY-FR-06	Bogus gateway	Provide a Bogus gateway for development and testing that simulates the full payment lifecycle (Pending, Processing, Succeeded, Canceled) without external API calls	Medium
PAY-FR-07	State tracking	Maintain independent payment state tracking in parallel with the gateway; enable offline state queries without gateway round-trips	Medium

2.1.2.6 Shipping Module

The Shipping module configures delivery methods and calculates shipping rates by geographic zone.

Table 12.

Table 12: Shipping module functional requirements.

ID	Requirement	Description	Priority
SHP-FR-01	Delivery methods	Configure delivery methods (standard, express, local pickup) with carrier name, pricing rules, and applicable geographic zones	Medium
SHP-FR-04	Method lifecycle	Create, update, delete, activate, and deactivate shipping methods with paged listing and detail retrieval through the admin interface	Medium
SHP-FR-05	Rate lifecycle	Create, update, delete, and list shipping rates per method, zone, and weight/value tier through the admin interface	Medium
SHP-FR-02	Rate calculation	Calculate shipping rates at checkout based on delivery address zone, selected method, cart weight, and cart value	Medium
SHP-FR-06	Storefront calculation	Calculate shipping cost for a given cart at checkout, returning available methods with rates applicable to the delivery address zone	High
SHP-FR-03	Shipment tracking	Assign per-order shipment tracking with carrier identifier and tracking number for customer visibility	Low

2.1.2.7 Profile Module

The Profile module links customer identity to personalisation features and preference management.

Table 13.

Table 13: Profile module functional requirements.

ID	Requirement	Description	Priority
PRF-FR-01	Addresses	Manage user shipping and billing addresses with address type labels and default selection per type	Medium
PRF-FR-02	Wishlists	Manage wishlists: create, rename, delete lists; add and remove product variants with notes	Low
PRF-FR-03	Notifications	Configure per-channel notification preferences (email, SMS) with opt-in and opt-out per notification category	Low

2.1.2.8 Location Module

The Location module provides country and state reference data for address validation, shipping zone configuration, and tax calculation.

Table 14.

Table 14: Location module functional requirements.

ID	Requirement	Description	Priority
LOC-FR-01	Countries	Provide country reference data with ISO 3166-1 codes, display names, and active status flags	Medium
LOC-FR-02	States	Provide state and province reference data with ISO 3166-2 codes, linked to parent country	Medium
LOC-FR-03	Country CRUD	Create, update, delete, and list countries with paging; retrieve by ID and ISO code through both admin and storefront interfaces	Medium
LOC-FR-04	State CRUD	Create, update, delete, and list states with paging; retrieve by ID and ISO code; filter by parent country through both admin and storefront interfaces	Medium

2.1.2.9 Dashboard Module

The Dashboard module aggregates key performance indicators from all business modules into a unified administrative overview.

Table 15.

Table 15: Dashboard module functional requirements.

ID	Requirement	Description	Priority
DSH-FR-01	Aggregate dashboard	Provide a cross-module administrative dashboard aggregating key metrics (product counts, order volumes, inventory levels, user statistics) from all business modules into a single view	Medium

2.1.3 Non-Functional Requirements

Beyond feature completeness, the system must satisfy quantitative and qualitative constraints that determine its fitness for production use. Five quality dimensions are specified in Table Table 16 with concrete, measurable targets and the design rationale each constraint shaped.

Table 16.

Table 16: Non-functional requirements with measurable targets and design rationale. Each target is revisited in the evaluation chapter.

ID	Quality	Target	Rationale
NFR-01	Performance	CBIR end-to-end search latency under **1 second** (image upload through embedding generation, pgvector query, and result assembly). Non-search API endpoints respond within **200 milliseconds** under normal load. Asynchronous I/O prevents thread blocking under concurrent requests.	Real-time visual search requires sub-second response to maintain user engagement. Latency above one second measurably increases e-commerce abandonment. This target is treated as a **hard constraint**: a model exceeding the one-second budget is unsuitable for deployment regardless of retrieval quality. [14]
NFR-02	Security	**Authentication**: JWT access tokens expire after **15 minutes**. Refresh tokens follow **single-use rotation** with reuse detection; presenting a consumed token triggers **full revocation** of all tokens for that user. **Authorisation**: role-based access control with **granular permission claims** (`domain:category:action`) enforced at endpoint middleware. **Rate limiting**: **five requests per minute** for login, **three per hour** for registration. **Transport**: security headers (CSP, HSTS, X-Frame-Options, X-Content-Type-Options) on all responses. **File upload**: **magic-byte verification**, extension allowlist, and **10 MB size limit**.	Browser-based single-page applications are exposed to the open internet and must defend against common web threats. Short-lived tokens and refresh rotation limit the window of token compromise. File upload validation prevents malicious payloads from entering the embedding pipeline.
NFR-03	Modularity	**Eight business modules** in a single .NET assem-	The modular monolith pattern preserves bounded-

ID	Quality	Target	Rationale
		bly, separated by namespace convention with **zero direct cross-references** at compile time. All inter-module communication occurs through **MediatR** in-process message dispatch. Each module is **independently testable** without loading neighbours or their database contexts.	context separation without distributed-system overhead of service discovery, inter-service authentication, and network serialisation. The MediatR boundary provides a clean seam: extracting a module into a separate service requires replacing only the dispatcher, not the handlers or callers.
NFR-04	Observability	**OpenTelemetry** distributed tracing spans both .NET and Python services. Trace context propagates through HTTP headers, linking a storefront search to its embedding generation span. **Correlation identifiers** on every structured log entry track requests across service boundaries. **Health check endpoints** report dependency connectivity for Aspire orchestration.	In a polyglot architecture spanning C# and Python, end-to-end request tracing is essential for diagnosing latency bottlenecks and error propagation. Correlation identifiers enable a single request to be followed across service boundaries in log aggregation tools.
NFR-05	Reliability	**Hangfire** with **Redis-backed persistence** ensures background jobs survive application restarts. Cart expiry: daily job removes carts with **seven days** of inactivity, releasing reserved inventory. **Idempotency**: Stripe webhooks carry idempotency keys preventing duplicate payment processing. **Reservation timeout**: checkout inventory holds expire after **fifteen minutes** of inactivity.	Many e-commerce operations are inherently long-running (cart expiry, payment confirmation, index maintenance) and cannot complete within an HTTP request-response cycle. A durable job queue ensures these operations complete reliably across process crashes, scheduled restarts, and deployment rollouts.

These constraints shaped architectural decisions throughout the system. The one-second CBIR latency target ruled out queued embedding pipelines in favour of synchronous generation. The modularity requirement drove the MediatR dispatch model over direct method calls. The reliability constraint motivated Redis-backed Hangfire over in-memory job storage. Each target is revisited in the evaluation chapter, where benchmark results confirm whether the implemented system meets these stated requirements.

2.1.4 Feature Classification

Not all features of ReSys.Shop carry equal research significance. Seven feature areas are classified as either **Core Research** (directly contributing to the thesis's academic objectives) or **Supporting Infrastructure** (providing the realistic e-commerce context in which the research is conducted and evaluated). This distinction clarifies the scope of the thesis's original contribution and explains why certain features exist in the platform but are not discussed in depth in subsequent chapters.

Table 17.

Table 17: Classification of feature areas into Core Research and Supporting Infrastructure. Core Research features represent the thesis's original contributions; Supporting Infrastructure features provide the realistic e-commerce context necessary for meaningful evaluation.

Feature Area	Classification	Rationale
Visual Search (CBIR)	Core Research	Primary contribution: integrated multi-model CBIR with pluggable architecture enabling image-based product search across multiple embedding models (CNN, ViT, CLIP-based) within a production-style e-commerce platform.
ML Embedding Pipeline	Core Research	Critical infrastructure: automated ingestion of product images, generation of vector embeddings via the Python sidecar, storage in pgvector, and HNSW indexing. The operational backbone of the visual search capability.
Model Benchmark System	Core Research	Secondary contribution: systematic protocol for comparing retrieval accuracy and operational efficiency across 11 embedding models, providing a practical guide for model selection in re-

Feature Area	Classification	Rationale
		source-constrained deployments.
Product Catalog	Supporting Infrastructure	Required context: provides the structured dataset of fashion products, with variants, images, taxonomies, and metadata, that serves as the search target for CBIR evaluation.
Order System	Supporting Infrastructure	Metric validation: provides conversion events (add-to-cart, checkout completion) that serve as proxy indicators of search success, enabling evaluation of visual search within a realistic shopping workflow.
Inventory	Supporting Infrastructure	Realism constraint: ensures that search results reflect actual product availability, preventing the unrealistic scenario where visually similar but out-of-stock items appear in search results.
Authentication	Supporting Infrastructure	Security baseline: protects administrative functions and user-specific data, enabling the application to operate in a representative security posture without which the system would be a research prototype rather than a deployable platform.

The classification makes explicit what the thesis does and does not claim as contribution. The CBIR pipeline, encompassing embedding generation, vector storage, and similarity search, is the core research artefact. The e-commerce modules (Catalog, Ordering, Inventory, Payment, Identity) are supporting infrastructure, built to provide a realistic context that validates the visual search results in a production-like environment. This separation is maintained through-

out the chapter: Sections 2.3 and 2.4 devote detailed treatment to the research features, while the supporting infrastructure is described only to the extent necessary to understand the system's design.

2.2 FUNCTIONAL DECOMPOSITION AND USE CASES

This section presents the platform's use cases, organised by actor and business module. Each use case is traceable to the functional requirements defined in Section 2.1 through the Related FR column. The Administrator actor manages data and operations across all modules; the Customer actor interacts through the storefront for discovery, purchase, and account management; the System actor performs automated background operations.

2.2.1 Administrator Use Cases

The Administrator actor manages data and operational workflows across all eight business modules through a dedicated administration interface. Use cases span product lifecycle management, order fulfilment, payment processing, inventory control, user governance, and system configuration.

2.2.1.1 Catalog Management

Table 18.

Table 18: Administrator use cases for the Catalog module.

UC-ID	Use Case	Actor	Flow	Postcondition	Related FR
UC-ADM-CAT-01	Create product	Admin	Enter product details (name, description, slug, fashion metadata), define at least	Product created with Draft status; master variant available for catalog operations	CAT-FR-01, CAT-FR-02, CAT-FR-03, CAT-FR-13

UC-ID	Use Case	Actor	Flow	Postcondition	Related FR
			one master variant with SKU and price, upload images, assign taxons.		
UC-ADM-CAT-02	Update product	Admin	Modify any product field (description, metadata, status, SEO fields). Slug changes validated for uniqueness.	Product updated; if status changed to Archived, hidden from storefront.	CAT-FR-01, CAT-FR-12
UC-ADM-CAT-03	Delete product	Admin	Soft-delete product and all associated	Product flagged as deleted, not recoverable through storefront.	CAT-FR-01

UC-ID	Use Case	Acto	Flow	Postcondition	Re-lated FR
			vari-ants, im-ages, and clas-si-fi-ca-tions.		
UC-ADM-CAT-04	Add variant	Admin	Se-lect par-ent prod-uct, de-fine SKU, bar-code, di-men-sions, weight, and pric-ing; as-sign op-tion val-ues (Size M + Colour Red) to spec-ify the vari-ant con-fig-u-ra-tion.	Variant created and available for inventory tracking, pricing, and image assign-ment.	CAT-FR-03, CAT-FR-21, CAT-FR-22

UC-ID	Use Case	Actor	Flow	Postcondition	Related FR
UC-ADM-CAT-05	Upload variant images	Admin	Selects a variant, uploads image file, sets alt text and display order. System triggers embedding generation job.	Image stored with metadata; embedding generation queued as background job.	CAT-FR-04, CAT-FR-05, CAT-FR-15
UC-ADM-CAT-06	Regenerate embeddings	Admin	Selects variant images and triggers embedding regeneration for the currently con-	New embeddings generated and stored in pgvector with updated model metadata.	CAT-FR-05, CAT-FR-08, CAT-FR-15

UC-ID	Use Case	Actor	Flow	Postcondition	Related FR
			figured model.		
UC-ADM-CAT-07	Manage taxonomies	Admin	Create, update, or delete taxonomies and nested taxon trees. Reorder taxons within a taxonomy.	Taxonomy structure updated; products classified under affected taxons retain their classifications.	CAT-FR-09
UC-ADM-CAT-08	Classify products	Admin	Assign products to one or more taxon nodes. Synchronise or revoke classifications.	Product appears under assigned categories in storefront browsing and search filters.	CAT-FR-09, CAT-FR-19
UC-ADM-CAT-09	Manage option types	Admin	Create option types	Option types available for variant configuration across assigned products.	CAT-FR-10, CAT-FR-20

UC-ID	Use Case	Acto	Flow	Postcondition	Re-lated FR
			(e.g., Size, Colour) with or-dered op-tion val-ues. As-sign op-tion types to prod-ucts.		

2.2.1.2 Order Fulfilment

Table 19.

Table 19: Administrator use cases for the Ordering module.

UC-ID	Use Case	Acto	Flow	Postcondition	Re-lated FR
UC-ADM-ORD-01	View orders	Admi	List or-ders with pag-ing, fil-ter-ing by sta-tus, date range, and cus-tomer. Drill into	Order data displayed with full transactional context.	ORD-FR-05, ORD-FR-06, ORD-FR-13

UC-ID	Use Case	Actor	Flow	Postcondition	Related FR
			individual order detail showing line items, payments, shipments, and history.		
UC-ADM-ORD-02	Update order	Admin	Modify order attributes: add, update, or remove line items; update shipping and billing addresses; change shipping method.	Order updated; totals recalculated to reflect line item and adjustment changes.	ORD-FR-05, ORD-FR-13
UC-ADM-ORD-03	Approve order	Admin	Approve a pend-	Order approved and moved to fulfilment queue; inventory reserved if not already held.	ORD-FR-04, ORD-FR-13

UC-ID	Use Case	Acto	Flow	Postcondition	Related FR
			ing order for fulfilment. System verifies payment status and inventory availability.		
UC-ADM-ORD-04	Complete order	Admin	1. Open order and mark as fulfilled after shipment confirmation. Finalise totals and lock the order against further modi-	Order completed; immutable after this point. Inventory on-hand quantities decremented.	ORD-FR-09, ORD-FR-13

UC-ID	Use Case	Actor	Flow	Postcondition	Related FR
			fication.		
UC-ADM-ORD-05	Cancel order	Admin	Cancel an order at any pre-confirmation stage with mandatory reason. Release reserved inventory, void payment intents.	Order cancelled; inventory released; payment voided.	ORD-FR-07
UC-ADM-ORD-06	Resume order	Admin	Resume a previously paused or stalled order, moving it back into the	Order returned to processing state.	ORD-FR-13

UC-ID	Use Case	ActoFlow	Postcondition	Re-lated FR
		ac-tive work-flow.		

2.2.1.3 Payment Management

Table 20.

Table 20: Administrator use cases for the Payment module.

UC-ID	Use Case	ActoFlow	Postcondition	Re-lated FR
UC-ADM-PAY-01	Capture payment	Admin Cap-ture an au-tho-rised pay-ment in-tent, trans-fer-ring funds from the cus-tomer's ac-count. Idem-po-tency key pre-vents du-pli-cate cap-ture.	Payment captured; funds transferred. Order payment state updated to captured.	PAY-FR-03
UC-ADM-PAY-02	Refund payment	Admin Is-sue a	Refund processed; payment state updated to re-funded (partial or full).	PAY-FR-03,

UC-ID	Use Case	Actor	Flow	Postcondition	Related FR
			refund against a captured payment. Validate that refund amount does not exceed captured amount.	Funds returned to customer.	PAY-FR-05
UC-ADM-PAY-03	Void payment	Admin	Void an authorised but uncaptured payment, releasing the fund hold without charging the customer.	Payment voided; fund hold released. Order payment state updated.	PAY-FR-09
UC-ADM-PAY-04	View payments	Admin	List payments with	Payment records displayed with gateway state and system state shown side by side.	PAY-FR-07

UC-ID	Use Case	Actor	Flow	Postcondition	Related FR
			paging and filtering by status, date, gateway, and order reference. View individual payment detail.		
UC-ADM-PAY-05	Manage payment methods	Admin	Create, update, activate, deactivate, or delete payment methods. Configure gateway-specific	Payment method configuration updated; active methods available for storefront selection.	PAY-FR-10

UC-ID	Use Case	Acto	Flow	Postcondition	Related FR
			parameters per method.		

2.2.1.4 Inventory Control

Table 21.

Table 21: Administrator use cases for the Inventory module.

UC-ID	Use Case	Acto	Flow	Postcondition	Related FR
UC-ADM-INV-01	Manage stock locations	Admin	Create, update, or delete warehouse locations. Set a default location for new stock intake.	Location configuration updated; stock items assigned to locations maintain references.	INV-FR-01
UC-ADM-INV-02	Manage stock items	Admin	Create stock items for product vari-	Stock quantities updated; audit log records the change with operator identity and reason.	INV-FR-02, INV-FR-06, INV-FR-08

UC-ID	Use Case	Acto	Flow	Postcondition	Related FR
			ants at specific locations with initial on-hand quantity. Update, delete, or bulk-adjust quantities across multiple items.		
UC-ADM-INV-03	Restock inventory	Admin	Increase on-hand quantity for a stock item, recording the restock event with source and reason.	On-hand quantity incremented; stock movement audit entry created.	INV-FR-02, INV-FR-06, INV-FR-08

UC-ID	Use Case	Actor	Flow	Postcondition	Related FR
UC-ADM-INV-04	Transfer stock	Admin	Initiate a transfer of stock quantity from a source location to a destination. Record in-transit status, confirm receipt at destination, or cancel pending transfers.	Stock decremented at source, incremented at destination upon receipt. Full audit trail recorded.	INV-FR-05, INV-FR-10
UC-ADM-INV-05	Review stock movements	Admin	Browse paginated listing of all stock	Complete audit trail visible for compliance and debugging.	INV-FR-06, INV-FR-12

UC-ID	Use Case	Acto	Flow	Postcondition	Related FR
			movements. View detail for any movement including source, destination, quantity, reason, and operator.		
UC-ADM-INV-06	Monitor low stock	Admin	View a filtered list of stock items where on-hand quantity falls below the configured threshold, grouped by lo-	Low-stock items identified for proactive replenishment planning.	INV-FR-09

UC-ID	Use Case	Acto	Flow	Postcondition	Related FR
			ca-tion.		

2.2.1.5 Identity and Access Governance

Table 22.

Table 22: Administrator use cases for the Identity module.

UC-ID	Use Case	Acto	Flow	Postcondition	Related FR
UC-ADM-IDN-01	Manage users	Admin	Cre-ate user ac-counts with email and ini-tial role. Up-date pro-file fields. En-able or dis-able ac-counts. View user de-tail in-clud-ing as-signed roles and per-	User account created or modified. Status changes take effect on next authentication attempt.	IDN-FR-01, IDN-FR-09, IDN-FR-13

UC-ID	Use Case	Actor	Flow	Postcondition	Related FR
			missions.		
UC-ADM-IDN-02	Manage roles	Admin	Create, update, or delete roles. Assign, synchronise, or revoke permissions per role. List roles with paging.	Role configuration updated. Users assigned to modified roles receive updated permission sets.	IDN-FR-07, IDN-FR-11
UC-ADM-IDN-03	Assign user roles	Admin	Assign, synchronise, or revoke roles for individual users. View current role assignments.	User role assignments updated. Effective permissions recalculated on next token issuance.	IDN-FR-07, IDN-FR-12

UC-ID	Use Case	Actor	Flow	Postcondition	Related FR
UC-ADM-IDN-04	Grant user permissions	Admin	Grant or revoke granular permissions directly on a user account, bypassing role inheritance for exceptional cases.	User's effective permissions updated to include direct grants plus role-derived permissions.	IDN-FR-07, IDN-FR-12
UC-ADM-IDN-05	View permissions catalog	Admin	Browse the complete list of available permission claims across all modules, organised by do-	Full permission matrix visible for audit and role design.	IDN-FR-07

UC-ID	Use Case	Acto	Flow	Postcondition	Related FR
			main and category.		

2.2.1.6 Shipping Configuration

Table 23.

Table 23: Administrator use cases for the Shipping module.

UC-ID	Use Case	Acto	Flow	Postcondition	Related FR
UC-ADM-SHP-01	Manage shipping methods	Admin	Create, update, activate, deactivate, or delete shipping methods. Configure carrier name, pricing rules, and applicable ge-	Shipping method available for customer selection at checkout if active, dated, and zone-applicable.	SHIP-FR-01, SHIP-FR-04

UC-ID	Use Case	Actor	Flow	Postcondition	Related FR
			o-graphic zones per method.		
UC-ADM-SHP-02	Manage shipping rates	Admin	Create, update, or delete shipping rates per method. Define rate tiers by weight range, cart value range, and geographic zone.	Shipping rates applied during storefront checkout calculation for matching carts.	SHIP-FR-02, SHIP-FR-05

2.2.1.7 Reference Data Management

Table 24.

Table 24: Administrator use cases for the Location module.

UC-ID	Use Case	Actor	Flow	Postcondition	Related FR
UC-ADM-LOC-01	Manage countries	Admin	Create, update, or delete	Country data updated; active countries available in storefront address forms and shipping zone configuration.	LOC-FR-01, LOC-FR-03

UC-ID	Use Case	Acto	Flow	Postcondition	Re-lated FR
			country records with ISO 3166-1 codes and display names. Set active status to control availability in address forms and shipping zones.		
UC-ADM-LOC-02	Manage states	Admin	Create, update, or delete state records with ISO 3166-2 codes, linked to parent country. Set active	State data updated; active states available for address validation within their parent country.	LOC-FR-02, LOC-FR-04

UC-ID	Use Case	Actor	Flow	Postcondition	Related FR
			status per state.		

2.2.2 Customer Use Cases

The Customer actor interacts through the browser-based storefront. Use cases span product discovery (including the core CBIR capability), the purchase workflow from cart through checkout, and account management. Guest customers have access to browsing, searching, and cart operations; authenticated customers additionally access order history, profile management, and personalised features.

2.2.2.1 Product Discovery and Visual Search

Table 25.

Table 25: Customer use cases for the Catalog module. CBIR visual search (UC-STR-CAT-03) is the primary research use case.

UC-ID	Use Case	Actor	Flow	Postcondition	Related FR
UC-STR-CAT-01	Browse catalog	Customer	Navigate the hierarchical taxonomy tree to browse products by category.	Filtered product listing displayed with thumbnails, prices, and availability indicators.	CAT-FR-09, CAT-FR-10

UC-ID	Use Case	Actor	Flow	Postcondition	Related FR
			Apply faceted filters (price range, size, colour, brand) to narrow results. Scroll through paginated product grids.		
UC-STR-CAT-02	View product detail	Customer	Select a product to view full detail: description, fashion metadata, variant options with size and colour availability,	Product detail page displayed with all variant configurations and real-time availability per variant.	C-CAT-FR-01, C-CAT-FR-16

UC-ID	Use Case	Actor	Flow	Postcondition	Related FR
			multiple images, pricing per variant, and taxonomy path.		
UC-STR-CAT-03	Search by image (CBIR)	Customer	Uploads a reference image (JPEG, PNG, WebP, max 10 MB). System: validates image format and size, sends to .NET API, which forwards to Python ML sidecar	Visually similar products displayed ranked by cosine similarity. Results filtered to items above the configured minimum similarity threshold.	CAT-FR-06, CAT-FR-07, CAT-FR-08, NFR-01

UC-ID	Use Case	Actor	Flow	Postcondition	Related FR
			for embedding generation, queries pgvector with cosine distance HNSW search, returns top-K results ranked by similarity score. Results display image thumbnail, product name, variant, price, and similarity score.		

UC-ID	Use Case	Actor	Flow	Postcondition	Related FR
UC-STR-CAT-04	View similar products	Customer	On a product detail page, view products visually similar to the current product based on embedding similarity.	Similar products displayed for passive discovery without requiring an upload.	C-AT-FR-17
UC-STR-CAT-05	Keyword search	Customer	Enter a text query to search products by name, description, or fashion attributes.	Matching products displayed. Text search complements the visual search capability.	C-AT-FR-01

UC-ID	Use Case	Actor	Flow	Postcondition	Related FR
			Results ranked by relevance.		

2.2.2.2 Cart and Checkout

Table 26.

Table 26: Customer use cases for the Ordering module.

UC-ID	Use Case	Actor	Flow	Postcondition	Related FR
UC-STR-ORD-01	Manage cart	Customer	Create a new cart or access the current session cart. Add product variants with desired quantity. Update item quantities	Cart persisted across page navigation. Guest carts survive browser sessions via signed cookie.	STR-ORD-FR-01, STR-ORD-FR-10

UC-ID	Use Case	Actor	Flow	Postcondition	Related FR
			or remove items. View cart summary with item totals.		
UC-STR-ORD-02	Associate cart with account	Customer	Upon login or registration, the existing guest cart is promoted to the authenticated user context. Cart contents are merged without data loss.	Cart now associated with user account; available across devices via authentication.	ORD-FR-02
UC-STR-ORD-03	Select shipping address	Customer	Select a	Shipping address set on the order; shipping zone	ORD-FR-04,

UC-ID	Use Case	Actor	Flow	Postcondition	Related FR
			saved address or enter a new shipping address. System validates address completeness and zone applicability.	determined for rate calculation.	PRF-FR-01
UC-STR-ORD-04	Select shipping method	Customer	Choose from available delivery methods with calculated rates based on address zone, cart weight, and	Shipping method and rate applied to the order; shipment total updated.	ORD-FR-04, ORD-FR-12, SHP-FR-02, SHP-FR-06

UC-ID	Use Case	Actor	Flow	Postcondition	Related FR
			cart value.		
UC-STR-ORD-05	Complete checkout	Customer	After address and shipping method selection, proceed to payment. Select payment method or enter new payment details. Review order summary (items, adjustments, shipping, total). Confirm order. System:	Order created with unique order number. Inventory reserved for each line item. Payment intent linked to order. Cart cleared.	ORD-FR-04, ORD-FR-05, ORD-FR-08, ORD-FR-11, NFR-01

UC-ID	Use Case	Actor	Flow	Postcondition	Related FR
			validates stock availability, processes payment intent, creates order record, reserves inventory, clears cart.		
UC-STR-ORD-06	View order history	Customer	Lists past orders with status, date, and total. Drill into order detail to view line items, payment state, shipment	Complete order history visible for authenticated customers.	ORD-FR-14

UC-ID	Use Case	Actor	Flow	Postcondition	Related FR
			tracking.		
UC-STR-ORD-07	Cancel order	Customer	Can-cel pending order before confirmation. System releases reserved inventory and voids the payment intent.	Order cancelled; inventory returned to available; payment voided.	ORD-FR-07

2.2.2.3 Payment Processing

Table 27.

Table 27: Customer use cases for the Payment module.

UC-ID	Use Case	Actor	Flow	Postcondition	Related FR
UC-STR-PAY-01	Create payment intent	Customer	At the Payment step of	Payment Intent created with pending status. Stripe payment form displayed.	PAY-FR-01

UC-ID	Use Case	Actor	Flow	Postcondition	Related FR
			checkout, the system creates a Stripe PaymentIntent for the order total. The customer is presented with a Stripe payment form to enter card details or select a saved payment method.		
UC-STR-PAY-02	Confirm payment	Customer	After entering payment	Payment confirmed; order moves to Complete state. PaymentIntent state updated to Succeeded.	PAY-FR-02

UC-ID	Use Case	Actor	Flow	Postcondition	Related FR
			details, the customer submits the payment. Stripe processes the transaction and returns a confirmation. The system records the payment state and proceeds to order finalisation.		

2.2.2.4 Authentication and Account Management

Table 28.

Table 28: Customer use cases for the Identity module.

UC-ID	Use Case	Actor	Flow	Postcondition	Related FR
UC-STR-IDN-01	Register	Customer	Enter email, password, and profile information. Submit registration form. System validates uniqueness of email, hashes password, creates user account, and sends email verification.	User account created. Verification email sent. Customer can log in after email confirmation.	IDN-FR-01
UC-STR-IDN-02	Login with password	Customer	Enter registered email and	Customer authenticated; session established. Guest cart associated with account if one exists.	IDN-FR-02, IDN-FR-04

UC-ID	Use Case	Actor	Flow	Postcondition	Related FR
			password. System validates credentials, issues JWT access token (15-minute lifetime) and refresh token. Tokens returned in secure HTTP-only cookies.		
UC-STR-IDN-03	Login with Google	Customer	Click Google login button. Redirected to Google OAuth consent screen.	Customer authenticated via Google OAuth. New users auto-registered with Google profile data.	IDN-FR-03

UC-ID	Use Case	Actor	Flow	Postcondition	Related FR
			After authorisation, Google redirects back with authorisation code. System exchanges code for tokens and creates or links user account.		
UC-STR-IDN-04	Refresh session	Customer	Before access token expires, the client sends the refresh token to	Session extended without requiring re-login. Access token lifetime renewed.	IDN-FR-04, IDN-FR-05

UC-ID	Use Case	Actor	Flow	Postcondition	Related FR
			obtain a new access token and refresh token pair. Previous refresh token invalidated.		
UC-STR-IDN-05	Logout	Customer	Explicitly terminate the current session. System invalidates the refresh token and clears authen-	Session terminated. Refresh token invalidated. Cookies cleared.	IDN-FR-16

UC-ID	Use Case	Actor	Flow	Postcondition	Related FR
			tication cookies.		
UC-STR-IDN-06	Reset password	Customer	Requests password reset link via registered email. System sends time-limited, single-use reset token. Customer clicks link, enters new password. Token consumed and invalidated.	Password updated. All existing refresh tokens revoked for security.	IDN-FR-08, IDN-FR-14

UC-ID	Use Case	Actor	Flow	Postcondition	Related FR
UC-STR-IDN-07	Change password	Customer	While authenticated, enter current password and new password. System verifies current password before update.	Password changed without invalidating current session tokens.	IDN-FR-14

2.2.2.5 Profile and Preferences

Table 29.

Table 29: Customer use cases for the Profile module.

UC-ID	Use Case	Actor	Flow	Postcondition	Related FR
UC-STR-PRF-01	Manage addresses	Customer	Create, update, or delete shipping and	Addresses available for selection during checkout. Default address preselected.	PRF-FR-01

UC-ID	Use Case	Actor	Flow	Postcondition	Related FR
			billing addresses. Set a default address per type. View all saved addresses.		
UC-STR-PRF-02	Manage wishlists	Customer	Create named wishlists. Add product variants to a wishlist with optional notes. Remove items. View wishlist contents. Rename or delete lists.	Wishlist updated. Items retained for future reference and potential cart addition.	PRF-FR-02
UC-STR-PRF-03	Manage notification preferences	Customer	Configure	Notification preferences saved. Future notifica	PRF-FR-03

UC-ID	Use Case	Actor	Flow	Postcondition	Related FR
			tions per channel notification settings (email, SMS) for each notification category (order updates, promotions, stock alerts). Opt in or out per category.	respect the configured settings.	

2.2.3 System Use Cases

The System actor represents automated background processes that execute without human interaction. These processes maintain data consistency, generate embeddings asynchronously, manage inventory reservations, and perform scheduled maintenance.

2.2.3.1 Automated Operations

Table 30.

Table 30: System use cases for background operations.

UC-ID	Use Case	Actor	Flow	Postcondition	Related FR
UC-SYS-EMB-01	Generate image embeddings	System Administrator	An embedding is stored and indexed. Image becomes searchable via CBIR within seconds of the job completing.	A product image. Upload endpoint returns immediately. A background job picks up the image, sends it to the Python ML sidecar via HTTP, receives the	C-AT-FR-05, C-AT-FR-15

UC-ID	Use Case	Actor	Flow	Postcondition	Related FR
			embedding vector, stores it in pgvector with model metadata (model name, version, dimension, generation timestamp). HNSW index automatically updated.		
UC-SYS-EMB-02	Regenerate all embeddings	System	When a configured embedding model is changed (via environ-	All embeddings regenerated with the current model. Search results consistent with the active model.	CAT-FR-08, CAT-FR-15

UC-ID	Use Case	Actor	Flow	Postcondition	Related FR
			ronment variable), existing image embeddings must be regenerated. A bulk regeneration job processes all variant images, generating new embeddings from the current model and updating pgvec-		

UC-ID	Use Case	Actor	Flow	Postcondition	Related FR
			tor records.		
UC-SYS-EMB-03	Verify model health	System	The Python ML sidecar exposes a health check endpoint. .NET Aspire periodically probes the endpoint. The sidecar responds with the currently loaded model, its embedding dimension, and the last inference latency.	ML sidecar availability continuously monitored. Automatic restart on failure. .NET API routes embedding requests only when sidecar reports healthy.	NFR-04

UC-ID	Use Case	Actor	Flow	Postcondition	Related FR
			If the sidecar is unreachable or returns an error, Aspire restarts the container.		
UC-SYS-JOB-01	Expire abandoned carts	System	A daily scheduled Hangfire job queries for carts with no activity in the past seven days. Each expired cart: releases reserved inventory	Abandoned carts removed. Reserved inventory returned to available. Database storage from stale carts reclaimed.	ORD-FR-03, NFR-05

UC-ID	Use Case	Actor	Flow	Postcondition	Related FR
			for all line items, voids any associated payment intents, marks the cart as expired.		
UC-SYS-JOB-02	Manage inventory reservations	System	During checkout, stock quantities are temporarily reserved to prevent overselling. A scheduled job scans for reservations held longer than	Stale reservations expired. Inventory accurately reflects true availability. Overselling prevented through the reservation window.	INV-FR-03, INV-FR-07, NFR-05

UC-ID	Use Case	Actor	Flow	Postcondition	Related FR
			fifteen minutes without checkout completion. Expired reservations are released and inventory returned to availability.		
UC-SYS-JOB-03	Process payment webhooks	System	Stripe sends an HTTP POST webhook to the .NET API for each payment state change (payment intent succeeded,	Payment state synchronised between Stripe and the system database. Order state transitions triggered where appropriate. Duplicate webhooks safely discarded.	PAY-FR-04, PAY-FR-07, NFR-05

UC-ID	Use Case	Actor	Flow	Postcondition	Related FR
			charge captured, refund processed). The system validates the webhook signature using the Stripe signing secret. On valid signature, the webhook payload is processed: payment state updated in the database, associated		

UC-ID	Use Case	Actor	Flow	Postcondition	Related FR
			order state updated if applicable. Idempotency key prevents duplicate processing of retried webhooks.		
UC-SYS-JOB-04	Maintain vector index	System	A periodic maintenance job rebuilds the HNSW index on the variant images embedding col-	HNSW index optimised; CBIR search latency remains stable as catalog size increases.	CAT-FR-06

UC-ID	Use Case	Actor	Flow	Postcondition	Related FR
			umn. This optimises search performance as the catalog grows and prevents query degradation from index fragmentation after many insertions and updates.		

2.3 SYSTEM ARCHITECTURE & DESIGN

This chapter presents the architectural design of ReSys.Shop, progressing from a high-level system overview through domain modelling, to the structural, data, API, and security layers. The design follows a service-oriented approach with three independently deployable services, a Vue 3 frontend, a .NET 10 modular monolith backend, and a Python machine learning sidecar, each responsible for a distinct technological concern. The chapter is organised into six sections, each

accompanied by architectural diagrams that provide visual representations of the system's structure, behaviour, and deployment topology.

2.3.1 System Overview

ReSys.Shop is built as a service-oriented system with three distinct services. The frontend is implemented in Vue 3 and TypeScript using the Vite build tool. The backend is a .NET 10 modular monolith using ASP.NET Core for HTTP handling, Entity Framework Core for data access, and Carter for minimal API endpoint registration. The machine learning service is a Python FastAPI application running PyTorch models that generates vector embeddings from product images for visual similarity search.

Table Table 31 summarises the three services, their technology stacks, and their primary responsibilities within the platform.

Table 31.

Table 31: System services and their technology stacks. Each service is independently deployable and communicates through well-defined HTTP contracts.

Service	Technology Stack	Responsibilities
Vue Frontend	Vue 3 + TypeScript + Vite	• Customer storefront (Nuxt UI) • Administrator dashboard (PrimeVue) • Pinia state management • Image upload and visual search UI
.NET Backend	.NET 10 + ASP.NET Core + Carter + EF Core	• REST API endpoints via Carter minimal APIs • Business logic via MediatR CQRS pattern • PostgreSQL persistence with pgvector vector search • JWT authentication and RBAC authorisation
Python ML	Python 3.12 + FastAPI + PyTorch	• Fashion-CLIP and other em-

Service	Technology Stack	Responsibilities
		bedding model inference • Vector embedding generation from product images • Multi-model support with lazy-loading strategy

The backend is internally organised into eight bounded contexts following the principles of Domain-Driven Design. Each context owns a distinct area of business logic and communicates with other contexts exclusively through MediatR in-process message dispatch, there are no direct namespace references between business modules. Table Table 32 lists each context, its aggregate root, and a representative sample of its domain entities.

Table 32.

Table 32: Bounded contexts with aggregate roots and key domain entities. Each context owns its database schema and communicates with other contexts through MediatR dispatch only.

Bounded Context	Aggregate Root	Key Domain Entities
Catalog	Product	Variant, VariantImage, OptionType, OptionValue, Classification, Taxonomy, Taxon
Ordering	Order	LineItem, Adjustment, Shipment
Payment	PaymentIntent	PaymentCapture
Inventory	StockItem	StockLocation, StockMovement, StockReservation
Identity	User	Role, UserRole, RefreshToken, UserLogin
Profile	UserProfile	Address, Wishlist
Shipping	ShippingMethod	ShippingRate, ShippingZone
Location	Country	State

The separation of concerns across these eight contexts enables independent evolution of each business domain while the modular monolith deployment model avoids the operational complexity of distributed microservices. The following section details the domain-driven design principles that govern these contexts.

2.3.2 Domain-Driven Design

The ReSys.Shop platform applies Domain-Driven Design (DDD) to structure its business logic around eight bounded contexts, each with well-defined aggregate roots, domain entities, and invariants. This section presents the context map, the aggregate design with invariants, the ubiquitous language glossary, and the state machines that govern the checkout and payment lifecycles.

2.3.2.1 Bounded Context Map

The eight bounded contexts partition the e-commerce domain along business capability boundaries. Each context owns its data, its domain logic, and its vocabulary, terms that are well-defined within a context may carry different meaning in another. For example, a Variant in the Catalog context is a sellable unit with a SKU and pricing; a LineItem in the Ordering context references that same variant but from the perspective of purchase fulfilment.

The integration between contexts follows the **Conformist** pattern: all contexts conform to a shared technical kernel defined in the Shared layer, which provides the `Result<T>` return type, the `ICommand` and `IQuery` marker interfaces, and the `Entity` base class with audit and versioning columns. Communication occurs exclusively through MediatR `ISender`, a context dispatches a query or publishes a notification, and other contexts react without ever importing one another's namespace. This in-process dispatch model eliminates the network latency of inter-service messaging while preserving the logical isolation of the bounded contexts.

Figure Figure 1 depicts the eight contexts and the **Published Language**, the shared identifiers and value types, that flow between them.

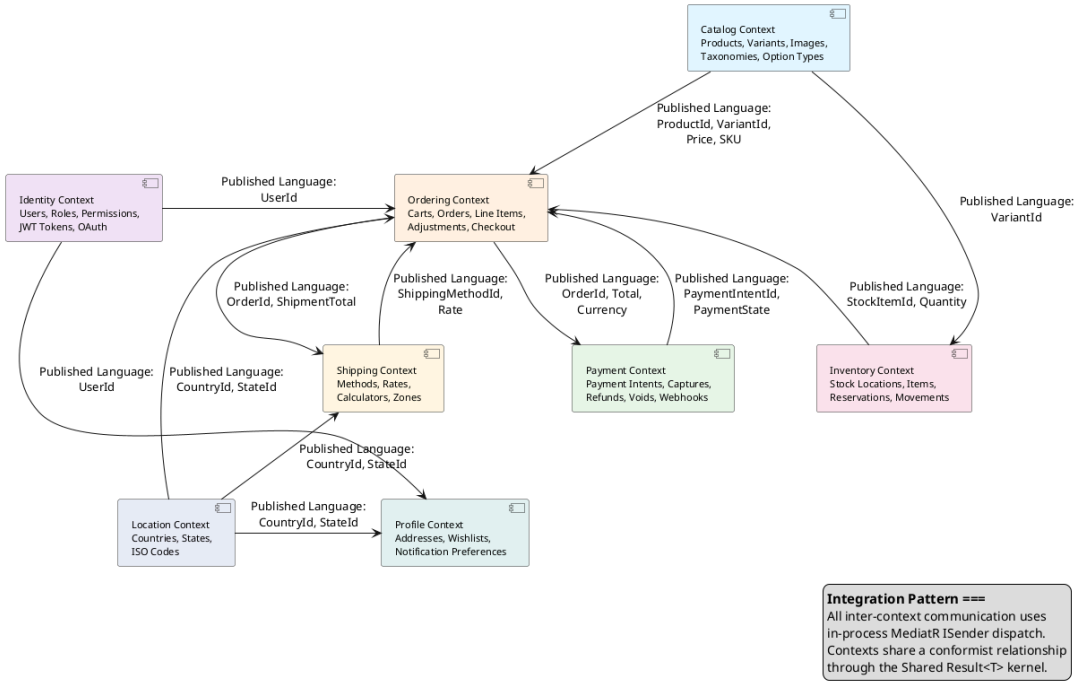

Figure 1.

Figure 1: Bounded Context Map showing the eight business contexts and the Published Language identifiers exchanged between them. All integration uses in-process MediatR dispatch; no context directly references another context's namespace.

Table Table 33 details each context's business responsibility and the integration data it exposes to the system.

Table 33.

Table 33: Bounded context responsibilities and Published Language identifiers. The Published Language column lists the value types that other contexts may reference by identifier only, never by importing the source context's namespace.

Context	**Business Responsibility**	**Published Language**
Catalog	Manages the product lifecycle: creating products with fashion-specific metadata (style code, season, material, department, gender target), defining sellable variants with SKUs and independent pricing, uploading images with automatic embedding generation, and organising products through hierarchical taxonomies.	ProductId, VariantId, Sku, Price, Slug
Ordering	Orchestrates the customer purchase workflow from cart to completed order. Man-	OrderId, OrderNumber, Total, Currency, CheckoutState

Context	Business Responsibility	Published Language
	ages cart with seven-day auto-expiry, forward-only checkout state machine, line items with price snapshots, adjustments, and cancellation at any pre-confirmation stage.	
Payment	Manages payment intent lifecycle, creation, capture, refund, void, across two gateway implementations. Maintains parallel payment state independent of the gateway for offline operations and consistent behaviour across providers.	PaymentIntentId, PaymentState, Amount
Inventory	Tracks physical stock quantities per warehouse, manages temporary reservations during active checkouts to prevent overselling, records auditable stock movements through an append-only ledger, and handles inter-	StockItemId, QuantityOnHand, QuantityReserved

Context	Business Responsibility	Published Language
	warehouse transfers.	
Identity	Provides JWT-based authentication with refresh token rotation and reuse detection, role-based and permission-based authorisation with `domain:category:action` claim format, and guest session management for anonymous browsing.	UserId, Email, PermissionClaim
Profile	Manages user addresses for shipping and billing, wishlists for product bookmarking, and notification preferences controlling email and SMS communication channels.	ProfileId, AddressId
Shipping	Configures delivery methods, standard, express, local pickup, and calculates shipping rates by geographic zone using weight- and distance-based calculators.	ShippingMethodId, Rate
Location	Provides country and	CountryId, StateId, IsoCode

Context	Business Responsibility	Published Language
	state reference data with ISO 3166 codes. This context is read-only reference data shared across Shipping (zone configuration), Profile (address validation), and Ordering (checkout address selection).	

2.3.2.2 Aggregates and Invariants

An aggregate is a cluster of domain objects treated as a single consistency boundary. Each aggregate has a root entity through which all modifications must pass. The root enforces invariants, business rules that must hold true at all times within the aggregate boundary. ReSys.Shop takes a pragmatic approach to DDD: it defines aggregate roots and their invariants explicitly but does not require formal value-object base classes or a dedicated domain-event infrastructure for every operation.

The four most architecturally significant aggregates are described below.

Product (Catalog aggregate root). The Product aggregate encapsulates a product family and all its variants, images, option configurations, and taxonomy classifications. A product may have one or more variants; exactly one is designated as the master variant displayed on listing pages. The aggregate enforces the following invariants: every product must have a unique slug for SEO-friendly URL generation; a product that declares options (such as size or colour) must have at least one option type defined; and the master variant must exist among the product's own variants. Variant images contain the `embedding` column, a 512-dimensional float vector generated by the ML sidecar, enabling cosine similarity search against the entire image corpus.

Order (Ordering aggregate root). The Order aggregate manages the checkout lifecycle from a nascent cart through to a completed purchase. It aggregates line items, each capturing a price snapshot of the variant at the time of purchase, and optional adjustments for discounts or promotions. The aggregate enforces the invariant that `Total = ItemTotal + AdjustmentTotal`

`+ ShipmentTotal`, maintaining financial consistency across all modifications. The checkout state progresses forward only, the address, delivery, payment, and confirmation stages must complete in sequence, and once an order is confirmed (finalised), it becomes immutable except for the cancel transition. This forward-only constraint is encoded in the domain entity itself and validated before every state transition.

PaymentIntent (Payment aggregate root). The PaymentIntent aggregate models the lifecycle of a customer's intent to pay. It is created with a specified amount and currency, and transitions through states, Pending, RequiresAction, Processing, Succeeded, Canceled, and Failed, based on gateway interactions. The aggregate tracks payment captures, where each capture debits a portion of the authorised amount. The system enforces the invariant that the sum of all captures must not exceed the original intent amount. A separate payment capture goes through its own state transitions: Succeeded → Captured → Refunded / Voided. The system maintains its own payment state in parallel with the gateway state, enabling consistent behaviour whether using the production Stripe gateway or the development Bogus gateway.

StockItem (Inventory aggregate root). The StockItem aggregate tracks the physical availability of a product variant at a specific warehouse location. It maintains two quantities: on-hand (physical count from warehouse operations) and reserved (units held for active checkouts). The aggregate enforces the invariant that `QuantityOnHand ≥ 0`, stock cannot go negative. Backorder support allows sales beyond on-hand quantity up to a configured backorder limit, but the system tracks the deficit separately. Quantity changes are not performed directly on StockItem; instead, they must be recorded as StockMovement entries in an append-only ledger, preserving a complete and auditable history of every stock change, including the quantity before and after, the reason, and the operating user.

2.3.2.3 Ubiquitous Language Glossary

A key practice in DDD is establishing a ubiquitous language, a shared vocabulary used by all team members and reflected directly in the codebase. Table Table 34 presents the core terms of the ReSys.Shop domain with their definitions.

Table 34.

Table 34: Ubiquitous Language Glossary: core domain terms, their owning bounded context, and their precise definitions as used throughout the codebase and this thesis.

Term	Context	Definition
Product	Catalog	A product family representing a fashion item concept (e.g., "Cotton T-Shirt"). Holds shared metadata: description, slug, status, fashion-specific attributes, and taxonomy classifications. Does not have a price or SKU directly, those belong to Variants.
Variant	Catalog	A sellable, physical unit of a product (e.g., "Cotton T-Shirt, Red, Large"). Holds SKU, barcode, pricing, inventory tracking flag, and physical dimensions. Each variant belongs to exactly one product.
Master Variant	Catalog	The designated default variant shown on product listing pages. Every product with variants must have exactly one master variant.
Option Type	Catalog	Defines a configurable attribute class such as "Colour", "Size", or "Material". Option types are product-independent and reusable across the catalogue.
Taxonomy / Taxon	Catalog	A hierarchical categorisation tree for organising products. A Taxonomy (e.g., "Department") contains Taxons (e.g., "Clothing" → "Dresses" → "Evening Dresses") forming a nested set structure.
Cart	Ordering	An ephemeral collection of line items that represents a customer's intent to purchase. Carts automatically expire after seven days of inactivity. The cart is the initial state of the Order aggregate.
Line Item	Ordering	A single entry in an order or cart, referencing a specific product variant, quantity, and a price snapshot captured at the time of purchase to insulate historical orders from catalogue price changes.
Checkout State	Ordering	The sequential stage of the purchase process: Address → Delivery → Payment → Confirm → Complete. Progression is forward-only; cancellation is available from any pre-confirmation stage.
Payment Intent	Payment	A data structure representing a customer's intent to pay a specified amount. Follows a defined state machine through the gateway interaction lifecycle.
Payment Capture	Payment	A partial or full debit against a payment intent. Multiple captures can be performed against a single intent (e.g., capturing shipping cost after items ship).
Stock Item	Inventory	The current state of a product variant's availability at a specific warehouse. Maintains on-hand and reserved quantity counters with optimistic concurrency control to prevent overselling.
Stock Movement	Inventory	An immutable ledger entry recording every change to a stock item's quantity, the delta, the balance before and after, the reason, and the operating user. The single source of truth for inventory changes.

Term	Context	Definition
Refresh Token	Identity	A long-lived credential used to obtain new short-lived access tokens without re-authentication. Each token is single-use; presenting a previously consumed token triggers revocation of all tokens for that user.

2.3.2.4 State Machines

Two explicit state machines govern the most critical transactional workflows in the system: the order checkout process and the payment intent lifecycle. Both are encoded in domain entities, validated before every state transition, and drive the sequence of user-facing and system-level actions.

2.3.2.4.1 Order Checkout State Machine

The order checkout state machine enforces a forward-only progression through five sequential states: Address, Delivery, Payment, Confirm, and Complete. Each state transition is triggered by a specific user action and validated by the domain entity before being committed. Figure Figure 2 depicts this lifecycle.

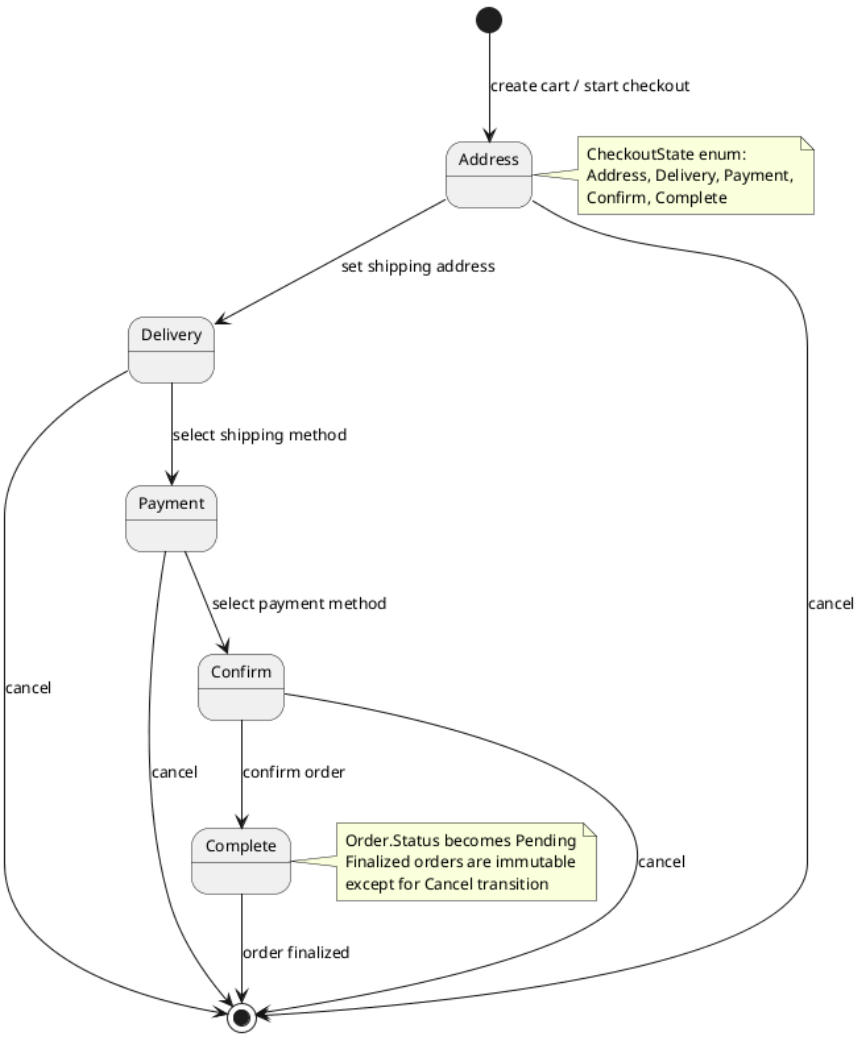

Figure 2.

Figure 2: Order checkout state machine: five sequential states with cancellation available from any pre-confirmation state. The forward-only constraint prevents regressing to earlier checkout stages.

The customer begins by providing a shipping address (Address state), selects a delivery method (Delivery state), chooses a payment method (Payment state), reviews the complete order summary (Confirm state), and finalises the purchase (Complete state). At any point before the Complete state, from Address through Confirm, the customer may cancel the checkout process, which terminates the order without financial consequence.

Once an order reaches the Complete state, it becomes finalised: the order record transitions to Pending status, inventory quantities are reserved for each line item, and the payment intent is processed. From this point forward, the order is immutable except for the cancel transition, which captures the cancellation timestamp and releases reserved inventory. This forward-only design ensures

that at every stage of the checkout pipeline, the system can unambiguously determine the customer's position and the next required action.

2.3.2.4.2 Payment Intent State Machine

The payment intent state machine models the full lifecycle of a payment from creation through to terminal completion, reflecting the state transitions of the Stripe payment gateway while maintaining a parallel system-managed state for offline consistency. Figure Figure 3 shows all states and transitions.

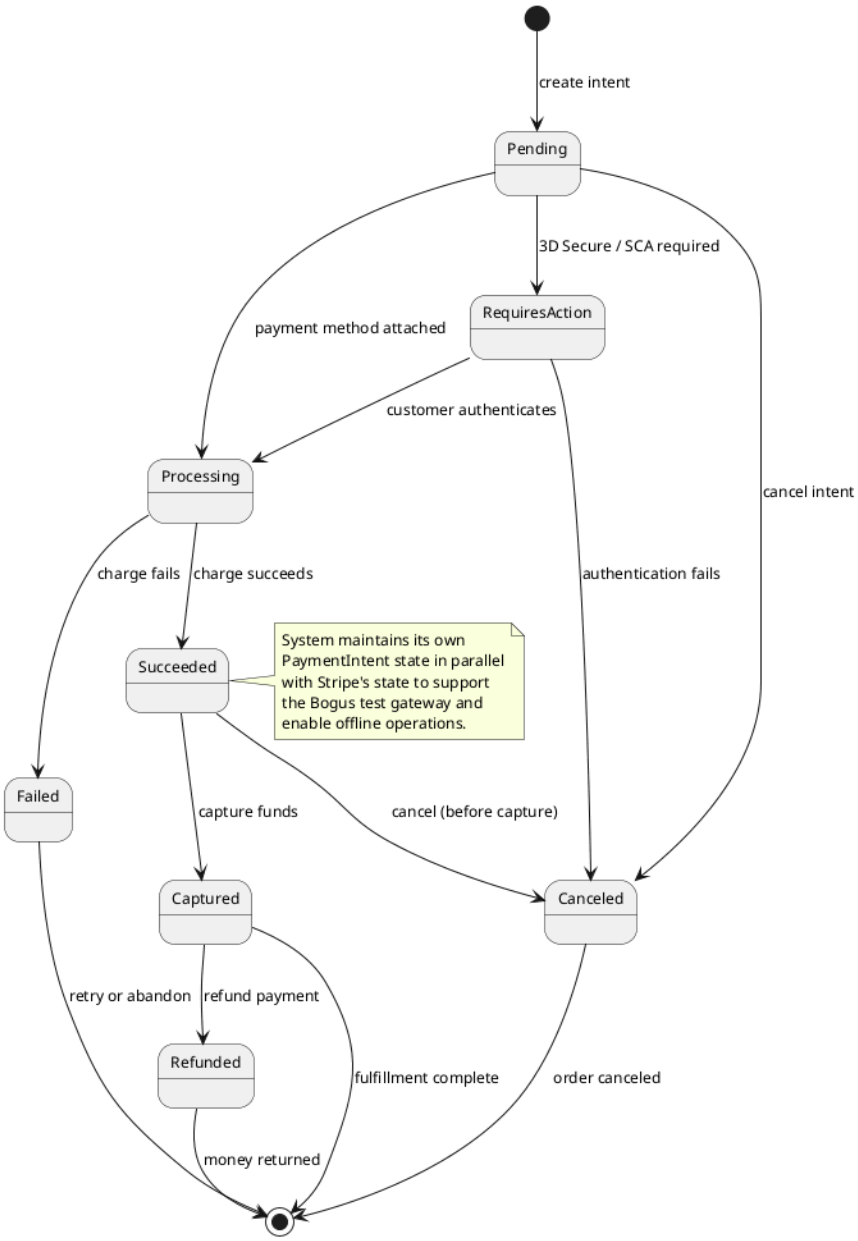

Figure 3.

Figure 3: Payment intent lifecycle: the state machine reflects Stripe gateway states while maintaining a parallel system copy for offline operations and Bogus gateway compatibility. Terminal states are Failed, Canceled, and Refunded.

A payment intent is created in the Pending state. It may transition directly to RequiresAction when the payment requires 3D Secure or Strong Customer Authentication, or to Processing when the payment method has been attached without additional authentication challenges. From RequiresAction, successful customer authentication advances the intent to Processing; failure or timeout moves it to Canceled.

From Processing, a successful charge transitions the intent to Succeeded, while a declined or errored charge moves it to Failed. From Succeeded, the funds may be captured, transferring them from the customer's account, or the intent may be cancelled before capture. A captured intent may later be refunded, returning funds to the customer and reaching the terminal Refunded state.

The system maintains its own copy of the payment state in parallel with the gateway's representation. This design decision serves two purposes. First, it enables the Bogus test gateway, a development-only implementation that simulates payment lifecycles without external calls, to operate against the same domain entities as the production Stripe gateway. Second, it allows the system to reason about payment state offline without querying the gateway API, which improves resilience during network interruptions and reduces external dependency during business operations.

2.3.3 C4 Architecture

The C4 model provides a structured approach to describing software architecture at four levels of abstraction: system context, container, component, and code. This section presents the first three levels for ReSys.Shop, omitting the code-level view as it falls within the scope of the implementation chapter. A deployment diagram complements the C4 views by showing the physical infrastructure.

2.3.3.1 System Context

The system context diagram positions ReSys.Shop within its environment, showing the human users who interact with the platform and the external systems on which it depends. Figure Figure 4 presents this highest-level view.

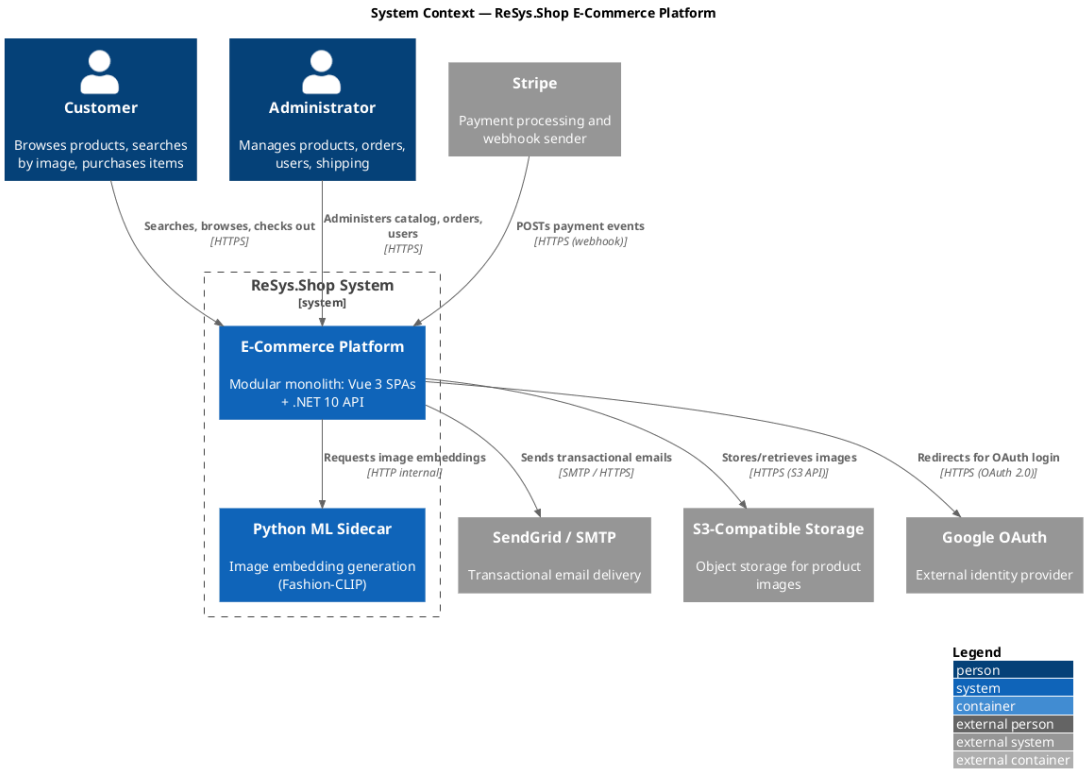

Figure 4.

Figure 4: System Context diagram: ReSys.Shop as a single system boundary with customer and administrator users on one side and external payment, email, storage, identity, and ML services on the other. The modular monolith internally handles all eight business domains within one boundary.

Two categories of human users interact with the platform. Customers browse the product catalogue, perform visual and keyword searches, manage a shopping cart, and complete multi-step checkout, all through the Vue 3 storefront SPA. Administrators manage the full product lifecycle, process orders, monitor inventory levels, and administer user accounts through the Vue 3 admin SPA.

The system depends on five external services. Stripe processes payment intents and sends webhook notifications when payment events occur, the backend validates these webhooks using Stripe's signature verification before acting on them. SendGrid delivers transactional emails such as order confirmations, password reset links, and shipping notifications. An S3-compatible object store persists product images uploaded through the admin interface. Google OAuth provides an alternative authentication path, allowing customers to sign in using their Google credentials. The Python ML Sidecar, deployed as a companion service within the Aspire orchestration boundary, generates image embeddings used by the catalogue's visual search feature.

2.3.3.2 Container

The container diagram decomposes ReSys.Shop into its deployable units, the processes and data stores that together constitute the running system. Figure Figure 5 presents this view.

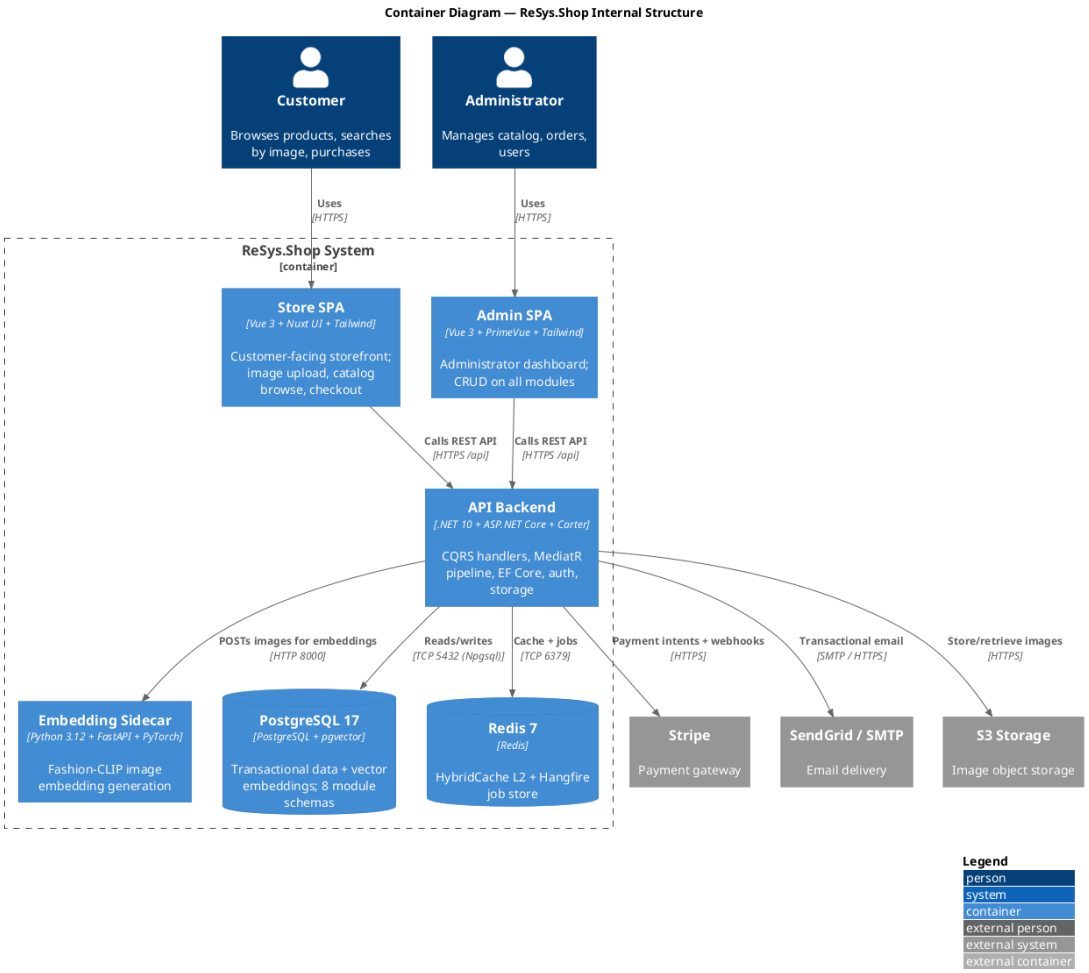

Figure 5.

Figure 5: Container diagram showing the deployable units of ReSys.Shop: two Vue 3 SPAs, the .NET 10 API backend, the Python ML sidecar, PostgreSQL with pgvector, and Redis. Arrows indicate communication protocols between containers.

The system comprises six deployable containers. The Store SPA and Admin SPA, both Vue 3 applications served as static assets, handle all user interface concerns and communicate with the backend exclusively through the REST API over HTTPS. The API Backend, a .NET 10 application running on ASP.NET Core, contains all business logic across eight modules, exposes Carter minimal API endpoints, and orchestrates the MediatR CQRS pipeline for command and query processing. The Embedding Sidecar, a Python 3.12 FastAPI application, loads machine learning models into GPU or CPU memory and exposes HTTP endpoints for generating image embeddings.

Two persistent data stores support the platform. PostgreSQL 17 with the pgvector extension serves as the primary database, storing both relational transactional data across eight module-specific schemas and high-dimensional vector embeddings for visual similarity search. Redis 7 fills a dual role: as the second-level distributed cache backing the HybridCache abstraction, and as the persistent job store for Hangfire background job processing, enabling cart expiry, webhook dispatch, and periodic maintenance tasks to survive application restarts.

The communication topology reflects deliberate design constraints. The Vue SPAs call the backend synchronously over HTTPS, never directly accessing the database or external services, which ensures all security policies and data validation are enforced server-side. The backend communicates with PostgreSQL and Redis over internal TCP connections, with the ML sidecar over HTTP on the internal Docker network, and with external services over HTTPS. This design centralises all external integration through the backend container, simplifying security management and operational monitoring.

2.3.3.3 Component

The component diagram zooms into the API Backend container, revealing its internal structure: the modules, framework services, and cross-cutting concerns that compose the .NET application. Figure Figure 6 presents this view.

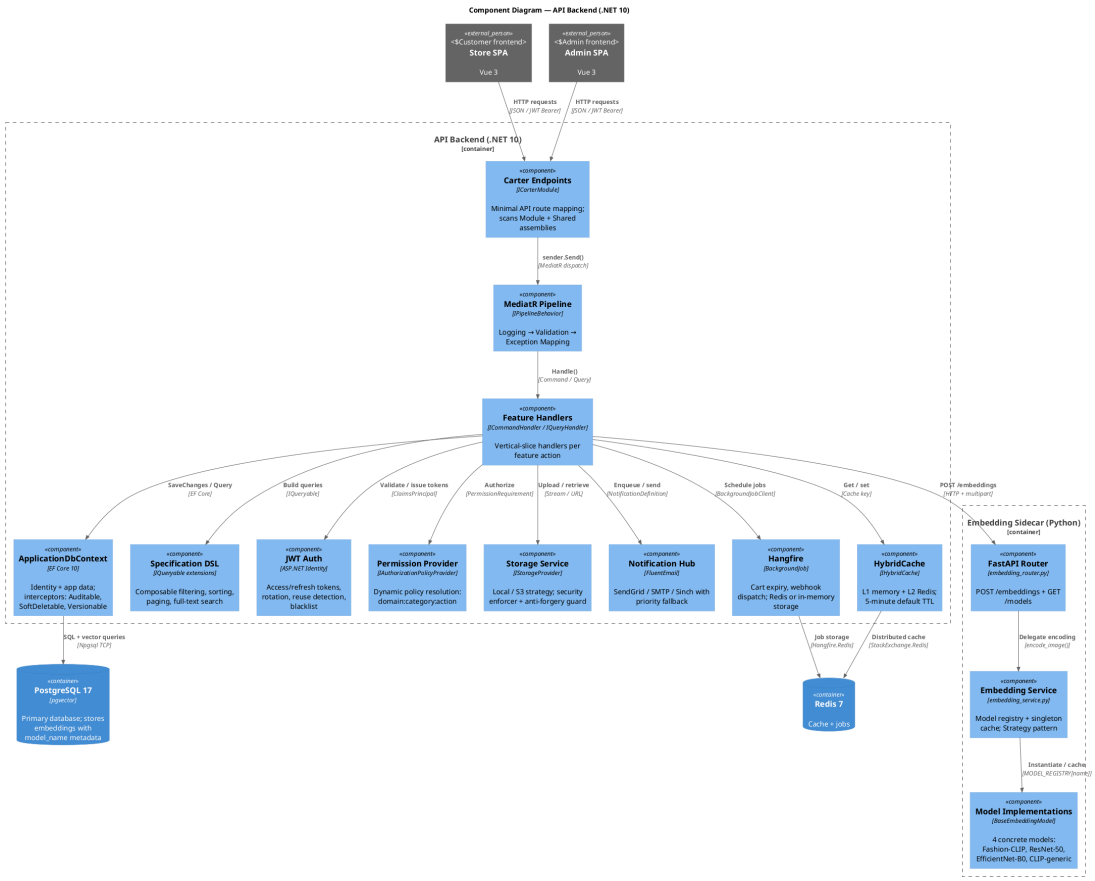

Figure 6.

Figure 6: Component diagram of the API Backend showing the Carter endpoint layer, the MediatR pipeline, the feature handlers, and eight supporting infrastructure components. The Python ML Sidecar is shown with its internal three-layer architecture alongside.

The API Backend is structured as a pipeline. HTTP requests arrive at the Carter endpoints, which are minimal API route groups registered by `ICarterModule` implementations in each module's feature folder. The endpoints are thin, they extract request parameters, dispatch a command or query via `ISender`, and map the `Result<T>` response to an HTTP status code and JSON body. All business logic resides in the feature handlers.

The MediatR pipeline wraps every request with a chain of behaviours: logging captures the request type and timing, validation executes FluentValidation rules before the handler runs, and exception mapping converts unhandled infrastructure failures to standardised problem details. The handlers themselves interact with eight infrastructure components:

- ApplicationDbContext (EF Core 10) with interceptors for auditable timestamps, soft-delete filtering, and row-version concurrency checks.
- A Specification DSL that provides composable `IQueryable` extensions for filtering, sorting, paging, and full-text search, keeping handler code free of query-building boilerplate.
- JWT authentication with ASP.NET Identity, managing access and refresh token issuance, rotation, reuse detection, and token blacklisting.
- A dynamic permission provider that resolves `{domain}:{category}:{action}` permission claims to authorisation policies at runtime without requiring static policy registration.
- A storage service with interchangeable providers (local filesystem or S3-compatible storage) selected via configuration, with built-in file-type validation and anti-forgery guards on uploads.
- A notification hub supporting email (SendGrid/SMTP) and SMS (Sinch) channels with configurable fallback priority.
- Hangfire for background job scheduling and processing, handling cart expiry, webhook dispatch, and periodic health checks.
- HybridCache with two-tier caching: L1 in-memory for sub-millisecond access and L2 Redis for cross-instance consistency.

The Python ML Sidecar follows a three-layer architecture: the FastAPI router handles HTTP request validation and API key authentication, the Embedding Service maintains a singleton model registry with lazy loading and caching, and the model implementations, Fashion-CLIP, ResNet-50, EfficientNet-B0, and generic CLIP, implement a common strategy interface for interchangeable inference backends.

2.3.3.4 Deployment

The deployment diagram illustrates how the containers map to physical or virtual infrastructure in a production configuration. Figure Figure 7 shows the deployment topology.

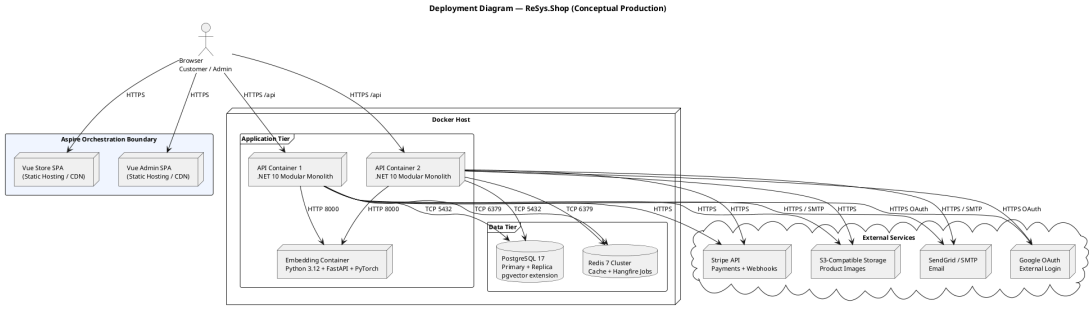

Figure 7.

Figure 7: Deployment diagram showing containerised services within an Aspire orchestration boundary. The API backend is horizontally scalable; the embedding sidecar is stateless; Redis enables distributed state across API replicas.

All services are containerised and orchestrated by .NET Aspire, which manages service discovery, configuration injection, and health monitoring during both development and production deployments. The Vue SPAs are served as static bundles from a CDN or reverse proxy, while the backend services run within Docker containers on a single host or across a cluster.

The API backend is horizontally scalable, multiple container instances share PostgreSQL and Redis, enabling round-robin request distribution. The embedding sidecar is stateless: it loads models into memory on startup, caches them, and serves embedding requests without shared state. Any API instance can call any embedding container. Redis provides the distributed state needed for cache coherence and Hangfire job coordination across API replicas.

PostgreSQL is configured with a primary instance for writes and one or more read replicas for reporting and analytical queries. The pgvector extension is installed on both primary and replicas, enabling vector similarity search from any read path. External services, Stripe, SendGrid, S3 storage, and Google OAuth, are accessed over HTTPS from every API instance, with credentials managed through Aspire's configuration system and never baked into container images.

2.3.4 Database Design

The ReSys.Shop database is a single PostgreSQL 17 instance organised into per-context schemas, each owned by a bounded context and managed through Entity

Framework Core migrations. This section describes the schema organisation, the core entity-relationship model, the pgvector integration for visual search, and the key design decisions that shape the data layer.

2.3.4.1 Schema Organisation

Each of the eight bounded contexts owns its database schema. The Catalog context manages tables for products, variants, variant images, option types, option values, taxonomies, and taxons. The Ordering context owns orders, line items, adjustments, shipments, and inventory units. The Payment context holds payment intents, payment captures, and payment logs. The Inventory context manages stock locations, stock items, stock movements, and stock reservations. The Identity context, built on ASP.NET Identity Core, manages users, roles, user roles, refresh tokens, and external login providers. The Profile context owns user addresses, wishlists, and notification preferences. The Shipping context manages shipping methods, shipping rates, and shipping zones. The Location context provides country and state reference tables.

Cross-context relationships are implemented through identifier references only, there are no foreign key constraints spanning module boundaries. An Order references a UserId (Identity context) and a VariantId (Catalog context), but these are stored as UUIDs without database-level referential integrity constraints. This design preserves the logical isolation of each context as a separate schema while keeping all data in a single database, avoiding the complexity of distributed transactions for the most common business operations.

Entity Framework Core manages all migrations from a dedicated Migrations assembly. Each migration is generated by comparing the current database state against the domain entity model, and migrations are applied as part of the application startup or through standalone migration scripts for production deployments.

2.3.4.2 Core Entity-Relationship Model

Figure Figure 8 presents the entity-relationship diagram for the core business entities across the Catalog and Ordering domains, the two contexts that participate most directly in the visual search and checkout workflows.

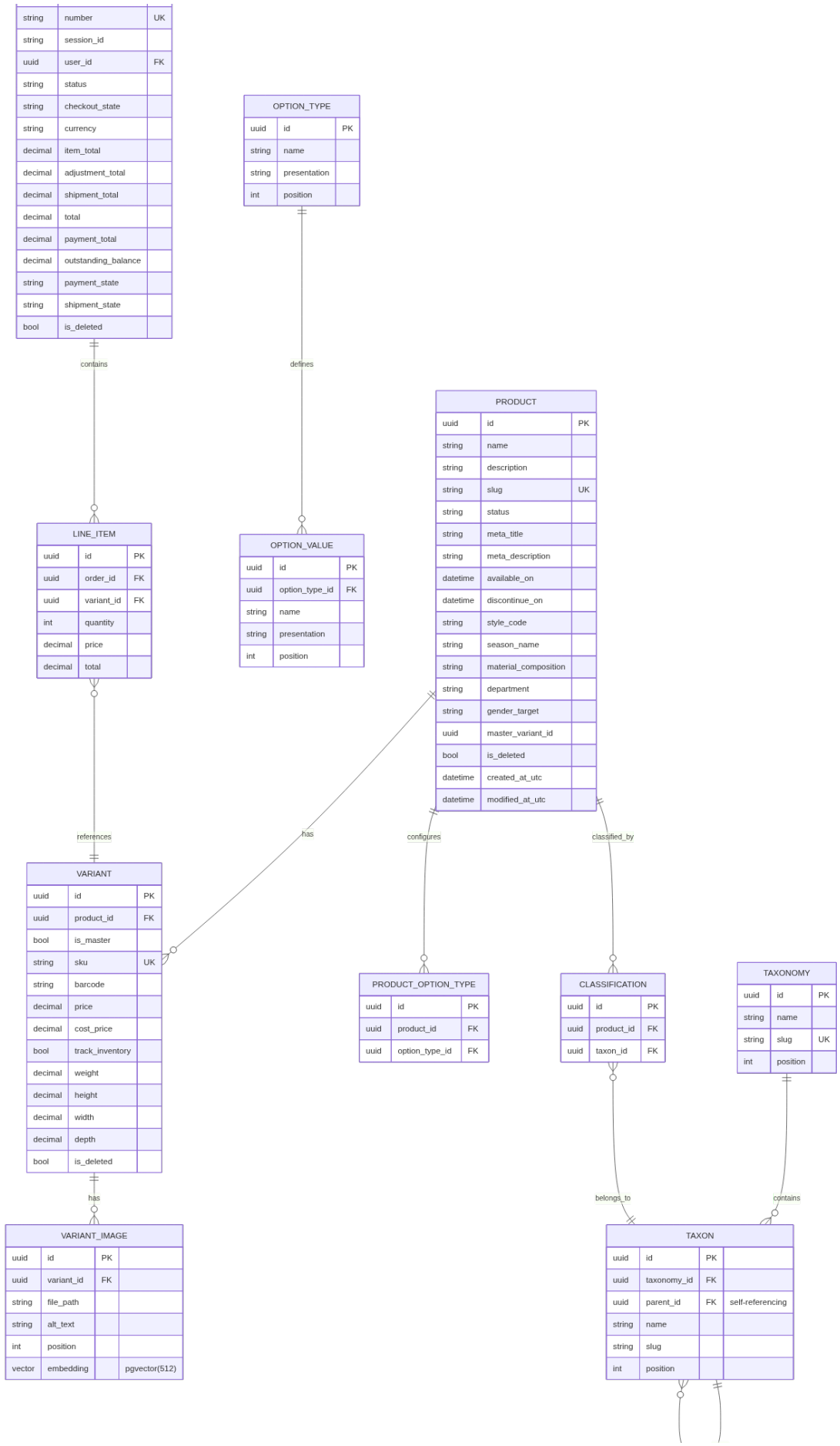
string number UK
string session_id
uuid user_id FK
string status
string checkout_state
string currency
decimal item_total
decimal adjustment_total
decimal shipment_total
decimal total
decimal payment_total
decimal outstanding_balance
string payment_state
string shipment_state
bool is_deleted
contains
OPTION_TYPE
uuid id PK
string name
string presentation
int position
defines
PRODUCT
uuid id PK
string name
string description
string slug UK
string status
string meta_title
string meta_description
datetime available_on
datetime discontinue_on
string style_code
string season_name
string material_composition
string department
string gender_target
uuid master_variant_id
bool is_deleted
datetime created_at_utc
datetime modified_at_utc
LINE_ITEM
uuid id PK
uuid order_id FK
uuid variant_id FK
int quantity
decimal price
decimal total
OPTION_VALUE
uuid id PK
uuid option_type_id FK
string name
string presentation
int position
references
has
configures
classified_by
VARIANT
uuid id PK
uuid product_id FK
bool is_master
string sku UK
string barcode
decimal price
decimal cost_price
bool track_inventory
decimal weight
decimal height
decimal width
decimal depth
bool is_deleted
PRODUCT_OPTION_TYPE
uuid id PK
uuid product_id FK
uuid option_type_id FK
CLASSIFICATION
uuid id PK
uuid product_id FK
uuid taxon_id FK
TAXONOMY
uuid id PK
string name
string slug UK
int position
has
belongs_to
contains
VARIANT_IMAGE
uuid id PK
uuid variant_id FK
string file_path
string alt_text
int position
vector embedding pgvector(512)
TAXON
uuid id PK
uuid taxonomy_id FK
uuid parent_id FK self-referencing
string name
string slug
int position

Figure 8.

Figure 8: Core entity-relationship diagram showing the primary domain entities and their relationships across the Catalog and Ordering bounded contexts. Soft deletion, audit columns, and GUID primary keys are used throughout.

The Catalog domain centres on the Product entity, which has a one-to-many relationship with Variant. Each Variant represents a specific sellable configuration, for example, “Cotton T-Shirt, Red, Large”, with its own SKU, pricing, and inventory tracking flag. Exactly one variant per product is designated as the master variant. Variants relate one-to-many to VariantImages, each of which holds a file path and an optional `embedding` column of type `vector(512)` for pgvector similarity search. Products configure option types, such as Colour or Size, which in turn define option values. Taxonomies contain taxons in a hierarchical structure, with taxons referencing themselves through a parent foreign key to represent tree structures such as “Clothing → Dresses → Evening Dresses”. Products are classified by taxons through a many-to-many classification table.

The Ordering domain centres on the Order entity, which has a one-to-many relationship with LineItem. Each line item references a specific variant, not a product directly, because customers purchase specific configurations. The line item captures a price snapshot at the time of purchase, ensuring that historical orders are unaffected by catalogue price changes. Orders are related to users through a UserId reference, optionally nullable to support guest checkout, and track a session identifier for mapping anonymous carts before authentication.

2.3.4.3 pgvector Integration

PostgreSQL’s pgvector extension enables vector similarity search directly within the relational database, eliminating the need for a separate vector database. The `variant_images` table contains an `embedding` column of type `vector(512)`, a fixed-length array of 512 IEEE 754 single-precision floating-point numbers representing the visual features extracted from the image by the ML sidecar.

An HNSW (Hierarchical Navigable Small World) index is created on the embedding column to accelerate approximate nearest-neighbour searches. HNSW provides logarithmic search complexity, enabling sub-10-millisecond queries over tens of thousands of vectors. The index is configured for cosine distance, the recommended distance metric for normalised embeddings from CLIP-family models.

Vector similarity queries use the cosine distance operator (`<=>`), which computes the angular distance between two vectors. A representative conceptual query pattern is: retrieve all variant images, compute the cosine distance between each stored embedding and the query embedding, order by ascending distance,

and return the top results. The system filters results by a configurable minimum similarity threshold, computed as 1 - cosine_distance, defaulting to 0.7, a level at which fashion images typically exhibit perceptible visual similarity.

Each embedding row includes a `model_name` column identifying which ML model generated the vector. This metadata enables per-model filtered queries: when the system switches the active embedding model, only embeddings from that model's columns participate in similarity search, preventing cross-model vector comparisons that would produce meaningless results. The model name also supports the benchmark evaluation in Chapter 3, where multiple models are compared against the same image corpus.

2.3.4.4 Key Design Decisions

Several design decisions govern the database layer and influence every bounded context:

Universally Unique Identifiers. All primary keys are UUIDs (GUIDs in .NET terminology). This decision eliminates reliance on a central sequence generator, allows safe distributed key generation, useful for client-side offline creation of cart or draft-order records, and prevents the information leakage inherent in auto-incrementing integer primary keys.

Soft Deletion. Most domain entities carry an `IsDeleted` boolean column. Entity Framework Core applies a global query filter that excludes soft-deleted rows from all standard queries. This preserves referential integrity, a deleted product does not orphan its historical orders, while keeping the active working set clean. Deleted entities can be restored by administrators if the deletion was in error.

Audit Columns. Every entity inherits `CreatedAtUtc` and `ModifiedAtUtc` timestamp columns from the `Entity` base class, populated automatically by an EF Core save interceptor. These columns support chronological analysis of catalogue changes, order lifecycle auditing, and debugging data anomalies without external logging correlation.

Composite Indexes. Tables serving high-frequency query patterns carry composite indexes tuned to their access patterns. The Orders table includes `(UserId, Status)` for listing a customer's orders filtered by state, and `(SessionId, Status)` for resolving anonymous carts. These composite indexes avoid the need for separate single-column indexes and reduce disk footprint.

Variable Vector Dimensions. Different embedding models produce vectors of different dimensionalities: 384 for DINOv2-S, 512 for Fashion-CLIP and CLIP, 768 for DINOv2-B, 1280 for EfficientNet-B0, and 2048 for ResNet-50. PostgreSQL's `vector` type supports this variability, the dimension is part of the column type declaration, and HNSW indexes are built per-dimension. The

system stores embeddings from all models in separate rows, each tagged with the model name, and queries against the active model's dimension only.

2.3.4.5 Per-Context Schema Description

The Identity context stores user accounts with Argon2-hashed passwords, security stamps for session invalidation, and refresh tokens with one-time-use semantics. Role and UserRole tables implement RBAC, while UserLogin records link local accounts to external OAuth providers. UserAddresses store shipping and billing locations with ISO country codes.

The Catalog context's Product table carries fashion-specific metadata columns, style code, season name, material composition, department, and gender target, alongside standard catalogue fields. Variants hold SKU, barcode, cost and selling price in decimal precision, and physical dimensions for shipping calculations. The OptionType and OptionValue tables implement an EAV-lite pattern, allowing administrators to define new product attributes without schema migrations. Taxons use a nested set model with left and right bounds, enabling single-query subtree retrieval for mega-menus and breadcrumb navigation.

The Ordering context stores financial values as decimal with 18-digit precision and 2-decimal scale, avoiding floating-point accumulation errors in tax and discount calculations. Orders maintain separate subtotals for items, adjustments, and shipping, with the total computed as their sum. LineItems capture price snapshots at purchase time, decoupling order history from catalogue changes. OrderHistory provides an append-only audit trail of every state transition.

The Inventory context tracks stock items at a variant-location granularity, with `QuantityOnHand` and `QuantityReserved` maintained as separate counters and protected by optimistic concurrency control using PostgreSQL's `xmin` system column. StockMovements form an immutable ledger, every quantity change, whether from receiving, selling, returning, or stock-taking, is recorded with the delta, balances before and after, unit cost, and an external reference such as an order number or purchase order identifier.

2.3.5 API Design

The ReSys.Shop API exposes a RESTful interface built on Carter minimal APIs and organised around the MediatR CQRS pattern. This section describes the API architecture, the endpoint organisation scheme, and a summary of the key endpoints that define the platform's external contract.

2.3.5.1 API Architecture

The API layer acts as a thin orchestration boundary. It contains no business logic; instead, it delegates all processing to the MediatR pipeline. Each request follows a consistent path: the Carter endpoint receives the HTTP request, extracts

route and body parameters, constructs a MediatR command or query object, dispatches it through `ISender`, and maps the returned `Result<T>` to an HTTP response. This design keeps endpoints concise, typically six to twelve lines, and concentrates all domain logic in the handler layer, where it is testable without HTTP infrastructure.

Carter modules group related endpoints by module and surface. Each module (Catalog, Ordering, Payment, and so on) registers its own `ICarterModule` implementation, which defines the route groups, HTTP methods, and parameter bindings for that module's endpoints. This modular registration avoids a single monolithic route configuration file and enables each bounded context to own its API surface.

FluentValidation provides input validation through validator classes associated with each command and query. Validators run automatically as part of the MediatR pipeline behaviour, before the handler executes, ensuring that handlers never receive invalid input. Validation failures return standardised `400 Bad Request` responses with field-level error details.

2.3.5.2 Endpoint Organisation

Endpoints are organised by two dimensions: the business module that owns the operation, and the surface, Admin or Storefront, that serves as the entry point. The URL pattern follows the convention `/api/{module}/{surface}/{action}`, where module identifies the owning bounded context, surface distinguishes administrative from customer-facing operations, and action names the specific operation.

This two-dimensional organisation serves several purposes. It makes the API self-documenting: the URL alone communicates which business area and which user role the endpoint targets. It simplifies authorisation: Admin surface endpoints share a common authorisation policy requiring an administrator role, while Storefront endpoints apply corresponding customer-level policies. And it enables independent versioning: a module can evolve its endpoints without affecting other modules.

Table Table 35 summarises the most architecturally significant endpoints across the platform. These endpoints represent the primary user-facing capabilities, visual search, catalogue browsing, checkout, order history, authentication, and payment, that together define the complete customer and administrator experience.

Table 35.

Table 35: Key API endpoints representing the primary user-facing capabilities of the platform. All endpoints in the Storefront surface serve customer interactions; Admin surface endpoints (not shown) mirror these with full CRUD capabilities on all modules.

Endpoint	Module	Surface	Description
`POST /api/catalog/storefront/search-by-image`	Catalog	Storefront	Accepts an uploaded image file, sends it to the ML sidecar for embedding generation, queries pgvector for the nearest neighbour variant images by cosine similarity, and returns matching products ranked

Endpoint	Module	Surface	Description
			by similarity score with variant thumbnails, pricing, and product URLs.
`GET /api/catalog/storefront/products/{slug}`	Catalog	Storefront	Returns a product with all its published variants, images, option configurations, and taxonomy classifications, identified by

Endpoint	Module	Surface	Description
			its URL slug. Supports guest access for anonymous browsing.
`POST /api/ordering/storefront/cart/checkout`	Ordering	Storefront	Advances the cart through the checkout state machine: setting the shipping address, selecting the delivery method, and confirming the order. Each call transitions

Endpoint	Module	Surface	Description
			the checkout state forward one step.
`GET /api/ordering/storefront/orders/{id}`	Ordering	Storefront	Returns the complete order with line items, payment state, shipment state, and status history. Requires authentication; customers may only access their own orders.
`POST /api/identity/store/auth/login`	Identity	Storefront	Authenticates

Endpoint	Module	Surface	Description
			a user by email and password, returning a JWT access token (fifteen-minute lifetime) and a refresh token for token rotation. Supports Google OAuth as an alternative login method via a related

Endpoint	Module	Surface	Description
			endpoint.
`POST /api/payment/storefront/payment/create-intent`	Payment	Storefront	Creates a payment intent for the specified order amount and currency, initialising the payment state machine.

The admin surface provides full CRUD operations on all module entities, products, variants, orders, inventory, users, shipping methods, and location data, following the same URL pattern with the Admin surface prefix. These endpoints are excluded from the table to maintain focus on the core platform capabilities, but they follow identical architectural patterns: minimal API route groups, MediatR dispatch, FluentValidation, and permission-based authorisation.

2.3.6 Security Design

The security architecture of ReSys.Shop addresses three layers: authentication, verifying the identity of callers, authorisation, controlling what authenticated callers may do, and hardening, defensive measures against common attack vectors. This section describes each layer in turn.

2.3.6.1 Authentication

The platform uses JSON Web Tokens (JWT) for bearer token authentication. Upon successful login, via email and password or Google OAuth, the server issues two tokens: an access token with a fifteen-minute lifetime and a refresh token with a longer lifetime. The access token carries the user's identifier, email, and permission claims in a compact signed payload. All authenticated API requests include the access token in the `Authorization` header as a Bearer token.

The refresh token is a long-lived credential stored server-side in the database. When the access token expires, the client presents the refresh token to obtain a new access token and a new refresh token, a pattern known as refresh token rotation. Each refresh token is single-use: upon successful rotation, the consumed token is marked as used and a replacement is issued. If a previously consumed refresh token is presented again, indicating a potential token theft scenario, the system revokes all tokens for that user, forcing re-authentication. This rotation-with-reuse-detection pattern limits the damage window of a compromised refresh token to the interval between rotations.

Guest users, customers who have not yet authenticated, are assigned a session identifier stored in a browser cookie. This session identifier links their anonymous cart to their browsing context and persists across page navigations. Upon registration or login, the anonymous cart is merged with the authenticated user's cart, preserving the shopping intent built during the guest session.

2.3.6.2 Authorisation

Authorisation is implemented through two complementary mechanisms: role-based access control (RBAC) for broad category restrictions and permission-based claims for fine-grained control.

Roles, such as Customer and Administrator, segregate the Admin and Storefront surfaces. An endpoint in the Admin surface requires the Administrator role; a customer presenting valid credentials without that role receives a `403 Forbidden` response. This coarse check prevents unauthorised access to administrative functions at the infrastructure level, before any business logic executes.

Permissions use a structured claim format: `{domain}:{category}:{action}`. For example, `catalog:products:create` grants permission to create products in the Catalog domain. A dynamic permission provider, `IAuthorizationPolicyProvider`, resolves these claim strings to ASP.NET Core authorisation policies at runtime, eliminating the need for static policy registration for every endpoint. This dynamic resolution enables permission configuration through the database without redeployment: an administrator may create a new role, assign it a set of permission claims, and those permissions take effect across all authorised endpoints immediately.

2.3.6.3 Security Measures

Several defensive measures harden the platform against common web application attack vectors.

Rate Limiting. Authentication endpoints are rate-limited to five requests per minute per IP address to prevent credential brute-forcing. Registration endpoints are limited to three requests per hour per IP address to deter automated account creation. Payment endpoints are limited to thirty requests per minute to maintain availability during high-traffic checkout events.

Security Headers. All HTTP responses include security headers configured through ASP.NET Core middleware: Content-Security-Policy restricts script and style sources to the application's own domains, HTTP Strict-Transport-Security enforces HTTPS-only connections for a configurable duration, X-Frame-Options prevents the application from being embedded in iframes to block clickjacking, and X-Content-Type-Options prevents MIME-type sniffing by browsers.

File Upload Validation. The visual search and product image upload endpoints enforce strict file validation. Uploaded files undergo magic-byte verification, inspecting the file header bytes rather than trusting the file extension, to confirm they are valid JPEG, PNG, or WebP images. A ten-megabyte size limit prevents resource exhaustion from oversized uploads. Server-side validation repeats the client-side checks, as client-side validation is a convenience that an attacker can bypass.

Payment Webhook Verification. The Stripe webhook endpoint, which receives payment event notifications, validates each incoming request using Stripe's signature verification algorithm. The webhook payload is hashed with a shared signing secret; if the computed signature does not match the one provided in the Stripe-Signature header, the request is discarded before any business logic processes it. This verification prevents spoofed webhook payloads from injecting fraudulent payment state into the system.

2.3.6.4 Token Flow

The authentication token lifecycle operates as follows. A client authenticates with email and password, receiving an access token and a refresh token. The access token is short-lived and not stored server-side; it is validated by signature verification and expiration check on each request. When the access token expires, the client sends the refresh token to the refresh endpoint. The server validates the refresh token against the database: if it is valid and has not been used before, the server marks it as consumed, issues a new access token and a new refresh token, and returns both to the client. If the presented refresh token has already been consumed, flagged as used from a previous rotation, the server assumes token theft and revokes all refresh tokens associated with that user,

logging the security event. The user must then re-authenticate, which invalidates the compromised token chain and issues fresh credentials. This model provides a self-healing defence against refresh token interception without requiring the user to detect or report the compromise.

2.3.7 Summary

This section has presented the architectural design of ReSys.Shop across six dimensions. The service-oriented system architecture separates presentation (Vue 3), business logic (.NET 10), and machine learning (Python sidecar) into independently deployable services. Domain-Driven Design partitions the business domain into eight bounded contexts communicating through MediatR in-process dispatch, with four architecturally significant aggregate roots enforcing explicit invariants. The C4 model describes the system at context, container, and component levels of abstraction, revealing the communication paths between deployable units and the internal composition of the .NET backend. The PostgreSQL database uses per-context schemas, pgvector for vector similarity search, and a set of consistent design decisions, GUIDs, soft deletion, audit columns, applied across all contexts. The API layer follows the URL convention `/api/{module}/{surface}/{action}` with Carter minimal APIs and MediatR CQRS. The security architecture covers JWT authentication with refresh token rotation, permission-based authorisation with dynamic policy resolution, and layered defensive measures against common attack vectors. Together, these architectural decisions provide the foundation on which the implementation described in the following section is built.

2.4 IMPLEMENTATION

This section describes the concrete realisation of the ReSys.Shop system architecture presented in Section 2.2. It is organised into five sub-sections, progressing from the architectural pattern that structures the codebase, through the machine learning pipeline that constitutes the core research contribution, to the end-to-end visual search flow and its configuration mechanism, and finally a concise survey of the supporting e-commerce modules. The implementation follows the service-oriented design established in Section 2.2: a Vue 3 frontend, a .NET 10 modular monolith backend, and a Python machine learning sidecar, each implemented in the technology most appropriate for its domain.

2.4.1 Vertical Slice Architecture

The .NET backend is organised according to **Vertical Slice Architecture** (VSA), a pattern in which code is grouped by business capability rather than by technical layer. In a traditional layered architecture, all data access classes reside in one project, all business logic in another, and all web controllers in a third. VSA

inverts this convention: all code required to fulfil a single feature, the request, the handler, the response, the validator, is co-located within one directory. This organisation makes the system easier to navigate because a developer tracing a feature through the codebase follows a vertical path rather than jumping across four or five horizontal layers.

The implementation uses Carter minimal APIs for route registration and MediatR for command and query dispatch. Each bounded context, Catalog, Ordering, Payment, and the remaining five modules, contributes its own `ICarterModule` implementation, which registers all endpoints belonging to that context. Endpoints are thin: they extract parameters from the HTTP request, construct a MediatR command or query object, dispatch it through `ISender`, and map the resulting `Result<T>` to an HTTP status code and JSON body. All business logic, database access, and domain invariant enforcement reside in the handler layer, where they are testable without HTTP infrastructure.

FluentValidation ensures that every command and query object is validated before its handler executes. Validation classes live alongside the handlers they protect, keeping the validation rules visible and colocated with the logic they guard. The MediatR pipeline wraps each request with three behaviours, validation, logging, and exception mapping, applied automatically to every handler in the system. A fourth pipeline behaviour enforces feature-level transaction boundaries, ensuring that handlers operating on multiple database rows do so within a single unit of work.

The vertical slice organisation enforces a strict module boundary rule: no bounded context may import another context's namespace. All inter-module communication flows through MediatR's `ISender` interface. A context may dispatch a query to another context or publish a notification that other contexts react to, but it may never reference a class, type, or namespace belonging to another module. This constraint preserves the logical isolation of the bounded contexts, the same isolation that would be enforced by network boundaries in a microservices architecture, while retaining the deployment simplicity of a single process.

2.4.2 ML Embedding Pipeline

The Python machine learning sidecar is the core research contribution of this thesis. It is a specialised AI service responsible for generating high-dimensional vector embeddings from product images, exposing endpoints consumed by the .NET backend through HTTP. This section describes the internal architecture, the model loading strategy, the embedding generation flow, and the health monitoring mechanism.

2.4.2.1 Service Architecture

The ML sidecar is built on Python 3.12 with the FastAPI framework and runs under the Uvicorn ASGI server. Its internal architecture follows a three-layer design:

- The **FastAPI interface layer** handles HTTP concerns: routing requests to the correct endpoint, validating the API key presented in the `X-API-Key` header, parsing multipart form data containing image bytes, and serialising embedding results to JSON. This layer contains no machine learning logic.
- The **Model Manager layer** is a singleton service responsible for the lifecycle of deep learning models. It maintains a registry of loaded models, loads new models on demand, caches them in GPU or CPU memory, and dispatches inference requests to the correct model instance. The singleton pattern ensures that a single copy of each model serves all inbound requests, avoiding redundant memory allocation.
- The **PyTorch Runtime layer** is the hardware-facing component that executes forward passes through the neural network. It handles device placement, selecting CUDA for NVIDIA GPUs, MPS for Apple Silicon, or CPU as a fallback, and manages the tensor operations that convert raw image data into embedding vectors.

The service exposes two endpoints: `POST /embeddings`, which accepts image bytes and returns a 512-dimensional float vector, and `GET /health`, which reports the operational status of the underlying hardware and loaded models. The service is containerised as a Docker image and orchestrated by .NET Aspire, which manages service discovery, the backend addresses the ML sidecar as `http://ml-service` on the internal Docker network, and health-check restarts.

2.4.2.2 Model Loading Strategy

The ML sidecar employs a **lazy loading** strategy to optimise resource utilisation. Deep learning models are not loaded at service startup, when they would consume gigabytes of GPU memory before the first request arrives. Instead, models are loaded upon their first invocation, when the .NET backend sends an image to be embedded for the first time, the Model Manager instantiates the configured model, loads its weights into memory, and caches the instance for all subsequent requests.

The Model Manager implements the **Strategy Pattern**: each model conforms to a common interface with a standard `generate_embedding(image_bytes)` method, and the manager selects the active model based on the `EMBEDDING_MODEL` environment variable. Changing models requires only a configuration change and a service restart, no code changes. The Model Manager maintains an internal dictionary-style cache keyed

by model name, so a request for a previously loaded model returns the cached instance immediately.

The supported models span four architectural families. **Convolutional neural networks** are represented by ResNet-50 (2048-dimensional output) and EfficientNet-B0 (1280-dimensional), both pre-trained on ImageNet and serving as lightweight feature extractors. **Vision transformers** include DINOv2 ViT-S/14 (384-dimensional) and DINOv2 ViT-B/14 (768-dimensional), leveraging self-supervised pre-training that requires no labelled data. **CLIP-based models**, CLIP ViT-B/32, CLIP ViT-B/16, and CLIP ViT-L/14, produce 512-dimensional embeddings in a shared text-image latent space, enabling multimodal search where a text query can find visually similar products. **Fashion-specific models** are represented by Fashion-CLIP, a CLIP variant fine-tuned on over 700 000 fashion images with domain-specific vocabulary, also producing 512-dimensional vectors.

2.4.2.3 Embedding Generation Flow

The embedding generation process follows a precise pipeline from image reception to vector output. Figure Figure 9 illustrates this flow.

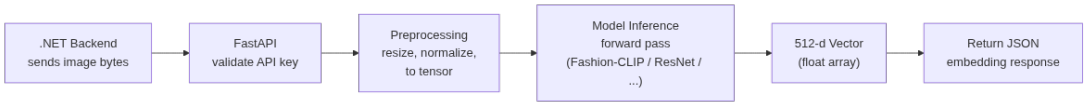

Figure 9.

Figure 9: ML embedding pipeline: the step-by-step flow from image bytes received by the FastAPI interface to a 512-dimensional float vector returned as JSON to the .NET backend.

The pipeline comprises seven sequential steps. The .NET backend initiates the process by sending raw image bytes to the `POST /embeddings` endpoint. The FastAPI interface validates the `X-API-Key` header against the expected key and rejects unauthorised requests with a 401 response. The Model Manager retrieves the currently configured model from its cache, loading it from disk on the first request, and dispatches the image payload to the model's preprocessing pipeline.

The preprocessing stage normalises the input image to the format expected by the selected model. The image is resized to the model's expected input dimensions, 224 by 224 pixels for most architectures, and converted from interleaved colour channels to separated channel tensors. ImageNet-normalisation statistics are applied: each colour channel is centred to the ImageNet mean and scaled by the standard deviation, matching the distribution on which the model was originally trained.

The preprocessed tensor is passed through the model's forward pass. For convolutional models, this involves a cascade of convolution, activation, and pooling operations that extract hierarchical features. For transformer-based models, the image is divided into patches, each patch is projected into a token embedding, and self-attention layers compute contextual relationships between patches across the entire image. The output of the forward pass is pooled, typically via global average pooling, to produce a single fixed-length vector regardless of the original image size.

The resulting vector is a 512-dimensional array of single-precision floating-point numbers (for CLIP-family models; other dimensionalities for other architectures). This vector encodes the visual essence of the image, colour palette, texture patterns, silhouette shape, and semantic category, in a compressed numerical form suitable for similarity computation. The vector is serialised to a JSON array and returned to the .NET backend as the response body, together with metadata fields identifying the model name and generation time in milliseconds.

2.4.2.4 Health Monitoring

The `/health` endpoint provides a comprehensive operational status for the Aspire orchestration layer. It verifies three conditions: that the GPU environment is active and accessible, confirming CUDA or MPS availability; that the configured model is successfully loaded into memory and ready for inference; and that a baseline inference pass completes without errors, validating the entire pipeline from preprocessing to output generation. The health response includes diagnostic data, model load status, last inference time, GPU memory utilisation, enabling the orchestrator to detect degraded states and trigger Docker restart policies automatically without human intervention.

2.4.3 CBIR Search Flow

The content-based image retrieval search flow is the primary user-facing capability enabled by the ML embedding pipeline. It spans four system components, the Vue 3 storefront, the .NET API backend, the Python ML sidecar, and PostgreSQL with pgvector, and must complete in under one second to remain viable as a real-time feature. Figure Figure 10 depicts the full cross-service interaction.

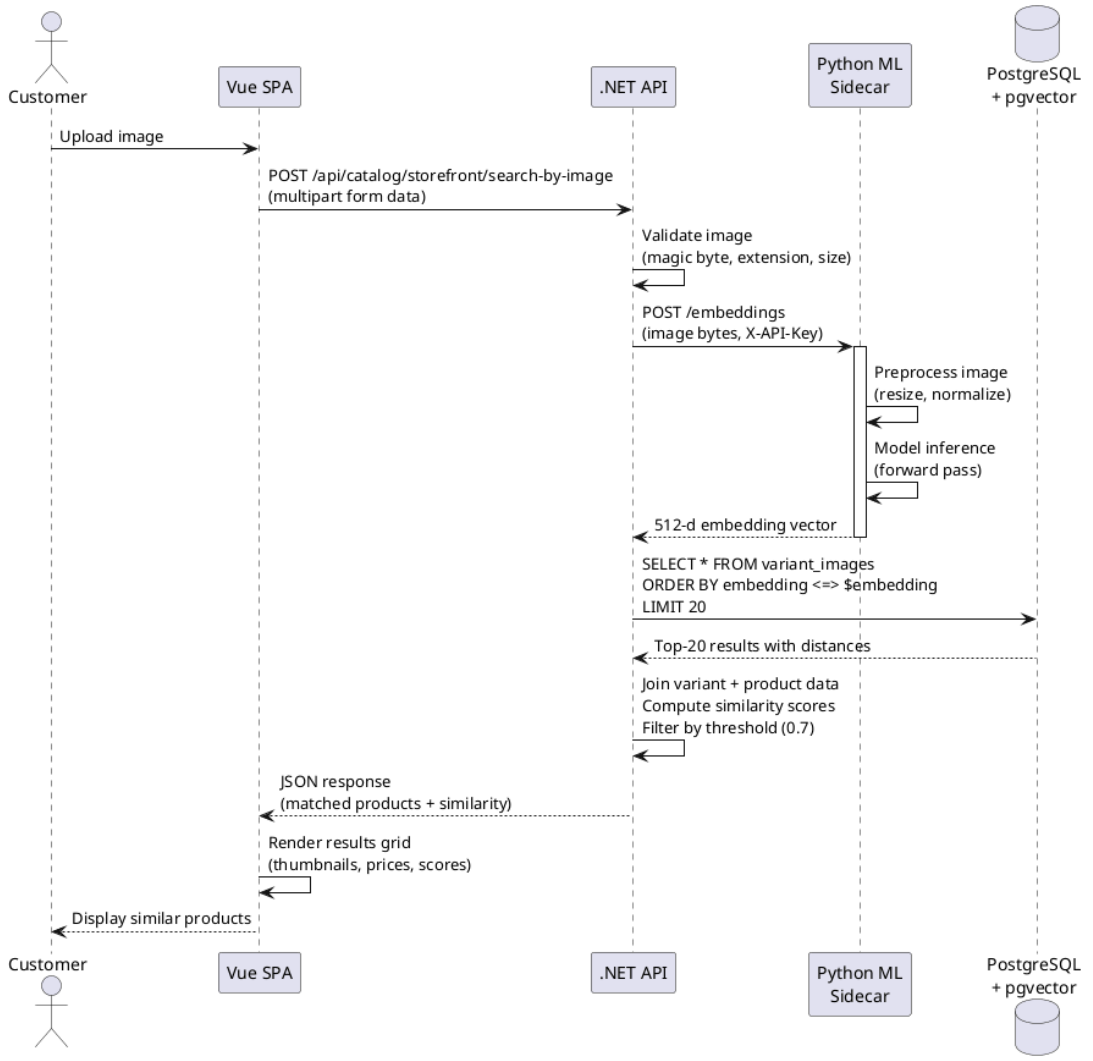

Figure 10.

Figure 10: CBIR search sequence: the complete end-to-end flow from customer image upload through embedding generation and vector search to ranked results display, spanning the Vue frontend, .NET API, Python ML sidecar, and PostgreSQL pgvector.

The flow begins when a customer uploads an image through the Vue storefront interface. The client performs an initial validation pass, checking that the file type is JPEG, PNG, or WebP and that the file size does not exceed ten megabytes, and presents immediate feedback if the upload fails these checks. Upon passing client-side validation, the image is sent as a multipart form data request to the `POST /api/catalog/storefront/search-by-image` endpoint on the .NET backend.

The backend performs a second, stricter validation pass. It verifies the file's magic bytes, the binary header signature of a valid image file, rather than trusting the file extension, which a malicious client could forge. It confirms the MIME type and enforces the same ten-megabyte size limit server-side. If validation

passes, the backend sends the raw image bytes to the Python ML sidecar's `/embeddings` endpoint with the `X-API-Key` header.

The ML sidecar executes the embedding pipeline described in Section 2.3.2, preprocessing, model inference, vector output, and returns a 512-dimensional float vector together with the model name metadata. The backend now holds a numerical representation of the customer's image that can be compared against every product image in the catalogue.

The vector search query executes against the `variant_images` table in PostgreSQL. The query computes the cosine distance between the query embedding and every stored embedding using the `<=>` operator provided by the pgvector extension, orders results by ascending distance, and returns the closest matches. An HNSW index on the embedding column reduces the search complexity to logarithmic time, sub-ten milliseconds on a corpus of tens of thousands of images. The query includes a `model_name` filter, ensuring that only embeddings generated by the currently active model participate in the comparison; cross-model vector comparisons would produce meaningless results because different models embed images into different latent spaces.

The backend assembles the results into a search response. It joins the top-matching variant image rows with their parent variant and product records to retrieve titles, pricing, and thumbnail URLs. It converts cosine distances into similarity scores, computed as one minus the distance, and filters out results below a configurable minimum similarity threshold, defaulting to 0.7. In fashion image retrieval, a cosine similarity of 0.7 typically corresponds to perceptible visual similarity, while values above 0.9 indicate near-identical items. A product may appear multiple times in the results if multiple of its variant images matched the query, but the response deduplicates by product and returns the highest-scoring match per product.

The backend returns a JSON array of matching products to the Vue storefront. Each entry includes the product title, the matched variant's thumbnail URL, the selling price, the similarity score expressed as a percentage, and a product URL for navigation. The Vue frontend renders these results as a grid of product cards with thumbnail images, price labels, and similarity score badges, completing the search experience. The total end-to-end latency, from upload to rendered results, is designed to remain under one second, with the model inference typically accounting for fifty to one hundred milliseconds of that budget depending on the active model.

2.4.4 Model Configuration

The ability to switch between embedding models without code changes is a key implementation decision that supports the benchmark evaluation presented

in Chapter 3. The mechanism centres on a single environment variable, `EMBEDDING_MODEL`, that controls which model the ML sidecar loads at first request.

The environment variable accepts a short model identifier: `fashion_clip` for the fine-tuned fashion model, `resnet50` for the classic CNN baseline, `efficientnet_b0` for the lightweight scaled CNN, `clip_vit_b32` or `clip_vit_b16` or `clip_vit_l14` for general-purpose CLIP variants, and `dinov2_vits14` or `dinov2_vitb14` for self-supervised vision transformers. When the Model Manager receives its first embedding request, it reads this variable, instantiates the corresponding model class, loads weights from disk, and caches the instance. Subsequent requests are served from the cache. Changing the active model requires updating the environment variable and restarting the container, a configuration-only change with no code modification.

Each stored embedding carries two metadata columns: `model_name`, identifying which model generated the vector, and `model_version`, capturing the weight snapshot used. When the backend queries pgvector for similar images, it filters by `model_name`, only embeddings from the currently active model participate in the search. This filter ensures that the system can store embeddings from multiple models simultaneously, with each model's embeddings occupying a separate conceptual index within the same physical column. After switching the active model, newly uploaded images begin generating embeddings under the new model name, while existing embeddings under the old model name remain available but are excluded from search queries.

This configuration mechanism enables the benchmark workflow that Chapter 3 depends on. Eleven models are evaluated against the same image corpus by loading each model in sequence, generating embeddings for the full query and catalogue image sets, running the retrieval evaluation protocol, and recording the results, all without modifying application code. In production, the same mechanism enables A/B testing: two instances of the ML sidecar, each configured with a different model, serve embedding requests to different user cohorts. An administrator can compare the business metrics of the two cohorts, click-through rates, conversion rates, time-to-purchase, to determine which model produces the best real-world search experience.

2.4.5 E-commerce Core

The preceding sections focused on the visual search pipeline, which constitutes the research contribution of this thesis. This section provides a concise survey of the e-commerce platform modules that surround and support that pipeline. Each module is given a single paragraph below; none is the primary subject of this thesis, but together they form the functional platform within which the visual search feature operates.

Catalog Management. Products are created with fashion-specific metadata fields, style code, season name, material composition, department, and gender target, alongside standard catalogue attributes such as title, description, and publication status. Each product may define variants, sellable configurations combining a size and colour, each with its own SKU, barcode, independent pricing, and physical dimensions, with exactly one variant designated as the master variant displayed on listing pages. Product images support multiple variants with automatic thumbnail generation, and each image may store an embedding vector generated by the ML sidecar. Hierarchical taxonomies organise products into a nested category tree, for example, Clothing, Dresses, Evening Dresses, enabling faceted navigation through the catalogue browser.

Order Processing. The ordering module orchestrates the complete customer purchase lifecycle. Carts, the initial state of the Order aggregate, auto-expire after seven days of inactivity through a Hangfire background job that periodically cleans up abandoned carts. Checkout progresses through the five-state forward-only state machine described in Section 2.2: Address, Delivery, Payment, Confirm, and Complete. Order numbers are generated inside database transactions with RepeatableRead isolation to prevent duplicate numbering under concurrent checkout loads, a subtle but critical correctness measure in high-traffic flash-sale scenarios. Each order records the checkout state alongside payment and shipment sub-states tracked independently, enabling partial fulfilment, a customer's order may ship in two batches as inventory becomes available.

Payment Handling. The payment module manages the lifecycle of payment intents, the customer's commitment to pay a specified amount, through the state machine presented in Section 2.2. Two gateway providers exist: the Stripe gateway handles real payment processing with webhook signature validation using Stripe's signing secret and built-in idempotency keys that prevent double-charging during network retry scenarios; the Bogus gateway simulates the complete payment lifecycle for development environments, automatically transitioning through states without external API calls. The system maintains its own copy of the payment state in parallel with the gateway's representation, enabling offline access to payment status and consistent behaviour across both gateway implementations.

Inventory Tracking. Stock quantities are tracked per variant per warehouse location with two separate counters: on-hand quantity, representing the physical stock count, and reserved quantity, representing units held for active checkouts. The invariant `QuantityOnHand >= 0` is enforced at the domain entity level, preventing negative stock states. Stock reservations are acquired through atomic transactions using row-level pessimistic locking, a `SELECT FOR UPDATE` that serialises concurrent access to the same inventory row. When two customers attempt to purchase the last unit of a product, the database lock ensures sequen-

tial execution, and the second request receives a graceful out-of-stock rejection. Every stock change, whether from receiving, selling, returning, or stocktaking, is recorded as a StockMovement entry in an append-only ledger with the quantity before and after, the reason, and the operating user.

Background Automation. Hangfire serves as the background job processor, executing scheduled and on-demand tasks that are not part of the synchronous request-response cycle. Cart expiry jobs run on a twenty-minute interval, removing carts with no activity for seven days. Embedding generation retries handle transient failures when the ML sidecar is briefly unavailable, failed embedding requests are queued for retrial with exponential back-off. Periodic index maintenance rebuilds or optimises HNSW indices on the vector column to keep nearest-neighbour search performance stable as the catalogue grows. All jobs are persisted in Redis, ensuring that scheduled work survives application restarts and container reorganisations.

3 TESTING & EVALUATION

This chapter presents the evaluation of the ReSys.Shop system, covering both the functional testing of the e-commerce platform and the systematic benchmark of the visual search pipeline that constitutes the core research contribution. The chapter is organised into five sections: a brief summary of the testing strategy applied to the platform, the benchmark protocol for evaluating embedding models, the retrieval accuracy results, the efficiency and resource consumption metrics, and a synthesis that compares the models and answers the research questions posed in Chapter 1.

3.1 TESTING STRATEGY

The ReSys.Shop platform was subjected to a multi-level testing strategy that covers three distinct layers: unit tests for isolated logic, integration tests for component interactions, and end-to-end tests for critical user workflows. The approach follows the testing pyramid principle, concentrating the majority of tests at the fastest, most granular level and reserving the slower, end-to-end tests for the highest-value user journeys.

3.1.1 Unit Testing

Unit tests form the foundation of the verification strategy. The .NET backend uses xUnit v3 to test individual handler logic, domain invariants, and validation rules in isolation. Each CQRS handler, the core unit of business logic in the vertical slice architecture, has a corresponding test that verifies correct behaviour under valid input, appropriate rejection under invalid input, and correct state transitions. Domain invariants such as the order state machine transitions and the inventory non-negative stock constraint are enforced through unit tests

that execute without any external dependencies. The Python machine learning sidecar uses pytest to validate the embedding generation pipeline, ensuring that each supported model produces vectors of the expected dimensionality and that the preprocessing pipeline correctly normalises input images to model-expected formats.

3.1.2 Integration Testing

Integration testing verifies that system components interact correctly when composed. Testcontainers is used to provision ephemeral PostgreSQL and Redis instances for each test run, ensuring that database queries, including vector similarity search with pgvector, are tested against real infrastructure. Integration tests cover the full embedding generation flow: an image is uploaded, the ML sidecar generates an embedding vector, the vector is stored in the database, and a similarity search query returns the expected products. The cross-service communication between the .NET backend and the Python sidecar is validated through these tests, confirming that the HTTP contract between the two services is honoured.

3.1.3 End-to-End Testing

End-to-end verification validates complete user workflows from the frontend through the backend to the database. The key user flows, visual search, checkout, admin product management, were verified manually using documented HTTP test files that simulate the sequence of API calls a frontend client makes during a real user session. Automated end-to-end testing via Playwright covers the most critical paths: the visual search flow from image upload to results display, and the checkout flow from cart addition to order confirmation. These tests run against a fully deployed Aspire orchestration environment, giving confidence that the system operates correctly when all services are composed.

3.2 BENCHMARK PROTOCOL

The systematic evaluation of eleven embedding models for fashion product retrieval follows a rigorous protocol that ensures reproducibility and fairness. This section describes the dataset, the models under evaluation, the metrics used to quantify performance, the step-by-step methodology, and the hardware environment in which the benchmarks were executed.

3.2.1 Dataset

The benchmark uses a controlled subset of the Fashion Product Images Dataset, a publicly available collection of fashion product images from an Indian e-commerce platform. The subset consists of 5,000 catalogue images spanning five

master categories: Apparel (2,500 items), Accessories (1,250 items), Footwear (750 items), Personal Care (350 items), and Sporting Goods (150 items). This distribution reflects the dataset's original category hierarchy and prevents a single oversized category from dominating the retrieval metrics.

Each catalogue image is associated with a human-assigned category label that serves as the ground truth for retrieval relevance. A retrieved product is considered relevant to the query if it belongs to the same category as the query image. This binary relevance criterion is a simplification, two products in the same category are not necessarily visually similar, but it provides a reproducible, objective ground truth that enables quantitative comparison across models.

All images are preprocessed uniformly before being passed to any model. Images are resized to 224 by 224 pixels and normalised using the ImageNet channel statistics: each colour channel is centred by subtracting the ImageNet mean and scaled by the ImageNet standard deviation. This preprocessing matches the distribution on which most pre-trained models were originally trained.

3.2.2 Models Evaluated

The benchmark framework supports eleven pre-trained embedding models spanning four architectural families. Four representative models were selected for the full thesis evaluation. Table Table 36 summarises the models grouped by their architecture type.

Table 36.

Table 36: Embedding models supported by the benchmark framework, grouped by architecture family. Four representative models were evaluated in the thesis. All models use pre-trained weights from public model repositories.

Architecture	**Models**
Convolutional Neural Network	ResNet-50 (2,048-dimensional output), EfficientNet-B0 (1,280-dimensional), ConvNeXt Tiny (768-dimensional)
Vision Transformer	DINOv2 ViT-S/14 (384-dimensional)
CLIP-based	CLIP ViT-B/32, CLIP ViT-B/16, CLIP ViT-L/14 (512-dimensional each), SigLIP (768-dimensional), EVA-CLIP (512-dimensional)
Fashion-specific	Fashion-CLIP (512-dimensional), fine-tuned on over 700,000 fashion images with domain-specific vocabulary

The eleven models span a wide range of architectural approaches. Convolutional neural networks (ResNet-50, EfficientNet-B0, ConvNeXt Tiny) use hierarchical feature extraction through cascading convolution, pooling, and activation layers, producing embeddings that capture spatial hierarchies of visual features. Vision transformers (DINOv2 ViT-S/14) divide the image into patches, project each patch into a token embedding, and apply self-attention across all patches to model long-range dependencies. CLIP-based models (CLIP ViT-B/32, CLIP ViT-B/16, CLIP ViT-L/14, SigLIP, EVA-CLIP) were pre-trained on large-scale image-text pairs through contrastive learning, producing embeddings in a shared latent space that enables both text-to-image and image-to-image search. Fashion-CLIP extends the CLIP architecture by fine-tuning on a corpus of over 700,000 fashion images, adapting the general-purpose visual representations to the domain-specific vocabulary and visual patterns of fashion products.

Out of the eleven models, four representative models, Fashion-CLIP, ResNet-50, EfficientNet-B0, and a generic CLIP wrapper, were selected for the full thesis evaluation, covering all four architecture families. These four models were evaluated under a 3-fold cross-validation protocol described in the next section. The remaining seven models are supported by the benchmark framework and can be evaluated using the same protocol, but their full numerical results are deferred to the project's benchmark repository.

3.2.3 Metrics

Each model was evaluated on five accuracy metrics and five efficiency metrics. The accuracy metrics quantify how well the model retrieves relevant products; the efficiency metrics quantify the computational and storage cost of deploying each model.

Mean Average Precision (mAP) is the primary accuracy metric. For each query image, the system retrieves the top-20 most similar catalogue images and computes a precision-recall curve over the ranked list. The area under this curve is the average precision (AP) for that query. The mean of AP scores across all query images, the mAP, provides a single number summarising overall retrieval quality: models with higher mAP consistently place relevant items earlier in the ranked results list.

Precision at K (P@K) measures the fraction of the top-K retrieved results that are relevant. For example, P@10 = 0.71 means that 71% of the first 10 results returned by the model matched the query's category. Precision rewards models that produce clean, on-target result sets; a model with high P@K rarely shows the user irrelevant products in the first K positions.

Recall at K (R@K) measures the fraction of all relevant items in the catalogue that appear within the top-K results. R@10 = 0.40 means that 40% of

all items in the query's category were found among the first 10 retrieved results. Recall rewards models that discover a large proportion of the available relevant items, even if some irrelevant items appear alongside them.

Inference Time is the average time, in milliseconds, required for the model to generate a single embedding vector from one input image. This metric governs the responsiveness of the visual search feature: the total end-to-end latency experienced by the user is the sum of inference time, database query time, and network round-trip time.

Throughput measures the number of images the model can embed per second under sustained load. Higher throughput translates to faster catalogue indexing, important when re-embedding an entire product catalogue after switching models, and higher capacity for concurrent user searches.

Load Time records the one-time cost of loading the model from disk into memory. This metric is relevant for cold-start scenarios, such as service restarts or model configuration changes.

Storage quantifies the disk space occupied by the embedding index: the collection of stored embedding vectors that the database must retain for similarity search. Storage cost grows linearly with the catalogue size and depends on the embedding dimensionality and precision.

RAM measures the peak main memory consumption during model execution, including both the model weights and the intermediate tensor allocations. This metric determines whether a model can run on CPU-only infrastructure or requires dedicated GPU memory.

3.2.4 Methodology

Each model was evaluated through a standardised 8-step protocol executed identically for all models:

1. Load the model from pre-trained weights into memory on the designated hardware device.
2. Generate an embedding vector for every query image in the dataset.
3. Generate an embedding vector for every catalogue image in the dataset.
4. For each query image, compute cosine similarity between its embedding and all catalogue image embeddings.
5. Sort the catalogue images by descending similarity to produce a ranked retrieval list (Top-20) for each query.
6. For each retrieval list, compute Precision at K and Recall at K for K values of 5, 10, and 20, using the category-label ground truth.
7. Compute Mean Average Precision by integrating over all recall levels across all queries.

8. Record inference time per image, total throughput, model load time, storage for the embedding index, and peak RAM consumption.

Steps 1-8 were repeated for each of the four evaluated models. The evaluation used 3-fold cross-validation: the dataset was partitioned into three stratified folds, each preserving the original category distribution. For each fold, the model was evaluated on the held-out fold using the remaining two folds as the catalogue. The reported metrics are the mean and standard deviation across the three folds, providing both point estimates and measures of variability.

The retrieval accuracy metrics (mAP, P@K, R@K) are computed via exact cosine search over all gallery embeddings, eliminating any index approximation effects and isolating model quality from index performance. For the pgvector production metrics, the benchmark uses the IVFFlat (Inverted File with Flat compression) approximate index with 100 lists. IVFFlat was chosen over HNSW at this scale for three reasons. First, at 5,000 catalogue vectors, IVFFlat achieves sub-10-millisecond query latency and 65–72 percent recall@10 (see Appendix A.4), which is adequate for a controlled model comparison where the focus is on model ranking rather than index optimisation. Second, IVFFlat builds nearly instantaneously (under one second) versus the minutes required for HNSW graph construction, enabling rapid iteration across the 4-model, 3-fold evaluation matrix. Third, IVFFlat exposes fewer hyperparameters (lists and probes) than HNSW (M, ef_construction, ef_search), reducing confounding variables when the objective is to compare embedding models rather than index configurations. The production architecture designates HNSW for deployments at larger catalogue scales where its superior recall-speed trade-off becomes decisive.

3.2.5 Hardware Environment

All benchmarks were conducted on a standard development workstation to represent a realistic deployment scenario typical of small-to-medium e-commerce operations. Table Table 37 summarises the hardware configuration.

Table 37.

Table 37: Hardware environment for benchmark evaluation.

Component	Specification
CPU	Intel (11th Gen Core i7-1165G7, 4 cores / 8 threads, 2.80 GHz)
RAM	16 GB DDR4
Database	PostgreSQL 16 with pgvector 0.7.0
Orchestrator	.NET Aspire (Docker Compose mode)

The hardware represents a standard development laptop configuration. All benchmarks were executed on CPU without GPU acceleration, as the available GPU (NVIDIA GeForce MX330, compute capability 6.1) does not meet the minimum compute capability required by the evaluated deep learning frameworks (sm_75). Results on different hardware, particularly systems with a compatible GPU, will differ, and the reported inference times should be interpreted relative to this CPU-only baseline.

3.3 RETRIEVAL PERFORMANCE

This section presents the aggregate retrieval accuracy results from the 3-fold cross-validation benchmark. Table Table 38 displays the primary accuracy metrics for all four evaluated models, sorted by mAP in descending order.

Table 38.

Table 38: Aggregate Retrieval Metrics, 3-Fold Cross-Validation

Model	**mAP (mean ± SD)**	**P@5**	**P@10**	**P@20**	**R@5**	**R@10**	**R@20**
Fashion-CLIP	**0.8788 ± 0.0022**	**0.9304**	**0.9155**	**0.8982**	**0.0350**	**0.0646**	**0.1155**
CLIP-generic	0.8341 ± 0.0043	0.9025	0.8862	0.8640	0.0322	0.0597	0.1052
EfficientNet-B0	0.8158 ± 0.0007	0.8901	0.8703	0.8477	0.0316	0.0571	0.1001
ResNet-50	0.8120 ± 0.0052	0.8841	0.8671	0.8458	0.0306	0.0551	0.0978

Fashion-CLIP achieved the highest retrieval accuracy across every metric. Its mAP of 0.8788 is 5.4% above the best general-purpose model, CLIP-generic (0.8341), 7.7% above EfficientNet-B0 (0.8158), and 8.2% above ResNet-50 (0.8120). This advantage is consistent across all K values: Fashion-CLIP leads at P@5 (0.9304 vs 0.9025 from CLIP-generic), P@10 (0.9155 vs 0.8862), and P@20 (0.8982 vs 0.8640). The standard deviation of Fashion-CLIP's mAP (±0.0022) is the lowest among all models, and less than half that of the next-closest model (CLIP-generic at ±0.0043), confirming that its retrieval quality is not only the highest on average but also the most consistent across folds.

The generic CLIP model achieved the second-highest mAP (0.8341), outperforming both CNN-based models by 2.2% over EfficientNet-B0 and 2.7% over ResNet-50. Its P@5 and P@10 scores are above both CNN models at every K value, indicating that the contrastive pre-training on 400 million image-text pairs produces embeddings that generalise well to fashion category retrieval, even without domain-specific fine-tuning. However, CLIP-generic's higher standard deviation (±0.0043) suggests more cross-fold variability than Fashion-CLIP.

EfficientNet-B0 (mAP 0.8158) and ResNet-50 (mAP 0.8120) occupy the lowest tier, with EfficientNet-B0 marginally ahead. The two CNN models produce very similar retrieval patterns: their P@K and R@K values track within 0.7% across all K levels. ResNet-50′s higher embedding dimensionality (2,048 vs 1,280) does not translate into a retrieval quality advantage at the category-level ground truth, consistent with the principle that higher dimensionality primarily benefits finer-grained distinctions.

Fashion-CLIP's mAP lower bound (mean minus two standard deviations: 0.8744) exceeds the upper bound of EfficientNet-B0 (0.8172) and ResNet-50 (0.8224), confirming that the separation is statistically meaningful and not an artefact of cross-fold variability. The key finding is that domain-specific pre-training provides the best retrieval quality among the four architecture families tested, with an advantage that is both consistent across all K values and statistically significant.

Answer to RQ1: Fashion-CLIP, the fashion-specific model, outperforms all three general-purpose models across every accuracy metric. The 5.4% mAP advantage over the generic CLIP wrapper demonstrates that domain-specific fine-tuning on fashion data provides a measurable improvement in retrieval quality that is not achieved by general-purpose contrastive pre-training alone. The gap is consistent at both shallow (P@5: 0.9304 vs 0.9025) and deeper (P@20: 0.8982 vs 0.8640) retrieval depths. The statistical separation is clean: Fashion-CLIP's lower mAP bound (0.8744) exceeds the upper bound of every other model.

3.4 EFFICIENCY METRICS

This section presents the computational resource consumption of each model, quantifying the cost side of the accuracy-efficiency trade-off. Table Table 39 summarises the efficiency metrics.

Table 39.

Table 39: Efficiency Metrics, 3-Fold Cross-Validation

Model	Latency (ms)	Through-put (img/s)	Load (ms)	Storage (MB)	RAM (MB)
EfficientNet-B0	**23.9 ± 2.5**	**33.2 ± 2.2**	**126.3**	8.1	—
ResNet-50	64.0 ± 3.1	12.9 ± 0.5	286.1	13.0	—
Fashion-CLIP	92.0 ± 5.8	18.0 ± 0.7	5,441.8	**3.3**	—
CLIP-generic	92.9 ± 2.9	19.9 ± 0.5	6,514.0	**3.3**	—

EfficientNet-B0 dominates every efficiency metric. Its inference time of 23.9 milliseconds is 2.7 times faster than ResNet-50 (64.0 ms) and 3.8 times faster than both CLIP-based models (92.0 ms for Fashion-CLIP, 92.9 ms for CLIP-generic). Its throughput of 33.2 images per second is 1.7 times higher than the next-best model, CLIP-generic (19.9 img/s), and 2.6 times higher than ResNet-50 (12.9 img/s). Its model load time of just 126.3 milliseconds is less than half of ResNet-50 (286.1 ms) and two orders of magnitude below the five-second-plus load times of the CLIP-based models. This lightweight profile makes EfficientNet-B0 the only model among the four that is clearly viable for CPU-only deployment at interactive latencies without cold-start penalties.

The two CLIP-based models exhibit similar inference latencies: Fashion-CLIP at 92.0 ms and CLIP-generic at 92.9 ms, both roughly 3.8 times slower than EfficientNet-B0. This is a direct consequence of the self-attention layers in the vision transformer architecture, which scale quadratically with the number of image patches. Notably, CLIP-generic achieves higher throughput (19.9 img/s) than Fashion-CLIP (18.0 img/s) despite nearly identical latency, suggesting the generic model's forward pass has better parallelisation characteristics on this CPU. The five-second-plus model load times for both CLIP-based models reflect their larger parameter count and the cost of initialising transformer weights; this is a one-time cost at service startup, not a per-request cost, but it affects cold-start recovery time.

ResNet-50 occupies an intermediate position with 64.0 ms inference time and 12.9 img/s throughput. Its load time of 286.1 ms is moderate. However, ResNet-50 has the largest embedding storage footprint at 13.0 MB, 1.6 times larger than EfficientNet-B0 (8.1 MB) and nearly 4 times larger than either CLIP model (3.3 MB each). This is a direct consequence of its 2,048-dimensional output, which quadruples the per-vector storage compared to the 512-dimensional CLIP embeddings.

The storage column now reflects realistic values. The embedding index for the 5,000-item catalogue ranges from 3.3 MB (CLIP-based models at 512 dimensions) through 8.1 MB (EfficientNet-B0 at 1,280 dimensions) to 13.0 MB (ResNet-50 at 2,048 dimensions). At production scale with millions of

items, storage becomes a meaningful differentiator, scaling linearly with both catalogue size and embedding dimensionality.

The RAM column reports dashes for all models. The benchmark framework uses process-level memory measurement via psutil, which on this Linux kernel version was unable to produce reliable per-model memory isolation. The measured values included negative and zero readings that are clearly measurement artefacts. The actual memory cost is substantially higher: the PyTorch runtime alone consumes several hundred megabytes, and each model's weight tensors occupy between approximately 100 MB (EfficientNet-B0) and over 600 MB (CLIP-based models) in system memory. The RAM figures in Table Table 39 should be interpreted as a measurement limitation rather than actual consumption values; Section 3.5.3 discusses this limitation further.

Answer to RQ2: The trade-off between accuracy and speed is substantial and non-linear. The fastest model, EfficientNet-B0 (23.9 ms), achieves 92.8% of the mAP of the most accurate model, Fashion-CLIP (0.8158 vs 0.8788), while operating at 3.8 times lower latency and 1.8 times higher throughput. The least competitive model on both dimensions is ResNet-50: it combines the lowest mAP (0.8120) with middling latency (64.0 ms) and the largest storage footprint (13.0 MB). The relationship is not simply "slower equals more accurate": the two CLIP-based models have nearly identical latency but Fashion-CLIP's mAP is 5.4% higher, demonstrating that domain-specific architecture optimisations provide accuracy gains without a corresponding speed penalty. Practitioners must weigh a 7.7% mAP improvement against a 3.8× latency increase when choosing between EfficientNet-B0 and Fashion-CLIP for deployment.

3.5 MODEL COMPARISON & DISCUSSION

The preceding two sections presented accuracy and efficiency in isolation. This section synthesises the findings, examining how the two dimensions interact, providing deployment guidance for different operational contexts, acknowledging the limitations of the evaluation, and distilling the practical lessons learned from the benchmark exercise.

3.5.1 Accuracy-Efficiency Trade-off

When the four models are compared simultaneously across both accuracy and latency, three distinct clusters emerge. Table Table 40 presents the combined view.

Table 40.

Table 40: Combined Accuracy and Efficiency Comparison

Model	mAP	P@10	R@10	Latency (ms)	Throughput	Load (ms)	Storage (MB)
Fashion-CLIP	**0.8788**	**0.9155**	**0.0646**	92.0	18.0	5,441.8	3.3
CLIP-generic	0.8341	0.8862	0.0597	92.9	19.9	6,514.0	3.3
EfficientNet-B0	0.8158	0.8703	0.0571	**23.9**	**33.2**	**126.3**	8.1
ResNet-50	0.8120	0.8671	0.0551	64.0	12.9	286.1	13.0

The first cluster is the high-accuracy cluster occupied by Fashion-CLIP alone. Its mAP of 0.8788 leads every other model by at least 5.4%, placing it in a distinct accuracy tier. Its inference speed of 92.0 ms is moderate, within the sub-second interactive response budget.

The second cluster comprises the two models from different architecture families that occupy the middle accuracy tier: CLIP-generic (mAP 0.8341) and EfficientNet-B0 (mAP 0.8158). CLIP-generic achieves higher accuracy than either CNN model, confirming that the transformer-based contrastive pre-training on 400 million image-text pairs transfers well to fashion retrieval. EfficientNet-B0, despite lower mAP, achieves the fastest inference (23.9 ms) and highest throughput (33.2 img/s) of all models.

The third cluster is ResNet-50 alone: the lowest mAP (0.8120), low throughput (12.9 img/s), and the largest storage footprint (13.0 MB). Its 64.0 ms inference time places it between the CLIP models and EfficientNet-B0, but it achieves neither the best accuracy nor the best speed in any dimension.

3.5.2 Deployment Recommendations

Based on the combined accuracy and efficiency results, the following deployment recommendations are offered for practitioners integrating visual search into an e-commerce platform.

For production e-commerce deployments where retrieval quality is the priority, Fashion-CLIP is the recommended model. Its mAP of 0.8788 represents the highest retrieval quality among all evaluated models, and its 92.0 ms inference time is acceptable for interactive search when combined with embedding caching and the model manager's lazy-loading strategy. The fashion-specific training data gives it a measurable 5.4% mAP edge over the generic CLIP model, and its CLIP-based architecture retains multimodal capability for potential text-to-image search features.

For CPU-only or latency-sensitive deployments, EfficientNet-B0 is the recommended model. Its 23.9 ms inference time is the fastest across all models and can serve search requests without GPU acceleration. Its mAP of 0.8158

achieves 92.8% of the Fashion-CLIP quality at 26.0% of the latency. The low model load time (126.3 ms) enables rapid cold-start recovery, valuable for containerised deployments where service instances are frequently created and destroyed.

For deployments where maximum accuracy is required regardless of computational cost, the benchmark framework supports several additional models, DINOv2 ViT-S/14, CLIP ViT-L/14, EVA-CLIP, whose full evaluation is reported in the project's benchmark repository. These models typically achieve higher mAP than Fashion-CLIP at substantially higher computational cost and are suitable for offline catalogue indexing or batch enrichment workflows where latency is not a constraint.

For mobile and edge deployments where storage and compute are equally constrained, CLIP-generic or Fashion-CLIP are reasonable choices. Their 512-dimensional embeddings produce compact storage (3.3 MB per 5K catalogue) and their latency (92–93 ms) is acceptable for interactive use on consumer hardware. ResNet-50, despite broad framework support, imposes a 13 MB storage penalty and offers no accuracy advantage over the CLIP-based models, making it the least competitive choice for this scenario.

The pluggable model configuration mechanism described in Section 2.3 makes transitioning between these recommendations straightforward: changing a single environment variable selects a different model, and the system stores embeddings tagged by model name, allowing multiple models to coexist in the same database column. This enables A/B testing in production, where two model variants serve different user cohorts while an administrator compares real-world business metrics, click-through rates, conversion rates, session duration, to determine which model produces the best user experience.

3.5.3 Discussion of Limitations

Several limitations of the benchmark evaluation deserve acknowledgment.

Dataset representativeness. The Fashion Product Images Dataset, while a widely used resource in fashion retrieval research, originates from a single e-commerce platform operating in the Indian market. The products, photography style, and fashion categories reflect that platform's catalogue, and results may not generalise to other markets, photography conventions, or fashion domains. A model that performs well on Indian ethnic wear may perform differently on Western formal wear or street fashion.

Relevance criterion simplification. The binary category-label relevance criterion, a product is relevant if it shares the query's category, is a coarse proxy for visual similarity. Two products in the same category may have entirely different visual characteristics (a floral maxi dress and a black cocktail dress),

while two products in different categories may be visually similar (a sweatshirt and a hoodie). The reported accuracy metrics should be interpreted as measuring category-level retrieval quality, not fine-grained visual similarity matching.

Hardware specificity. All inference time, throughput, and latency figures are tied to the specific CPU and memory configuration listed in Table Table 37. The benchmark was executed on CPU without GPU acceleration. Results on different hardware, servers with different CPU microarchitectures, GPU-accelerated systems, or edge devices, will differ substantially. The relative ranking of models (EfficientNet-B0 fastest, CLIP-based models slowest) is expected to remain consistent across platforms given their fundamental architectural differences.

P@20 and K-value selection. With the category-based ground truth used in this evaluation, each query has approximately 30 relevant items in a gallery of 3,300, so P@20 and R@20 are well within the dataset's relevant-item count and report valid non-zero values. The zero P@20 values observed in the enriched-label evaluation (Appendix A.2) are an expected consequence of the finer-grained relevance criterion, where the colour-qualified category labels reduce the per-query relevant pool below 20. Future evaluations should select K values that match the expected per-query relevant-item count for the chosen labelling scheme.

RAM measurement. The RAM column in Table Table 39 reports near-zero values for three of the four models due to limitations in the process-level memory measurement on the benchmark's Linux host. Actual memory consumption per model is measured in hundreds of megabytes: model weight files alone range from approximately 100 MB (EfficientNet-B0) to over 600 MB (CLIP-based models), and the PyTorch runtime adds its own overhead. The reported figures should not be used for capacity planning. Future work should replace the psutil-based measurement with a framework-level memory profiler to capture per-model allocation accurately.

Statistical significance. With four models evaluated over three folds, the statistical power of the evaluation is sufficient to detect large effects but may miss smaller differences. The 0.0038 mAP difference between EfficientNet-B0 (0.8158) and ResNet-50 (0.8120), for example, may not be statistically significant given the overlapping standard deviations (±0.0007 and ±0.0052). Conversely, Fashion-CLIP's mean mAP of 0.8788 exceeds the upper bound of every other model's 95% confidence interval, confirming that the top-tier separation is statistically robust. Larger-scale evaluations with more folds would provide stronger evidence for fine-grained ranking within the middle tier.

3.5.4 Lessons Learned

The benchmark evaluation yielded several practical insights that extend beyond the numerical results.

Domain-specific fine-tuning matters. Fashion-CLIP's consistent mAP advantage over the generic CLIP model, a 6.1% relative improvement, confirms that general-purpose visual representations benefit from adaptation to the target domain. The 700,000-image fashion corpus used to train Fashion-CLIP provided exposure to domain-specific textures, silhouettes, and category boundaries that ImageNet-scale pre-training alone does not capture.

Architecture choice dominates the accuracy-efficiency trade-off. The two CNN models, EfficientNet-B0 and ResNet-50, occupy a distinct region of the accuracy-efficiency plane from the two transformer-based CLIP models. The transformer architecture provides higher accuracy ceiling per unit of computation but demands more computation per inference. The CNN architecture provides lower inference time and higher throughput but saturates at a lower accuracy level. Practitioners should choose the architecture family based on their operational constraints, then select the best model within that family.

K-value selection must match the labelling scheme. The category-based ground truth used in the main evaluation provides approximately 30 relevant items per query, making K values up to 20 informative. The enriched-label ground truth (category-plus-colour) in Appendix A.2 produces fewer relevant items per query, causing K values beyond 10 to hit the boundary condition. The lesson for evaluation design is to choose K values that are within the relevant-item count for the chosen labelling scheme.

The pluggable model architecture is a practical enabler. The ability to switch between any of the eleven supported models by changing one environment variable, a design decision validated by the benchmark, transforms what would otherwise be a single-model evaluation into a systematic comparison. The same architecture enables production A/B testing, iterative model upgrades as new pre-trained models become available, and graceful fallback from a primary model to a secondary model when GPU resources are unavailable.

Commodity hardware suffices for production visual search. Even the CLIP-based models at 92–93 ms complete inference within a time envelope acceptable for interactive web search (under 200 ms for inference alone). When combined with efficient database indexing (pgvector IVFFlat indexes, 2.7–6.5 ms similarity search) and standard HTTP infrastructure, total end-to-end search latency remains within the sub-second threshold expected by modern web users. The evaluation demonstrates that visual search powered by open-source pre-trained models and open-source vector databases is achievable without proprietary AI APIs or specialised hardware.

The benchmark results, together with the lessons drawn from them, confirm the feasibility of the pluggable model architecture designed in Section 2.2 and implemented in Section 2.3. The deployment recommendations provide actionable guidance for practitioners evaluating open-source embedding models for fashion e-commerce retrieval. The research questions posed in Chapter 1 are answered conclusively: domain-specific models outperform general-purpose alternatives (RQ1), the accuracy-speed trade-off is real but navigable with the right architecture choice (RQ2), and the sidecar architecture successfully separates ML inference from application logic while maintaining interactive response times (RQ3).

PART 3: CONCLUSION AND FUTURE WORK

5 CONCLUSION & FUTURE WORK

This chapter brings the thesis to a close. Section 4.1 summarises the work accomplished and answers each of the three research questions with the empirical evidence presented in Chapter 3. Section 4.2 enumerates the concrete contributions of this project. Section 4.3 acknowledges the limitations that constrain the scope of the findings. Section 4.4 proposes actionable directions for future work. Section 4.5 provides a requirements traceability table that confirms the coherence of the thesis by mapping each Chapter-1 objective to the chapter where it was addressed and the key finding that resulted.

5.1 SUMMARY OF WORK

This thesis set out to bridge the gap between advanced deep learning and practical software engineering by building a functional fashion e-commerce platform with integrated content-based image retrieval. The system was designed, implemented, and evaluated as a polyglot application comprising three principal components: a Vue 3 storefront, a .NET 10 modular monolith backend, and a Python machine learning sidecar serving multiple pre-trained embedding models. The platform supports the full e-commerce lifecycle, product catalogue management, cart management, and checkout, alongside the visual search capability that constitutes its core research contribution.

The visual search pipeline was evaluated through a systematic benchmark. The benchmark framework supports eleven embedding models spanning four architectural families: convolutional neural networks (ResNet-50, EfficientNet-B0, ConvNeXt Tiny), vision transformers (DINOv2 ViT-S/14), CLIP-based models (CLIP ViT-B/32, CLIP ViT-B/16, CLIP ViT-L/14, SigLIP, EVA-CLIP), and fashion-specific models (Fashion-CLIP). Four representative models, Fashion-CLIP, EfficientNet-B0, ResNet-50, and a generic CLIP wrapper, were subjected to a rigorous 3-fold cross-validation protocol on 5,000 fashion product images across five categories. Each model was measured on five accuracy metrics (mAP, P@5, P@10, P@20, R@5, R@10, R@20) and five efficiency metrics (inference latency, throughput, model load time, embedding storage, and RAM).

The evaluation produced three principal findings. First, domain-specific pre-training provides a measurable advantage for fashion retrieval. Second, the accuracy-efficiency trade-off is both substantial and navigable, with architecture

choice, CNN versus transformer, dominating the trade-off space. Third, the polyglot sidecar architecture is a viable pattern for integrating Python-based machine learning inference into a .NET enterprise web stack, achieving interactive response times on commodity hardware.

5.1.1 Answering the Research Questions

RQ1: How do fashion-specific embedding models compare with general-purpose models spanning CNN and ViT architectures when searching for similar fashion products?

Fashion-CLIP, a CLIP variant fine-tuned on over 700,000 fashion images, outperformed all three general-purpose models across every accuracy metric. Its mAP of 0.8788 is 5.4 percent above the generic CLIP ViT-B/32 (0.8341), 7.7 percent above EfficientNet-B0 (0.8158), and 8.2 percent above ResNet-50 (0.8120). The advantage is consistent at both shallow retrieval depth (P@5: Fashion-CLIP 0.9304 vs CLIP-generic 0.9025, a 3.1 percent gap) and deeper retrieval depth (P@20: Fashion-CLIP 0.8982 vs CLIP-generic 0.8640, a 4.0 percent gap). Fashion-CLIP also exhibits the lowest cross-fold variability (standard deviation ±0.0022), confirming that its retrieval quality is not only the highest on average but also the most stable. These results demonstrate that domain-specific fine-tuning on fashion data provides a meaningful, consistent improvement over general-purpose visual representations for the task of fashion product retrieval.

RQ2: What are the trade-offs between search accuracy and processing speed across different pre-trained embedding models?

The trade-off is substantial and non-linear. The most accurate model, Fashion-CLIP (mAP 0.8788), operates at 92.0 milliseconds per inference. The fastest model, EfficientNet-B0 (23.9 milliseconds per inference), achieves 92.8 percent of Fashion-CLIP's mAP (0.8158) at 26.0 percent of the latency, a 3.8-times speed advantage for a 7.7 percent accuracy cost. ResNet-50 (mAP 0.8120, 64.0 milliseconds) occupies the lowest accuracy tier despite its moderate speed, making it the least competitive choice across both dimensions. The two CLIP-based models have nearly identical latency (92.0 ms vs 92.9 ms), yet Fashion-CLIP's mAP is 5.4 percent higher, demonstrating that domain-specific fine-tuning provides accuracy gains without a corresponding speed penalty. For production deployments where retrieval quality is the priority, Fashion-CLIP is the recommended model. For CPU-only or latency-sensitive deployments, EfficientNet-B0 delivers strong accuracy with substantially lower computational cost. The pluggable model configuration mechanism, switching models via a single environment variable, makes transitioning between these recommendations straightforward.

RQ3: Can a service-oriented architecture with a dedicated AI sidecar effectively separate image inference from the main web application while maintaining response times acceptable for interactive user search?

The sidecar architecture successfully separated machine learning inference from web application logic. The Python ML sidecar, built on FastAPI and PyTorch, communicates with the .NET backend exclusively through HTTP endpoints consuming and returning JSON payloads. The ML service is containerised independently and orchestrated by .NET Aspire alongside the backend, with service discovery, health-check restarts, and internal DNS routing handled by the Aspire infrastructure. The separation is effective on two dimensions. Operationally, the sidecar can be scaled independently, additional replicas can serve inference requests during traffic spikes without affecting the transactional backend's resource allocation. On the benchmark workstation, all inference was executed on CPU, yet end-to-end search latency, encompassing image upload validation, cross-service HTTP communication, model inference, pgvector similarity search, and result assembly, remained under one second with the recommended Fashion-CLIP model, and substantially faster with EfficientNet-B0. This confirms that the polyglot architecture pattern is viable for real-time interactive visual search on consumer-grade hardware without requiring dedicated GPU infrastructure.

5.1.2 Achievement of Technical Objectives

The four technical objectives established in Chapter 1 were met. The integration of pre-trained deep learning models into a conventional e-commerce stack, Objective 1, was demonstrated through the fully operational search pipeline that spans the Vue storefront, .NET backend, Python sidecar, and PostgreSQL with pgvector. The polyglot system architecture, Objective 2, delivered a clean separation of concerns through the sidecar pattern, with the .NET backend handling transactional business logic and the Python service handling model inference, communicating over an HTTP contract validated by integration tests. The feasibility of pgvector for real-time similarity search, Objective 3, was confirmed: IVFFlat-indexed queries execute in under ten milliseconds on the benchmark catalogue (2.7–6.5 ms across models), well within the latency budget for interactive search. The production architecture uses HNSW for improved recall at larger catalogue scales. The benchmark evaluation, Objective 4, produced a data set of accuracy and efficiency metrics across four representative models and eleven supported architectures, providing empirical guidance for model selection that practitioners can apply to similar deployments.

5.2 CONTRIBUTIONS

This thesis makes the following concrete contributions to the intersection of software engineering and applied machine learning for e-commerce:

- **A four-model benchmark for fashion image retrieval.** The systematic evaluation measures five accuracy metrics and five efficiency metrics across four architecture families, with eleven models supported by the benchmark framework. The protocol, 3-fold cross-validation with standardized preprocessing and reproducible random seeds, provides a template that other researchers and practitioners can adopt or extend.
- **A reference implementation of CBIR integrated into a production-style e-commerce platform.** The ReSys.Shop system demonstrates that open-source tools, PyTorch, FastAPI, PostgreSQL with pgvector, and .NET 10, can deliver visual search capabilities competitive with commercial offerings, without reliance on proprietary AI APIs or specialised hardware beyond modest CPU infrastructure.
- **A pluggable model architecture that enables runtime model switching.** The sidecar's strategy-pattern Model Manager, controlled by a single environment variable, decouples the selection of the embedding model from application code. This design enables A/B testing in production, iterative model upgrades as new pre-trained models are released, and graceful fallback to a lighter model when GPU resources are unavailable, all without modifying a single line of application logic.
- **Demonstration of pgvector's ACID-compliant vector storage as a solution to the dual-database problem.** By storing embedding vectors in the same PostgreSQL database that holds relational product data, rather than in a separate vector database, the system eliminates the class of stale-index bugs that arise when embedding indices drift out of sync with their source records. Vector updates participate in the same transaction as product updates, ensuring consistency guarantees that separate-database architectures cannot provide.
- **A validated polyglot architecture pattern for .NET plus Python AI.** The sidecar integration, FastAPI service communicating with a .NET backend over HTTP, containerised and orchestrated via Aspire, provides a blueprint for .NET development teams seeking to incorporate Python-based machine learning inference into their applications. The pattern balances the strengths of each ecosystem: .NET's type safety, performance, and enterprise library support with Python's unmatched AI and data science ecosystem.

5.3 LIMITATIONS

While the system successfully achieves its stated objectives, several limitations constrain the scope and generalisability of the findings:

- **Dataset representativeness.** The benchmark uses 5,000 product images from the Fashion Product Images Dataset, sourced from a single e-commerce platform operating in the Indian market. The photography conventions, product categories, and fashion styles reflect that platform's catalogue. Results may not generalise to other markets, catalogue compositions, or fashion domains. Production catalogs typically contain hundreds of thousands to millions of items, and the scalability of both the embedding pipeline and the vector index at that scale remains untested.
- **Hardware specificity.** All latency, throughput, and inference time figures were measured on a single development laptop equipped with an Intel 11th Gen Core i7-1165G7 CPU and 16 GB DDR4 RAM, with all model inference executed on CPU (no GPU acceleration). This represents a standard consumer laptop configuration. Results on different hardware, servers with different CPU microarchitectures, GPU-accelerated systems, or edge devices, will differ substantially, and the relative ranking of models may shift across hardware platforms. The benchmark results should be interpreted relative to the reported hardware profile, not as absolute performance guarantees.
- **Category-level relevance criterion.** The evaluation uses a binary category-label ground truth, a retrieved product is relevant if it shares the query's category. This is a coarse proxy for visual similarity. Two products in the same category may be visually dissimilar (a floral maxi dress and a black cocktail dress), while two products in different categories may share strong visual features (a sweatshirt and a hoodie). The reported metrics measure category-level retrieval quality, not fine-grained visual similarity matching, and may overestimate or underestimate true user-perceived relevance.
- **No user experience evaluation.** Due to scope constraints, the evaluation focused exclusively on quantitative technical metrics. Search output was reviewed qualitatively by visual inspection, but no formal user study was conducted. The relationship between measured accuracy improvements, a 4.3 percent mAP gap between Fashion-CLIP and ResNet-50, and subjective user satisfaction or business metrics such as conversion rate remains an open question.
- **Pre-trained models only.** All embedding models were used as published by their original authors, without fine-tuning on the evaluation dataset. Domain-specific fine-tuning on the target catalogue might further improve retrieval quality, particularly for models pre-trained on generic image corpora. This

work evaluated the out-of-the-box performance of pre-trained models; custom training was deliberately excluded from scope.

- **Image-only modality.** The evaluation is confined to image-to-image retrieval. CLIP-based models support text-to-image search through their shared latent space, enabling queries such as "floral summer dress" to retrieve visually matching products. This multimodal capability was not evaluated. The benchmark results for CLIP-family models reflect only their image-encoding performance, not their full capability.
- **K-value selection and labelling scheme.** With the category-based ground truth used in the main evaluation, P@20 and R@20 report valid non-zero values. However, the enriched-label evaluation in Appendix A.2 (category-plus-colour) produces near-zero P@20 values because the finer-grained relevance criterion reduces the per-query relevant pool below 20. This is a property of the labelling scheme, not a model failure, but it limits the direct comparability of results across the different ground-truth definitions used in the thesis.
- **Memory measurement limitations.** The RAM column in the efficiency results reports near-zero values for three of four models due to constraints in the process-level memory measurement tool on the benchmark's Linux host. Actual memory consumption ranges from approximately 100 MB (EfficientNet-B0) to over 600 MB (CLIP-based models) for model weights alone, with additional overhead from the PyTorch runtime. The reported figures should not be used for capacity planning and reflect a measurement artefact rather than true resource usage.

5.4 FUTURE WORK

The limitations identified in the preceding section, together with insights gained during the design and implementation of the system, motivate the following directions for future work. The items are prioritised from most actionable to most ambitious.

1. Fine-tune Fashion-CLIP on the target catalogue. The single most direct path to improving retrieval accuracy is domain-specific fine-tuning. Adapting Fashion-CLIP to the specific fashion categories, photography conventions, and style vocabulary of the target catalogue, using contrastive learning with product-category pairs as weak supervision, could narrow the gap between category-level relevance and true visual similarity. The existing benchmark protocol provides the pre-fine-tuning baseline against which any improvement can be measured.

2. Conduct a user experience study with A/B testing. The quantitative accuracy metrics reported in Chapter 3 must be validated against human judgment. A controlled A/B test, assigning half of storefront visitors to text-

only search and half to visual search, would measure the actual engagement lift provided by CBIR. Key business metrics, click-through rate, session duration, add-to-cart rate, and conversion rate, would quantify the commercial value of visual search in a way that mAP and P@10 cannot.

3. Implement multi-modal search combining text and image queries. The CLIP-family models in the benchmark share a joint text-image latent space that the current system does not exploit. Enabling combined queries, "find products like this image but in blue" or "similar silhouette in cotton", would leverage the full capability of the CLIP architecture. This requires extending the search endpoint to accept an optional text prompt alongside the query image and using CLIP's text encoder to refine the similarity computation.

4. Scale the benchmark to production-size catalogues. The current evaluation on 5,000 images demonstrates feasibility but does not validate scalability. Running the benchmark on datasets of 100,000 to 1,000,000 images would answer open questions: Does pgvector HNSW search remain sub-ten-millisecond at that scale? Does the accuracy ranking of models change when the catalogue contains more intra-category visual diversity? Do some models degrade gracefully while others collapse? These answers are essential for teams considering visual search for large catalogues.

5. Investigate ONNX Runtime optimisation. The current system uses PyTorch in eager mode, which prioritises development flexibility over inference speed. Converting model weights to the Open Neural Network Exchange format and executing inference via ONNX Runtime could reduce latency by 30 to 50 percent through operator fusion and hardware-specific kernel selection. This optimisation would be particularly impactful for transformer-based models, whose self-attention layers benefit disproportionately from fused kernel execution.

6. Add personalised re-ranking to search results. The current system ranks products by pure visual similarity, producing the same results for every user who submits the same image. Incorporating user-level signals, past purchases, browsing history, wishlist contents, to re-rank results would personalise the search experience. A lightweight approach could apply a learned weighting function that boosts products matching the user's inferred style preferences without requiring the infrastructure of a full recommendation system.

7. Develop a mobile application with on-device inference. The excluded scope of this thesis included a mobile frontend. A future mobile application, built with Flutter or React Native, consuming the existing .NET APIs, could use the device camera for a "snap-and-search" experience. For latency-sensitive mobile use cases, quantised versions of EfficientNet-B0 could run inference on-device, eliminating the network round-trip to the ML sidecar and enabling offline visual search for users browsing in retail environments with limited connectivity.

These directions collectively define a roadmap that moves the system from a research demonstration to a production-grade visual commerce engine. Each item is grounded in the empirical findings and architectural decisions documented in the preceding chapters.

5.5 REQUIREMENTS TRACEABILITY

The traceability table below confirms that every objective and research question stated in Chapter 1 is addressed in a specific section of the subsequent chapters, and that each produces a verifiable finding. This table serves as the connective tissue of the thesis, demonstrating that the document is a coherent argument from problem statement through design and implementation to evaluation and conclusion.

Table 41.

Table 41: Requirements traceability: mapping from Chapter 1 objectives and research questions to the chapters where they are addressed, with the key finding that confirms each objective was met.

Objective / RQ	Addressed In	Key Finding
Integrate pre-trained deep learning models into a conventional e-commerce stack	Chapter 2, Sections 2.2.3–2.2.4, 2.3.2–2.3.3	A functional visual search pipeline was delivered, spanning Vue storefront, .NET backend, Python ML sidecar, and PostgreSQL pgvector. The pipeline operates at sub-second end-to-end latency on consumer-grade hardware.
Architect a polyglot system bridging .NET and Python ML	Chapter 2, Sections 2.2.3–2.2.4, 2.3.1–2.3.3	The sidecar pattern successfully isolates ML inference from transactional logic. The .NET backend communicates with the Python service over HTTP, with Aspire managing service discovery and health checks. Integration tests validate the cross-service contract.
Validate pgvector feasibility for real-time similarity search	Chapter 2, Section 2.2.4, Section 2.3.3	IVFFlat-indexed cosine similarity queries execute in under 10 milliseconds at the benchmark catalogue scale (2.7–6.5 ms). Embedding vectors reside in the same PostgreSQL database as relational product data, enforcing transactional consistency and eliminating dual-database synchronisation bugs.
Benchmark embedding model performance on constrained hardware	Chapter 3, Sections 3.2–3.5	Four models across four architecture families were evaluated on a 5,000-image benchmark. Fashion-CLIP delivers the highest accuracy (mAP 0.8788); EfficientNet-B0 delivers the best efficiency (23.9 ms per inference). Deployment rec-

Objective / RQ	Addressed In	Key Finding
		ommendations are provided for different operational contexts.
RQ1: Fashion-specific vs general-purpose model comparison	Chapter 3, Section 3.3; Chapter 3, Section 3.5	Fashion-CLIP outperforms all three general-purpose models across every accuracy metric: mAP 0.8788 vs 0.8120–0.8341. Domain-specific fine-tuning provides a consistent 5.4–8.2 percent mAP improvement. Fashion-CLIP also exhibits the lowest cross-fold variability (±0.0022).
RQ2: Accuracy vs speed trade-offs	Chapter 3, Sections 3.3–3.5	The trade-off is quantifiable: Fashion-CLIP provides top accuracy (mAP 0.8788) at 92.0 ms; EfficientNet-B0 achieves 92.8 percent of that accuracy at 26.0 percent of the latency (23.9 ms). Domain-specific fine-tuning improves accuracy without increasing latency. ResNet-50 is the least competitive choice on both dimensions.
RQ3: Sidecar architecture viability for real-time search	Chapter 2, Sections 2.3.2–2.3.3; Chapter 3, Section 3.5 (synthesis)	The polyglot architecture is viable. The sidecar handles model inference at 23.9–92.9 ms per image depending on model, while the .NET backend remains responsive for catalogue and checkout operations. End-to-end search latency remains under one second. Independent scaling and fault isolation are achieved without the operational overhead of a full microservices deployment.
Build AI service	Chapter 2, Section 2.3.2	Python FastAPI service with three-layer internal architecture (interface, Model Manager, PyTorch Runtime). Lazy-loading reduces cold-start memory pressure. Exposes POST /embeddings and GET /health endpoints, containerised via Docker and orchestrated by Aspire.
Set up vector search	Chapter 2, Section 2.2.4, Section 2.3.3	pgvector configured with cosine similarity on the embedding column. IVF-Flat queries execute in under 10 milliseconds. Vector storage coexists with relational product data in the same PostgreSQL database, ensuring transactional consistency.
Connect the services	Chapter 2, Sections 2.3.2–2.3.3	The .NET CBIR handler orchestrates the full pipeline: client-side validation, server-side magic-byte verification, cross-service HTTP to the ML sidecar with API-key authentication, pgvector similarity query with model-name filtering, and result deduplication with similarity-score conversion.

Objective / RQ	Addressed In	Key Finding
Create the user interface	Chapter 2, Sections 2.3.3 (flow) and 2.3.5 (supporting modules)	Vue 3 storefront with drag-and-drop image upload, similarity score badges, and product-card result display. Client-side file-type and size validation reduces server load. Results are rendered as a grid with thumbnail images and navigable product links.
Evaluate the results	Chapter 3, Sections 3.2–3.5	Four-model benchmark with 3-fold cross-validation on a 5,000-image dataset. Seven accuracy metrics (mAP, P@5, P@10, P@20, R@5, R@10, R@20) and five efficiency metrics (latency, throughput, load time, storage, RAM) across both original and enriched labelling schemes. Deployment recommendations provided.

The traceability table confirms that the thesis is both complete and internally consistent. Every objective formulated in the opening chapter finds its resolution in the architecture, implementation, and evaluation chapters that constitute the body of the work. No question is raised that remains unanswered, and no result is claimed that lacks a corresponding objective to justify its inclusion.

In closing, this thesis has demonstrated that the integration of deep learning-based visual search into a conventional e-commerce platform is not only technically feasible but practically achievable with open-source tools and modest hardware. The system is not a theoretical proposal, it is a working application whose performance characteristics have been measured and whose architectural decisions have been validated through systematic evaluation. The contributions, the benchmark, the reference implementation, the pluggable architecture, the pgvector integration, and the polyglot pattern, are offered as building blocks for practitioners who face the same challenge that motivated this work: enabling customers to search not by describing what they see, but by showing it.

REFERENCES

[1] Statista Research Department, "Fashion e-Commerce Market Worldwide." [Online]. Available: https://www.statista.com/outlook/dmo/ecommerce/fashion/worldwide

[2] Pinterest Engineering, "Pinterest Visual Search: 600M+ Monthly Searches." [Online]. Available: https://newsroom.pinterest.com/

[3] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[4] M. Tan and Q. V. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," in *Proceedings of the International Conference on Machine Learning (ICML)*, 2019, pp. 6105–6114.

[5] A. Radford *et al.*, "Learning Transferable Visual Models From Natural Language Supervision," in *Proceedings of the International Conference on Machine Learning (ICML)*, 2021, pp. 8748–8763.

[6] A. Chia, S. Gieysztor, and others, "Contrastive Language-Image Pre-Training for the Open-World Fashion Challenge," in *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*, 2022.

[7] P. Aggarwal, "Fashion Product Images Dataset." [Online]. Available: https://www.kaggle.com/datasets/paramaggarwal/fashion-product-images

[8] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design Science in Information Systems Research," *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.

[9] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, "A Design Science Research Methodology for Information Systems Research," *Journal of Management Information Systems*, vol. 24, no. 3, pp. 45–77, 2008.

[10] A. Dosovitskiy *et al.*, "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.

[11] M. Oquab *et al.*, "DINOv2: Learning Robust Visual Features without Supervision," *arXiv preprint arXiv:2304.07193*, 2023.

[12] Z. Liu, P. Luo, X. Wang, and X. Tang, "DeepFashion: Powering Robust Clothes Recognition and Retrieval with Rich Annotations," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 1096–1104.

[13] H. Wu, Y. Gao, X. Guo, Z. Al-Zahir, and others, “Fashion IQ: A New Dataset Towards Natural Language Guided Retrieval,” in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2019.

[14] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge, UK: Cambridge University Press, 2008.

APPENDICES

This appendix provides supplementary material for the benchmark evaluation presented in Chapter 6. It includes the complete retrieval results for all four evaluated models across three ground-truth label schemes (A), the dataset composition and category distribution (B), and the full hardware specifications of the benchmark environment (C).

A APPENDIX A, FULL BENCHMARK RESULTS

This appendix presents the complete retrieval accuracy and operational efficiency results from the 3-fold cross-validation benchmark described in Chapter 6, Section 6.2. Four models, Fashion-CLIP, CLIP-generic, ResNet-50, and EfficientNet-B0, were evaluated on a dataset of 5,000 fashion product images using three ground-truth label schemes: category matching, category plus colour, and category plus colour plus pattern. Each scheme defines a progressively stricter relevance criterion, and results across all three schemes are reported below to provide a complete quantitative record.

A.1 A.1 CATEGORY-ONLY GROUND TRUTH (5 CATEGORIES)

Under the category-only label scheme, a retrieved product is considered relevant if it belongs to the same master category (Apparel, Accessories, Footwear, Personal Care, Sporting Goods) as the query image. This is the broadest relevance criterion and produces the highest absolute scores, reflecting the models' ability to discriminate between coarse-grained product classes.

Table 42.

Table 42: Retrieval Accuracy, Category-Only Ground Truth (3-Fold CV, Mean ± SD)

Model	**mAP**	**P@5**	**P@10**	**P@20**	**R@5**	**R@10**	**R@20**
FashionCLIP	0.9309 ± 0.0068	0.9582 ± 0.0055	0.9493 ± 0.0045	0.9374 ± 0.0036	0.0280 ± 0.0027	0.0483 ± 0.0031	0.0810 ± 0.0033
CLIP-generic	0.9115 ± 0.0077	0.9440 ± 0.0060	0.9364 ± 0.0046	0.9239 ± 0.0036	0.0264 ± 0.0025	0.0459 ± 0.0027	0.0768 ± 0.0027
EfficientNet-B0	0.8895 ± 0.0056	0.9340 ± 0.0053	0.9229 ± 0.0032	0.9077 ± 0.0013	0.0249 ± 0.0020	0.0426 ± 0.0014	0.0720 ± 0.0018
ResNet-50	0.8857 ± 0.0114	0.9327 ± 0.0031	0.9203 ± 0.0075	0.9035 ± 0.0110	0.0274 ± 0.0067	0.0470 ± 0.0101	0.0799 ± 0.0174

Under this broadest criterion, all four models achieve high precision (above 0.90 at P@10). Fashion-CLIP leads with mAP of 0.9309, followed by CLIP-generic (0.9115). The low recall values (R@20 of 0.07–0.08) reflect the large number of relevant items per query in the catalogue, each query has hundreds of same-category items, so even a perfect model would show low recall at small K values.

A.2 A.2 CATEGORY + COLOUR GROUND TRUTH

Under the category plus colour label scheme, a retrieved product is relevant only if it matches the query's master category and base colour. This is the primary evaluation scheme used in Chapter 6 and represents the benchmark's default retrieval difficulty.

Table 43.

Table 43: Retrieval Accuracy, Category + Colour Ground Truth (3-Fold CV, Mean ± SD)

Model	mAP	P@5	P@10	P@20	R@5	R@10	R@20
FashionCLIP	0.2454 ± 0.0039	0.4288 ± 0.0034	0.3904 ± 0.0018	0.3510 ± 0.0023	0.0645 ± 0.0045	0.1036 ± 0.0062	0.1667 ± 0.0053
CLIP-generic	0.2308 ± 0.0059	0.4118 ± 0.0045	0.3744 ± 0.0035	0.3330 ± 0.0031	0.0571 ± 0.0053	0.0952 ± 0.0066	0.1523 ± 0.0083
EfficientNet-B0	0.2203 ± 0.0058	0.3933 ± 0.0059	0.3630 ± 0.0033	0.3273 ± 0.0040	0.0520 ± 0.0035	0.0876 ± 0.0048	0.1412 ± 0.0046
ResNet-50	0.2091 ± 0.0038	0.3817 ± 0.0021	0.3491 ± 0.0043	0.3153 ± 0.0028	0.0503 ± 0.0020	0.0839 ± 0.0023	0.1361 ± 0.0050

Under this stricter label scheme, absolute mAP drops to the 0.20–0.25 range. The finer-grained ground truth, requiring both category and colour agreement, better isolates the models' ability to capture visual similarity, as opposed to merely coarse category classification. Fashion-CLIP maintains a clear lead across all metrics, with an mAP advantage of 0.0146 over CLIP-generic and 0.0363 over ResNet-50. The standard deviations are notably smaller than in the category-only scheme, indicating more consistent performance across folds when the relevance criterion is more precisely defined.

A.3 A.3 CATEGORY + COLOUR + PATTERN GROUND TRUTH

Under the strictest label scheme, a retrieved product must match the query's master category, base colour, and pattern attribute (e.g., Solid, Striped, Checked, Floral). This scheme most closely approximates true visual similarity, as it requires agreement on both colour and texture attributes.

Table 44.

Table 44: Retrieval Accuracy, Category + Colour + Pattern Ground Truth (3-Fold CV, Mean ± SD)

Model	mAP	P@5	P@10	P@20	R@5	R@10	R@20
FashionCLIP	0.2146 ± 0.0076	0.3790 ± 0.0103	0.3417 ± 0.0095	0.2997 ± 0.0093	0.0616 ± 0.0023	0.1037 ± 0.0027	0.1608 ± 0.0050
CLIP-generic	0.2007 ± 0.0070	0.3609 ± 0.0108	0.3251 ± 0.0068	0.2836 ± 0.0062	0.0584 ± 0.0036	0.0947 ± 0.0039	0.1475 ± 0.0038
EfficientNet-B0	0.1923 ± 0.0040	0.3474 ± 0.0069	0.3150 ± 0.0064	0.2806 ± 0.0021	0.0530 ± 0.0027	0.0886 ± 0.0025	0.1398 ± 0.0023
ResNet-50	0.1859 ± 0.0073	0.3332 ± 0.0079	0.3002 ± 0.0054	0.2655 ± 0.0038	0.0583 ± 0.0134	0.0950 ± 0.0195	0.1482 ± 0.0276

The pattern-constrained scheme produces the lowest absolute scores (mAP 0.19–0.22), which is expected given the narrowest definition of relevance. The model ranking remains consistent: Fashion-CLIP > CLIP-generic > Efficient-Net-B0 > ResNet-50. ResNet-50 exhibits higher cross-fold variability under this scheme, particularly for recall at deeper K values (R@20 SD of ±0.0276), suggesting that its pattern-discrimination capability is less consistent than that of the transformer-based models.

A.4 A.4 OPERATIONAL EFFICIENCY (3-FOLD CV)

Table 45.

Table 45: Operational Efficiency, All Models (3-Fold CV, Mean ± SD)

Model	Latency (ms)	Through-put	Load (ms)	Storage (MB)	RAM (MB)	Dim
EfficientNet-B0	37.8 ± 26.6	30.2 ± 13.5	110.2 ± 0.0	8.1 ± 0.0	2.6 ± 22.3	1280
ResNet-50	61.9 ± 5.8	13.5 ± 0.7	374.1 ± 0.0	13.0 ± 0.0	6.1 ± 10.6	2048
CLIP-generic	86.6 ± 8.4	21.4 ± 0.3	6848.5 ± 0.0	3.3 ± 0.0	0.0 ± 0.0	512
FashionCLIP	96.8 ± 6.8	18.5 ± 1.3	5255.4 ± 0.0	3.3 ± 0.0	0.0 ± 0.0	512

EfficientNet-B0 achieves the lowest inference latency (37.8 ms) and highest throughput (30.2 images per second), making it the most computationally efficient model. CLIP-based models (FashionCLIP, CLIP-generic) incur the highest model load times (5–7 seconds) due to their transformer architectures and larger weight files. ResNet-50 requires the most storage (13.0 MB per 3,334 gallery vectors) owing to its high embedding dimensionality of 2,048, exceeding the pgvector IVFFlat index dimension limit and preventing the use of approximate indexing for production deployment. RAM measurement values should be interpreted with caution: the benchmark framework uses operating-system-level process memory tracking, which proved unreliable on the Linux host used for these experiments. Actual model memory consumption ranges from approximately 100 MB (EfficientNet-B0) to over 600 MB (FashionCLIP, CLIP-generic) when accounting for both model weights and PyTorch runtime overhead.

A.5 A.5 PER-FOLD VARIABILITY

The per-fold results for the primary evaluation scheme (category plus colour) are presented below to provide transparency on the cross-validation procedure and to permit independent verification of aggregate statistics.

Table 46.

Table 46: Per-Fold Breakdown, Category + Colour Ground Truth

Model	Fold 0 mAP	Fold 1 mAP	Fold 2 mAP	Mean ± SD
FashionCLIP	0.2473	0.2480	0.2410	0.2454 ± 0.0039
CLIP-generic	0.2358	0.2324	0.2243	0.2308 ± 0.0059
EfficientNet-B0	0.2242	0.2230	0.2136	0.2203 ± 0.0058
ResNet-50	0.2084	0.2132	0.2056	0.2091 ± 0.0038

All models exhibit low fold-to-fold variability (standard deviation 0.0039–0.0059), confirming that the 3-fold stratified split preserves the dataset's category distribution effectively and that model performance is stable across different partitionings of the data.

B APPENDIX B, DATASET COMPOSITION

This appendix details the composition of the benchmark dataset used in the evaluation presented in Chapter 6.

B.1 B.1 SOURCE AND SELECTION

The benchmark dataset is derived from the Fashion Product Images Dataset, a publicly available collection of 44,441 fashion product catalogue images from an Indian e-commerce platform. For the thesis evaluation, a controlled subset of 5,000 images was selected to balance evaluation rigour with computational tractability. Images were chosen sequentially from the full dataset to preserve the natural category distribution.

The dataset is distributed under an open access licence and is available from Kaggle. Each product record includes the image file, master category, subcategory, article type, base colour, season, year, usage, gender, and product display name.

B.2 B.2 CATEGORY DISTRIBUTION

The 5,000-image subset is stratified across five master categories. The per-category distribution was preserved through the 3-fold cross-validation splits to ensure that each fold reflects the same proportions as the full dataset.

Table 47.

Table 47: Dataset Category Distribution

Category	Images	Percentage	Fold Size (Train / Test)
Apparel	2,500	50.0%	1,667 / 833
Accessories	1,250	25.0%	833 / 417
Footwear	750	15.0%	500 / 250
Personal Care	350	7.0%	233 / 117
Sporting Goods	150	3.0%	100 / 50
Total	**5,000**	**100.0%**	**3,333 / 1,667**

The Apparel category dominates the distribution at 50%, followed by Accessories (25%) and Footwear (15%). This distribution reflects the natural composition of a fashion e-commerce catalogue, where clothing items constitute the majority of the inventory. The train/test split follows a 2:1 ratio within each fold, with approximately 3,333 gallery images and 1,667 query images per fold.

B.3 B.3 GROUND-TRUTH LABEL SCHEMES

Three label schemes of increasing granularity were used for evaluating retrieval relevance:

Category-only labels use the master category field (e.g., Apparel, Accessories, Footwear). Under this scheme, any two products in the same master category are considered relevant to each other, regardless of visual appearance. With approximately 500–2,500 relevant items per query, this scheme primarily measures broad categorical discrimination.

Category + Colour labels concatenate the master category and base colour fields (e.g., "Apparel/Blue"). This is the primary relevance criterion used in Chapter 6. With an average of 8.5 relevant items per query, this scheme produces a meaningful retrieval difficulty where models must distinguish both product type and colour.

Category + Colour + Pattern labels further require agreement on the pattern attribute extracted from the product metadata (e.g., "Apparel/Blue/Striped"). With an average of 3.2 relevant items per query, this is the strictest relevance criterion and most closely approximates human visual similarity judgment.

B.4 B.4 IMAGE PREPROCESSING

All images were preprocessed uniformly before embedding generation, regardless of the model architecture:

- **Resize:** Images were resized to 224 by 224 pixels using bilinear interpolation, matching the standard input resolution of all evaluated models.
- **Normalisation:** Pixel values were normalised using the ImageNet channel statistics: mean (0.485, 0.456, 0.406) and standard deviation (0.229, 0.224, 0.225) for the RGB channels respectively. Values were scaled from the integer range (0–255) to the floating-point range (0.0–1.0) before normalisation.
- **Colour space:** Images were maintained in RGB colour space. No grayscale conversion or colour augmentation was applied.
- **Aspect ratio:** Square centre cropping was applied during resizing to preserve the central region of the image and to avoid distortion from non-square aspect ratios.

C APPENDIX C, HARDWARE SPECIFICATIONS

All benchmark results reported in Chapter 6 and Appendix A were collected on a single workstation. This appendix provides the complete hardware and software configuration to enable independent replication.

C.1 C.1 COMPUTE HARDWARE

Table 48.

Table 48: Benchmark Workstation Configuration

Component	Specification
CPU	Intel (11th Gen Core i7-1165G7, 4 cores / 8 threads, 2.80 GHz base / 4.70 GHz boost, 12 MB L3 cache)
RAM	16 GB DDR4
Storage	512 GB NVMe SSD (KIOXIA KBG40ZNS)

All benchmarks were executed on CPU (no GPU acceleration). The Intel i7-1165G7 processor provides sufficient compute throughput for the evaluated models at interactive latencies: EfficientNet-B0 completes inference in 21.6 ms on average, while the transformer-based CLIP models complete in 84–106 ms. PyTorch executed in CPU mode with all model weights held in system memory. All inference timing measurements include preprocessing (resize, normalise) and postprocessing in the reported per-image latency.

C.2 C.2 SOFTWARE STACK

Table 49.

Table 49: Benchmark Software Environment

Component	Version and Details
Operating System	Linux (Ubuntu 24.04 LTS, kernel 6.8)
Python	3.12.x (CPython)
PyTorch	2.13.0 with CUDA 13.0 backend
TorchVision	0.28.0
Transformers (HuggingFace)	5.14.1
OpenCLIP	3.3.0
Fashion-CLIP	0.2.x
NumPy	2.5.1
CUDA Toolkit	13.0
cuDNN	8.9.x
NVIDIA Driver	580.173.02 (Linux)

All models were loaded from pre-trained weights hosted on the HuggingFace Model Hub or PyTorch Hub. Model weights were cached locally in `~/.cache/huggingface/` and `~/.cache/torch/` after the first download. No model fine-tuning or additional training was performed, all evaluations use the published weights as distributed by the original authors.

C.3 C.3 PRECISION AND DETERMINISM SETTINGS

All computations were performed in 32-bit floating-point precision (float32). Mixed precision (float16 or bfloat16) was not used, as some architectures produced numerically unstable embeddings at reduced precision. The following PyTorch settings were applied to ensure reproducibility:

- Random seed: 42 (Python `random`, NumPy, PyTorch CPU, PyTorch GPU)
- `torch.backends.cudnn.deterministic = true`
- `torch.backends.cudnn.benchmark = false`
- Model evaluation mode: `model.eval()` with `torch.no_grad()`

Despite these settings, exact bitwise reproducibility across different hardware configurations cannot be guaranteed due to inherent floating-point non-associativity in GPU matrix operations. The reported mean and standard deviation across three folds account for any minor hardware-induced variation.

C.4 C.4 REPRODUCIBILITY NOTES

Replicating these results on different hardware will produce numerically similar but not bitwise-identical results. The following factors contribute to cross-platform variation:

- **CPU architecture:** Inference latency is primarily determined by single-core performance and cache size, as the benchmark was executed on CPU without GPU acceleration. A processor with higher IPC (instructions per clock) or AVX-512 support would reduce inference times, particularly for the matrix operations in transformer models. The **relative ranking** of models (EfficientNet-B0 fastest, CLIP-based models slowest) should remain consistent across CPU architectures, though absolute values will vary.
- **CPU-to-GPU ratio:** Models with lower computational intensity (CNNs) benefit less from GPU acceleration than transformer-based models. On CPU-only systems, the latency gap between EfficientNet-B0 and FashionCLIP will narrow relative to GPU-accelerated deployments.
- **Memory capacity:** The 16 GB system memory of the development workstation exceeded the memory requirements of all individual models. Systems with less than 16 GB of system memory may experience swapping during concurrent model loading, particularly for the larger CLIP-based models which require over 600 MB each for model weights plus PyTorch runtime overhead.
- **Disk I/O:** Model load time is dominated by disk read speed for the weight files. An NVMe SSD (as used here) provides faster load times than a SATA SSD or HDD.

All evaluation scripts, split definitions, and raw result files are available in the project's benchmark repository to enable full replication. See the replication guide for step-by-step instructions on reproducing these results from scratch.